



US012401822B2

(12) **United States Patent**
Kim et al.

(10) **Patent No.:** **US 12,401,822 B2**
(45) **Date of Patent:** ***Aug. 26, 2025**

(54) **BIT STREAM STRUCTURE FOR COMPRESSED POINT CLOUD DATA**

(71) Applicant: **Apple Inc.**, Cupertino, CA (US)

(72) Inventors: **Jungsun Kim**, Sunnyvale, CA (US);
Alexandros Tourapis, Los Gatos, CA (US); **Khaled Mammou**, Danville, CA (US); **Fabrice A. Robinet**, San Jose, CA (US); **Yeping Su**, Palo Alto, CA (US); **David W. Singer**, San Francisco, CA (US)

(73) Assignee: **Apple Inc.**, Cupertino, CA (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 25 days.

This patent is subject to a terminal disclaimer.

(21) Appl. No.: **18/189,099**

(22) Filed: **Mar. 23, 2023**

(65) **Prior Publication Data**

US 2023/0232044 A1 Jul. 20, 2023

Related U.S. Application Data

(63) Continuation of application No. 17/319,019, filed on May 12, 2021, now Pat. No. 11,647,226, which is a (Continued)

(51) **Int. Cl.**

H04N 19/597 (2014.01)

H04N 19/17 (2014.01)

G06T 9/00 (2006.01)

(52) **U.S. Cl.**

CPC **H04N 19/597** (2014.11); **H04N 19/17** (2014.11); **G06T 9/00** (2013.01)

(58) **Field of Classification Search**

CPC H04N 19/597; H04N 19/17; G06T 9/00

See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

1,145,003 A 7/1915 DeLong
5,793,371 A 8/1998 Deering
(Continued)

FOREIGN PATENT DOCUMENTS

CA 3096184 10/2019
CN 101198945 6/2008
(Continued)

OTHER PUBLICATIONS

U.S. Appl. No. 18/052,803, filed Nov. 4, 2022, Mammou, et al.
(Continued)

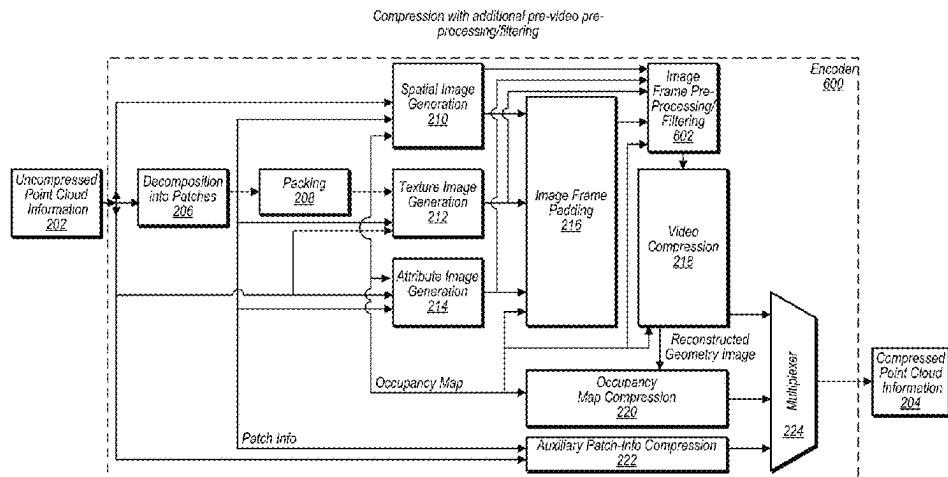
Primary Examiner — Zhubing Ren

(74) *Attorney, Agent, or Firm* — Alexander A. Knapp;
Kowert, Hood, Munyon, Rankin & Goetzel, P.C.

(57) **ABSTRACT**

A system comprises an encoder configured to compress attribute information and/or spatial information for a point cloud and/or a decoder configured to decompress compressed attribute and/or spatial information for the point cloud. To compress the attribute and/or spatial information, the encoder is configured to convert a point cloud into an image based representation. Also, the decoder is configured to generate a decompressed point cloud based on an image based representation of a point cloud. In some embodiments, a bit stream structure may be used to communicate compressed point cloud data. The bit stream structure may include point cloud compression network abstraction layer (PCCNAL) units that enable use of groups of frames (GOFs), frame, and sub-frame signaling of patch information. Such a bit stream structure may permit low delay streaming and random access reconstruction of point clouds amongst other applications.

20 Claims, 44 Drawing Sheets



Related U.S. Application Data

continuation of application No. 16/510,706, filed on Jul. 12, 2019, now Pat. No. 11,012,713.

- (60) Provisional application No. 62/697,369, filed on Jul. 12, 2018.

References Cited

U.S. PATENT DOCUMENTS

5,842,004	A	11/1998	Deering	9,734,595	B2	8/2017	Lukac
5,867,167	A	2/1999	Deering	9,753,124	B2	9/2017	Hayes
5,870,094	A	2/1999	Deering	9,787,321	B1	10/2017	Hemmer et al.
5,905,502	A	5/1999	Deering	9,800,766	B2	10/2017	Tsuji
5,933,153	A	8/1999	Deering	9,836,483	B1	12/2017	Hickman
6,018,353	A	1/2000	Deering	9,972,129	B2	5/2018	Michel et al.
6,028,610	A	2/2000	Deering	10,089,312	B2	10/2018	Tremblay et al.
6,088,034	A	7/2000	Deering	10,108,867	B1	10/2018	Vallespi-Gonzalez
6,188,796	B1	2/2001	Kadono	10,223,810	B2	3/2019	Chou et al.
6,215,500	B1	4/2001	Deering	10,259,164	B2	4/2019	Bader
6,239,805	B1	5/2001	Deering	10,277,248	B2	4/2019	Lee
6,256,041	B1	7/2001	Deering	10,372,728	B2	8/2019	Horhammer et al.
6,307,557	B1	10/2001	Deering	10,395,419	B1	8/2019	Godzaridis
6,429,867	B1	8/2002	Deering	10,462,485	B2	10/2019	Mammou et al.
6,459,428	B1	10/2002	Burk et al.	10,467,756	B2	11/2019	Karlinsky et al.
6,459,429	B1	10/2002	Deering	10,510,148	B2	12/2019	Qui
6,476,803	B1	11/2002	Zhang	10,559,111	B2	2/2020	Sachs
6,522,326	B1	2/2003	Deering	10,587,286	B1	3/2020	Flynn
6,522,327	B2	2/2003	Deering	10,607,373	B2	3/2020	Mammou et al.
6,525,722	B1	2/2003	Deering	10,659,816	B2	5/2020	Mammou et al.
6,525,725	B1	2/2003	Deering	10,699,444	B2	6/2020	Mammou et al.
6,531,012	B2	3/2003	Ishiyama	10,715,618	B2	7/2020	Bhaskar
6,559,842	B1	5/2003	Deering	10,762,667	B2	9/2020	Mekuria
6,603,470	B1	8/2003	Deering	10,783,668	B2	9/2020	Sinharoy et al.
6,628,277	B1	9/2003	Deering	10,789,733	B2	9/2020	Mammou et al.
6,747,644	B1	6/2004	Deering	10,805,646	B2	10/2020	Tourapis et al.
6,858,826	B2	2/2005	Mueller et al.	10,853,973	B2	12/2020	Tourapis et al.
7,071,935	B1	7/2006	Deering	10,861,196	B2	12/2020	Mammou et al.
7,110,617	B2	9/2006	Zhang et al.	10,867,413	B2	12/2020	Mammou et al.
7,215,810	B2	5/2007	Kaufman et al.	10,867,414	B2	12/2020	Mammou et al.
7,373,473	B2	5/2008	Bukowski et al.	10,869,059	B2	12/2020	Mammou et al.
7,737,985	B2	6/2010	Torzewski et al.	10,897,269	B2	1/2021	Mammou et al.
7,961,934	B2	6/2011	Thrun et al.	10,909,726	B2	2/2021	Mammou et al.
8,022,951	B2	9/2011	Zhirkov et al.	10,909,727	B2	2/2021	Mammou et al.
8,040,355	B2	10/2011	Burley	10,911,787	B2	2/2021	Tourapis et al.
8,055,070	B2	11/2011	Bassi et al.	10,939,123	B2	3/2021	Li
8,315,425	B2	11/2012	Appel	10,939,129	B2	3/2021	Mammou
8,520,740	B2	8/2013	Flachs	10,977,773	B2	4/2021	Hemmer
8,566,736	B1	10/2013	Jacob	11,010,907	B1	5/2021	Bagwell
8,643,515	B2	2/2014	Cideciyan	11,010,928	B2	5/2021	Mammou et al.
8,718,405	B2	5/2014	Fujiki	11,012,713	B2	5/2021	Kim et al.
8,780,112	B2	7/2014	Kontkanen et al.	11,017,566	B1	5/2021	Tourapis et al.
8,805,097	B2	8/2014	Ahn et al.	11,044,478	B2	6/2021	Tourapis et al.
8,884,953	B2	11/2014	Chen et al.	11,044,495	B1	6/2021	Dupont
8,996,228	B1	3/2015	Ferguson	11,113,845	B2	9/2021	Tourapis et al.
9,064,311	B2	6/2015	Mammou et al.	11,132,818	B2	9/2021	Mammou et al.
9,064,331	B2	6/2015	Yamashita	11,202,078	B2	12/2021	Tourapis et al.
9,117,105	B2	8/2015	Da	11,202,098	B2	12/2021	Mammou et al.
9,171,383	B2	10/2015	Ahn et al.	11,212,558	B2	12/2021	Sugio
9,191,670	B2	11/2015	Karczewicz	11,252,441	B2	2/2022	Tourapis et al.
9,199,641	B2	12/2015	Ferguson et al.	11,276,203	B2	3/2022	Tourapis et al.
9,214,042	B2	12/2015	Cai	11,282,238	B2	3/2022	Mammou et al.
9,223,765	B1	12/2015	Alakuijala	11,284,091	B2	3/2022	Tourapis et al.
9,234,618	B1	1/2016	Zhu et al.	11,348,284	B2	5/2022	Tourapis et al.
9,256,980	B2	2/2016	Kirk	11,363,309	B2	6/2022	Tourapis et al.
9,292,961	B1	3/2016	Korchev	11,386,524	B2	7/2022	Mammou et al.
9,300,321	B2	3/2016	Zalik et al.	11,398,833	B2	7/2022	Flynn et al.
9,317,965	B2	4/2016	Krishnaswamy et al.	11,409,998	B2	8/2022	Mammou et al.
9,412,040	B2	8/2016	Feng	11,430,155	B2	8/2022	Tourapis et al.
9,424,672	B2	8/2016	Zavodny	11,450,031	B2	9/2022	Flynn
9,430,837	B2	8/2016	Fujiki	11,461,935	B2	10/2022	Mammou et al.
9,530,225	B1	12/2016	Nieves	11,475,605	B2	10/2022	Flynn
9,532,056	B2	12/2016	Jiang et al.	11,494,947	B2	11/2022	Mammou et al.
9,613,388	B2	4/2017	Loss	11,503,367	B2	11/2022	Yea
9,633,146	B2	4/2017	Plummer et al.	11,508,095	B2	11/2022	Mammou et al.
9,678,963	B2	6/2017	Hernandez Londono et al.	11,527,018	B2	12/2022	Mammou et al.
9,729,169	B2	8/2017	Kalevo	11,552,651	B2	1/2023	Mammou et al.
				11,611,132	B2	3/2023	Kim et al.
				11,615,557	B2	3/2023	Flynn
				11,620,768	B2	4/2023	Flynn
				11,711,544	B2	7/2023	Tourapis et al.
				11,727,603	B2	8/2023	Mammou et al.
				11,748,916	B2	9/2023	Mammou et al.
				11,798,196	B2	10/2023	Kim et al.
				11,818,401	B2	11/2023	Tourapis et al.
				11,895,307	B2	2/2024	Mammou et al.
				2002/0181741	A1	12/2002	Masukura
				2003/0066949	A1	4/2003	Mueller
				2003/0108099	A1	6/2003	Nagumo

(56)	References Cited			2017/0337724	A1	11/2017	Gervais	
	U.S. PATENT DOCUMENTS			2017/0347100	A1	11/2017	Chou	
				2017/0347120	A1	11/2017	Chou et al.	
				2017/0347122	A1	11/2017	Chou et al.	
2004/0217956	A1	11/2004	Besl et al.	2017/0358063	A1	12/2017	Chen	
2006/0133508	A1	6/2006	Sekiguchi	2018/0018786	A1	1/2018	Jakubiak	
2006/0245628	A1	11/2006	Jeung	2018/0053324	A1	2/2018	Cohen et al.	
2007/0025624	A1	2/2007	Baumberg	2018/0063543	A1	3/2018	Reddy	
2007/0098283	A1	5/2007	Kim	2018/0075622	A1	3/2018	Tuffreau	
2007/0160140	A1	7/2007	Fujisawa	2018/0189982	A1	7/2018	Laroche	
2008/0050047	A1	2/2008	Bashyam	2018/0192061	A1	7/2018	He	
2008/0154928	A1	6/2008	Bashyam	2018/0253867	A1	9/2018	Laroche	
2008/0225116	A1	9/2008	Kang	2018/0260416	A1	9/2018	Elkaim	
2009/0016598	A1	1/2009	Lojewski	2018/0268570	A1*	9/2018	Budagavi	G06T 9/001
2009/0027412	A1	1/2009	Burley	2018/0308249	A1	10/2018	Nash	
2009/0087111	A1	4/2009	Noda	2018/0321375	A1	11/2018	Urzhumov	
2009/0213143	A1	8/2009	Igarashi	2018/0330504	A1	11/2018	Karlinsky	
2009/0243921	A1	10/2009	Gebben et al.	2018/0338017	A1	11/2018	Mekuria	
2009/0285301	A1	11/2009	Kurata	2018/0342083	A1	11/2018	Onno	
2010/0104157	A1	4/2010	Doyle	2018/0365898	A1	12/2018	Costa	
2010/0104158	A1	4/2010	Shechtman et al.	2019/0018730	A1	1/2019	Charamisinau et al.	
2010/0106770	A1	4/2010	Taylor	2019/0020880	A1	1/2019	Wang	
2010/0166064	A1	7/2010	Perlman	2019/0026956	A1	1/2019	Gausebeck	
2010/0208807	A1	8/2010	Sikora	2019/0041481	A1	2/2019	Subbarao	
2010/0260429	A1	10/2010	Ichinose	2019/0045157	A1	2/2019	Venshtain	
2010/0260729	A1	10/2010	Cavato	2019/0080483	A1	3/2019	Mammou	
2010/0296579	A1	11/2010	Panchal et al.	2019/0081638	A1	3/2019	Mammou et al.	
2011/0010400	A1	1/2011	Hayes	2019/0087978	A1	3/2019	Tourapis et al.	
2011/0107720	A1	5/2011	Oakey	2019/0087979	A1	3/2019	Mammou et al.	
2011/0142139	A1	6/2011	Cheng	2019/0088004	A1	3/2019	Lucas et al.	
2011/0182477	A1	7/2011	Tamrakar	2019/0089987	A1	3/2019	Won	
2012/0124113	A1	5/2012	Zalik et al.	2019/0108655	A1	4/2019	Lasserre	
2012/0188344	A1	7/2012	Imai	2019/0114504	A1	4/2019	Vosoughi	
2012/0246166	A1	9/2012	Krishnaswamy	2019/0114809	A1	4/2019	Vosoughi	
2012/0300839	A1	11/2012	Sze et al.	2019/0114830	A1	4/2019	Bouazizi	
2012/0314026	A1	12/2012	Chen	2019/0116257	A1	4/2019	Rhyne	
2013/0034150	A1	2/2013	Sadafale	2019/0116357	A1	4/2019	Tian et al.	
2013/0094777	A1	4/2013	Nomura	2019/0122393	A1*	4/2019	Sinharoy	G06T 5/77
2013/0106627	A1	5/2013	Cideciyan	2019/0139266	A1	5/2019	Budagavi et al.	
2013/0156101	A1	6/2013	Lu	2019/0156519	A1	5/2019	Mammou et al.	
2013/0195352	A1	8/2013	Nystad	2019/0156520	A1	5/2019	Mammou et al.	
2013/0202197	A1	8/2013	Reeler	2019/0195616	A1	6/2019	Cao	
2013/0321418	A1	12/2013	Kirk	2019/0197739	A1	6/2019	Sinharoy et al.	
2013/0322738	A1	12/2013	Oh	2019/0199995	A1	6/2019	Yip	
2013/0329778	A1	12/2013	Su	2019/0204076	A1	7/2019	Nishi	
2014/0036033	A1	2/2014	Takahashi	2019/0208200	A1	7/2019	Galpin	
2014/0098855	A1	4/2014	Tak	2019/0262726	A1	8/2019	Spencer et al.	
2014/0125671	A1	5/2014	Vorobyov et al.	2019/0289306	A1	9/2019	Zhao	
2014/0176672	A1	6/2014	Lu	2019/0304139	A1	10/2019	Joshi et al.	
2014/0198097	A1	7/2014	Evans	2019/0311502	A1	10/2019	Mammou et al.	
2014/0204088	A1	7/2014	Kirk et al.	2019/0313110	A1	10/2019	Mammou et al.	
2014/0294088	A1	10/2014	Sung	2019/0318488	A1*	10/2019	Lim	H04N 21/85406
2014/0334557	A1	11/2014	Schierl et al.	2019/0318519	A1	10/2019	Graziosi	
2014/0334717	A1	11/2014	Jiang	2019/0340306	A1	11/2019	Harrison	
2015/0003723	A1	1/2015	Huang et al.	2019/0341930	A1	11/2019	Pavlovic	
2015/0092834	A1	4/2015	Cote et al.	2019/0371051	A1	12/2019	Dore et al.	
2015/0139560	A1	5/2015	Deweert	2019/0373241	A1*	12/2019	Boyce	H04N 19/436
2015/0160450	A1	6/2015	Ou et al.	2019/0392651	A1	12/2019	Graziosi	
2015/0186744	A1	7/2015	Nguyen et al.	2020/0005518	A1	1/2020	Graziosi	
2015/0268058	A1	9/2015	Samarasekera	2020/0013235	A1	1/2020	Tsai	
2016/0035081	A1	2/2016	Stout et al.	2020/0020132	A1	1/2020	Sinharoy et al.	
2016/0071312	A1	3/2016	Laine et al.	2020/0020133	A1	1/2020	Najaf-Zadeh et al.	
2016/0086353	A1	3/2016	Lukac et al.	2020/0021847	A1	1/2020	Kim et al.	
2016/0100151	A1	4/2016	Schaffer	2020/0027248	A1	1/2020	Verschaeve	
2016/0142697	A1	5/2016	Budagavi	2020/0043220	A1	2/2020	Mishaev	
2016/0165241	A1	6/2016	Park	2020/0045344	A1	2/2020	Boyce et al.	
2016/0286215	A1	9/2016	Gamei et al.	2020/0105024	A1	4/2020	Mammou et al.	
2016/0295219	A1	10/2016	Ye et al.	2020/0107022	A1	4/2020	Ahn	
2017/0034520	A1	2/2017	Rosewarne	2020/0107048	A1	4/2020	Yea	
2017/0039765	A1	2/2017	Zhou et al.	2020/0137399	A1	4/2020	Li et al.	
2017/0063392	A1	3/2017	Kalevo	2020/0151913	A1	5/2020	Budagavi	
2017/0118675	A1	4/2017	Boch	2020/0153885	A1	5/2020	Lee	
2017/0155402	A1	6/2017	Karkkainen	2020/0154137	A1*	5/2020	Fleureau	H04N 19/136
2017/0214943	A1	7/2017	Cohen et al.	2020/0195946	A1	6/2020	Choi	
2017/0220037	A1	8/2017	Berestov	2020/0204808	A1	6/2020	Graziosi	
2017/0243405	A1	8/2017	Brandt	2020/0217937	A1	7/2020	Mammou et al.	
2017/0247120	A1	8/2017	Miller	2020/0219285	A1	7/2020	Faramarzi et al.	
2017/0249401	A1	8/2017	Eckart et al.	2020/0219288	A1	7/2020	Joshi	
2017/0323617	A1	11/2017	Yang	2020/0228836	A1	7/2020	Schwarz	

(56)

References Cited

U.S. PATENT DOCUMENTS

2020/0244993	A1	7/2020	Schwarz
2020/0260063	A1	8/2020	Hannuksela
2020/0262427	A1	8/2020	Taylor
2020/0273208	A1	8/2020	Mammou et al.
2020/0273258	A1	8/2020	Lasserre
2020/0275129	A1	8/2020	Deshpande
2020/0279435	A1	9/2020	Kuma
2020/0286261	A1	9/2020	Faramarzi et al.
2020/0288171	A1	9/2020	Hannuksela
2020/0294271	A1	9/2020	Ilola
2020/0302571	A1	9/2020	Schwarz
2020/0302578	A1	9/2020	Graziosi
2020/0302621	A1	9/2020	Kong
2020/0302651	A1	9/2020	Flynn
2020/0302655	A1	9/2020	Oh
2020/0359035	A1	11/2020	Chevet
2020/0359053	A1	11/2020	Yano
2020/0366941	A1	11/2020	Sugio et al.
2020/0374559	A1	11/2020	Fleureau
2020/0380765	A1	12/2020	Thudor
2020/0396489	A1	12/2020	Flynn
2020/0413096	A1	12/2020	Zhang
2021/0005006	A1	1/2021	Oh
2021/0006805	A1	1/2021	Urban et al.
2021/0006833	A1	1/2021	Tourapis
2021/0012538	A1	1/2021	Wang
2021/0014293	A1	1/2021	Yip
2021/0021869	A1	1/2021	Wang
2021/0027505	A1	1/2021	Yano
2021/0029381	A1	1/2021	Zhang et al.
2021/0056732	A1	2/2021	Han
2021/0074029	A1	3/2021	Fleureau
2021/0084333	A1	3/2021	Zhang
2021/0090301	A1	3/2021	Mammou et al.
2021/0097725	A1	4/2021	Mammou et al.
2021/0099701	A1	4/2021	Tourapis et al.
2021/0104014	A1	4/2021	Kolb, V
2021/0104073	A1	4/2021	Yea et al.
2021/0104075	A1	4/2021	Mammou et al.
2021/0105493	A1	4/2021	Mammou et al.
2021/0105504	A1	4/2021	Hur et al.
2021/0112281	A1	4/2021	Wang
2021/0119640	A1	4/2021	Mammou et al.
2021/0142522	A1	5/2021	Li
2021/0150766	A1	5/2021	Mammou et al.
2021/0150771	A1	5/2021	Huang
2021/0166432	A1	6/2021	Wang
2021/0166436	A1	6/2021	Zhang
2021/0168386	A1	6/2021	Zhang
2021/0195162	A1	6/2021	Chupeau et al.
2021/0201541	A1	7/2021	Lasserre
2021/0203989	A1	7/2021	Wang
2021/0211724	A1	7/2021	Kim et al.
2021/0217139	A1	7/2021	Yano
2021/0217203	A1	7/2021	Kim et al.
2021/0217206	A1	7/2021	Flynn
2021/0218969	A1	7/2021	Lasserre
2021/0233281	A1	7/2021	Wang et al.
2021/0248784	A1	8/2021	Gao
2021/0248785	A1	8/2021	Zhang
2021/0256735	A1	8/2021	Tourapis et al.
2021/0258610	A1	8/2021	Iguchi
2021/0264640	A1	8/2021	Mammou et al.
2021/0264641	A1	8/2021	Iguchi
2021/0266597	A1	8/2021	Kim et al.
2021/0281874	A1	9/2021	Lasserre
2021/0281880	A1*	9/2021	Litwic H04N 19/184
2021/0295569	A1	9/2021	Sugio
2021/0383576	A1	12/2021	Olivier
2021/0398352	A1	12/2021	Tokumo
2021/0400280	A1	12/2021	Zaghetto
2021/0407148	A1	12/2021	Flynn
2022/0005228	A1	1/2022	Tourapis et al.
2022/0020211	A1	1/2022	Vytyaz
2022/0030258	A1	1/2022	Zhang

2022/0070493	A1	3/2022	Mammou
2022/0084164	A1	3/2022	Hur
2022/0101555	A1	3/2022	Zhang
2022/0116659	A1	4/2022	Pesonen
2022/0239956	A1	7/2022	Tourapis
2022/0292722	A1	9/2022	Mammou et al.
2022/0383448	A1	12/2022	Valdez Balderas
2022/0405533	A1	12/2022	Mammou et al.
2023/0005188	A1	1/2023	Tourapis et al.
2023/0169658	A1	6/2023	Rhodes

FOREIGN PATENT DOCUMENTS

CN	101313578	11/2008
CN	10230618	1/2012
CN	102428698	4/2012
CN	102625127	8/2012
CN	102630011	8/2012
CN	103329524	9/2013
CN	103366006	10/2013
CN	103944580	7/2014
CN	104156972	11/2014
CN	104408689	3/2015
CN	105261060	1/2016
CN	105818167	8/2016
CN	106651942	5/2017
CN	106846425	6/2017
CN	107155342	9/2017
CN	108090960	5/2018
CN	108632607	10/2018
CN	108632621	10/2018
CN	109166160	1/2019
CN	109196559	1/2019
CN	109792520	5/2019
EP	1745442	1/2007
EP	2533213	12/2012
EP	3429210	1/2019
EP	3496388	6/2019
EP	3614674	2/2020
EP	3751857	12/2020
JP	2013111948	6/2013
RU	2674326	12/2018
WO	200004506	1/2000
WO	2008129021	10/2008
WO	2013022540	2/2013
WO	2018050725	3/2018
WO	2018094141	5/2018
WO	2018162406	9/2018
WO	2019011636	1/2019
WO	2019013430	1/2019
WO	2019076503	4/2019
WO	2019078696	4/2019
WO	2019079397	4/2019
WO	2019093834	5/2019
WO	2019129923	7/2019
WO	2019135024	7/2019
WO	2019143545	7/2019
WO	2019194522	10/2019
WO	2019199415	10/2019
WO	20190197708	10/2019
WO	2019069711	11/2019
WO	2020012073	1/2020
WO	2020066680	2/2020

OTHER PUBLICATIONS

U.S. Appl. No. 18/063,592 filed Dec. 8, 2022, Khaled Mammou, et al.
 G-PCC Future Enhancements^{***}, MPEG Meeting, Oct. 7-11, 2019, (Motion Picture Expert Group of ISO/IEC JTC1/ SC29-WG11), Retrieved from http://phenix.int-evry.fr/mpeg/doc_end_user/documents/128_Geneva/wg11/w18887.zipw18887/w18887 on Dec. 23, 2019, pp. 1-30.
 Miska M. Hannuksela, "On Slices and Tiles", JVET Meeting, The Joint Video Exploration Team of ISO/IEC, Sep. 25, 2018, pp. 1-3.
 David Flynn, "International Organisation for Standardisation Organisation International De Normalisation ISO/IEC JTC1/SC29/WG11 Coding of Moving Pictures and Audio", dated Apr. 2020. pages 1-9.

(56)

References Cited

OTHER PUBLICATIONS

- U.S. Appl. No. 17/691,754, filed Mar. 10, 2022, Khaled Mammou.
- Liu Chao, "Research on point cloud data processing and reconstruction," Full-text Database, Feb. 7, 2023.
- Yingzhan Xu, et al., "Dynamic Point cloud geometry compression via patch wise polynomial fitting," IEEE, ICASSP 2019, pp. 2287-2291.
- Ma Xiangyin, et al., "Stereo Matching by Using Adaptive Smoothness Term," *Journal of Computer-Aided Design & Computer Graphics*, vol. 22, No. 1, Jan. 2010, pp. 108-113.
- Kong Jianhong, et al., "Based on a Viewpoint-based 3D point of dynamic adaptive LOD Rendering 3D Point Cloud," vol. 19, No. 12, Apr. 2019, 2019 Sci. Tech. Engrg. China Academic Journal Electronic Publishing House, pp. 1-6. (no translation avail).
- Search Report from Chinese Application No. 201780047827.X, dated Feb. 26, 2023, pp. 1-5.
- U.S. Appl. No. 17/157,833, filed Jan. 25, 2021, Khaled Mammou.
- R. Mekuria, et al., "Design, Implementation and Evaluation of a Point Cloud Codec for Tele-Immersive Video", *IEEE Transactions on Circuits and Systems for Video Technology* 27.4, 2017, pp. 1-14.
- Jae-Kyun, et al., "Large-Scale 3D Point Cloud Compression Using Adaptive Radial Distance Prediction in Hybrid Coordinate Domains", *IEEE Journal of Selected Topics in Signal Processing*, vol. 9, No. 3, Apr. 2015, pp. 1-14.
- "David Flynn et al., "G-PCC: A hierarchical geometry slice structure", MPEG Meeting, Retrieved from http://phenix.intevry.fr/mpeg/doc_end_user/documents/131_Online/wg11/m54677-v1-m54677_v1.zip, Jun. 28, 2020, pp. 1-9".
- Jingming Dong, "Optimal Visual Representation Engineering and Learning for Computer Vision", Doctoral Dissertation, UCLA, 2017, pp. 1-151.
- Tim Golla et al., "Real-time Point Cloud Compression", *IROS*, 2015, pp. 1-6.
- Dong Liu, et al., "Three-Dimensional Point-Cloud Plus Patches: Towards Model-Based Image Coding in the Cloud", 2015 IEEE International Conference on Multimedia Big Data, IEEE Computer Society, pp. 395-400.
- Tilo Ochotta et al., "Image-Based Surface Compression", dated Sep. 1, 2008, pp. 1647-1663.
- Cohen Robert A et al., "Point Cloud Attribute Compression Using 3-D Intra Prediction and Shape-Adaptive Transforms", dated Mar. 30, 2016, pp. 141-150.
- Khaled Mammou et al., "Working Draft of Point Cloud Coding for Category 2 (Draft 1)", dated Apr. 2018, pp. 1-38.
- Khaled Mammou et al., "Input Contribution", dated Oct. 8, 2018, pp. 1-42.
- Benjamin Bross et al., "High Efficiency Video Coding (HEVC) Text Specification Draft 8", dated Jul. 23, 2012, pp. 1-86.
- JunTaek Park et al., "Non-Overlapping Patch Packing in TMC2 with HEVC-SCC", dated Oct. 8, 2018, pp. 1-6.
- Sebastian Schwarz, et al., "Emerging MPEG Standards for Point Cloud Compression", *IEEE Journal on Emerging and Selected Topics In Circuits and Systems*, vol. 9, No. 1, Mar. 2019, pp. 133-148.
- Li Li, et al., Efficient Projected Frame Padding for Video-based Point Cloud Compression, *IEEE Transactions on Multimedia*, doi: 10.100/TMM.2020.3016894, 2020, pp. 1-14.
- Lujia Wang, et al., "Point-cloud Compression Using Data Independent Method—A 3D Discrete Cosine Transform Approach", in *Proceedings of the 2017 IEEE International Conference on Information and Automation (ICIA)*, Jul. 2017, pp. 1-6.
- Ismael Daribo, et al., "Efficient Rate-Distortion Compression on Dynamic Point Cloud for Grid-Pattern-Based 3D Scanning Systems", *3D Research* 3.1, Springer, 2012, pp. 1-9.
- Yiting Shao, et al., "Attribute Compression of 3D Point Clouds Using Laplacian Sparsity Optimized Graph Transform", 2017 IEEE Visual Communications and Image Processing (VCIP), IEEE, 2017, p. 1-4.
- Siheng Chen, et al., "Fast Resampling of 3D Point Clouds via Graphs", *arXiv:1702.06397v1*, Feb. 11, 2017, pp. 1-15.
- Nahid Sheikhi Pour, "Improvements for Projection-Based Point Cloud Compression", MS Thesis, 2018, pp. 1-75.
- Robert Skupin, et al., "Multiview Point Cloud Filtering for Spatiotemporal Consistency", *VISAPP 2014—International Conference on Computer Vision Theory and Applications*, 2014, pp. 531-538.
- Pragyana K. Mishra, "Image and Depth Coherent Surface Description", Doctoral dissertation, Carnegie Mellon University, The Robotics Institute, Mar. 2005, pp. 1-152.
- Chou, et al., "Dynamic Polygon Clouds: Representation and Compression for VR/AR", *ARXIV ID: 1610.00402*, Published Oct. 3, 2016, pp. 1-28.
- Jang et al., Video-Based Point-Cloud-Compression Standard in MPEG: From Evidence Collection to Committee Draft [Standards in a Nutshell], *IEEE Signal Processing Magazine*, Apr. 2019, pp. 1-6.
- Ekekrantz, Johan, et al., "Adaptive Cost Function for Pointcloud Registration," *arXiv preprint arXiv: 1704.07910* (2017).
- Vincente Morell, et al., "Geometric 3D point cloud compression", Copyright 2014 Elsevier B.V. All rights reserved, pp. 1-8.
- Stefan Gumhold et al., "Predictive Point-Cloud Compression", dated Jul. 31, 2005, pp. 1-7.
- Pierre-Marie Gandoin et al., "Progressive Lossless Compression of Arbitrary Simplicial Complexes", Dated Jul. 1, 2002, pp. 1-8.
- Ruwen Schnabel et al., "Octree-based Point-Cloud Compression", *Eurographics Symposium on Point-Based Graphics*, 2006, pp. 1-11.
- Yuxue Fan et al., "Point Cloud Compression Based on Hierarchical Point Clustering", *Signal and Information Processing Association Annual Summit and Conference (APSIPA)*, IEEE, 2013, pp. 1-7.
- Kammert, et al., "Real-time Compression of Point Cloud Streams", 2012 IEEE International Conference on Robotics and Automation, RiverCentre, Saint Paul, Minnesota, USA, May 14-18, 2012, pp. 778-785.
- Garcia, et al., "Context-Based Octree Coding for Point Cloud Video", 2017 IEEE International Conference on Image Processing (ICIP), 2017, pp. 1412-1416.
- Mery et al., Compression of dense and regular point clouds, *Proceedings of the 4th international conference on Computer graphics, virtual reality, visualisation and interaction in Africa* (pp. 15-20). ACM. (Jan. 2006).
- Lustosa et al., Database system support of simulation data, *Proceedings of the VLDB Endowment* 9.13 (2016): pp. 1329-1340.
- Remi Cura et al., "Implicit Lod for Processing and Classification in Point Cloud Servers", dated Mar. 4, 2016, pp. 1-18.
- Yan Huang et al, Octree-Based Progressive Geometry Coding of Point Clouds, dated Jan. 1, 2006, pp. 1-10.
- Khaled Mammou, et al., "G-PCC codec description v1", International Organisation for Standardisation, ISO/IEC JTC1/ SC29/WG11, Oct. 2018, pp. 1-32.
- D. Graziosi et al, "An overview of ongoing point cloud compression standardization activities: video-based (V-PCC) and geometry-based (G-PCC)" *Asipa Transactions on Signal and Information Processing*, vol. 9, dated Apr. 30, 2020, pp. 1-17.
- "Continuous improvement of study text of ISO-IEC CD 23090-5 Video-Based Point Cloud Compression" dated May 8, 2019, pp. 1-140.
- Flynn D et al, "G-PCC Bypass coding of bypass bins", dated Mar. 21, 2019, pp. 1-3.
- Sharma K et al, "CABAC Packet-Based Stream", dated Nov. 18, 2011, pp. 1-6.
- Lasserre S et al, "On bypassed bit coding and chunks", dated Apr. 6, 2020, pp. 1-3.
- David Flynn et al, "G-pcc low latency bypass bin coding". dated Oct. 3, 2019, pp. 1-4.
- Chuan Wang, et al., "Video Vectorization via Tetrahedral Remeshing", *IEEE Transactions on Image Processing*, vol. 26, No. 4, Apr. 2017, pp. 1833-1844.
- W. Zhu, et al., "Lossless point cloud geometry compression via binary tree partition and intra prediction," 2017 IEEE 19th International Workshop on Multimedia Signal Processing (MMSP), 2017, pp. 1-6, doi: 1.1109/MMSP.2017.8122226 (Year 2017).
- Robert Cohen, "CE 3.2 point-based prediction for point loud compression", dated Apr. 2018, pp. 1-6.

(56)

References Cited

OTHER PUBLICATIONS

“Bin Lu, et al., ““Massive Point Cloud Space Management Method Based on Octree-Like Encoding””, Arabian Journal for Science Engineering, <https://doi.org/10.1007/s13369-019-03968-7>, 2019, pp. 1-15”.

“Wikipedia, ““k-d tree””, Aug. 1, 2019, Retrieved from URL: https://en.wikipedia.org/w/index.php?title=Kd_tree&oldid=908900837, pp. 1-9”.

U.S. Appl. No. 18/807,049, filed Aug. 16, 2024, Mammou, et al.

U.S. Appl. No. 18/806,983, filed Aug. 16, 2024, Tourapis, et al.

U.S. Appl. No. 18/415,416, filed Jan. 17, 2024, Khaled Mammou, et al.

U.S. Appl. No. 18/415,436, filed Jan. 17, 2024, Khaled Mammou, et al.

* cited by examiner

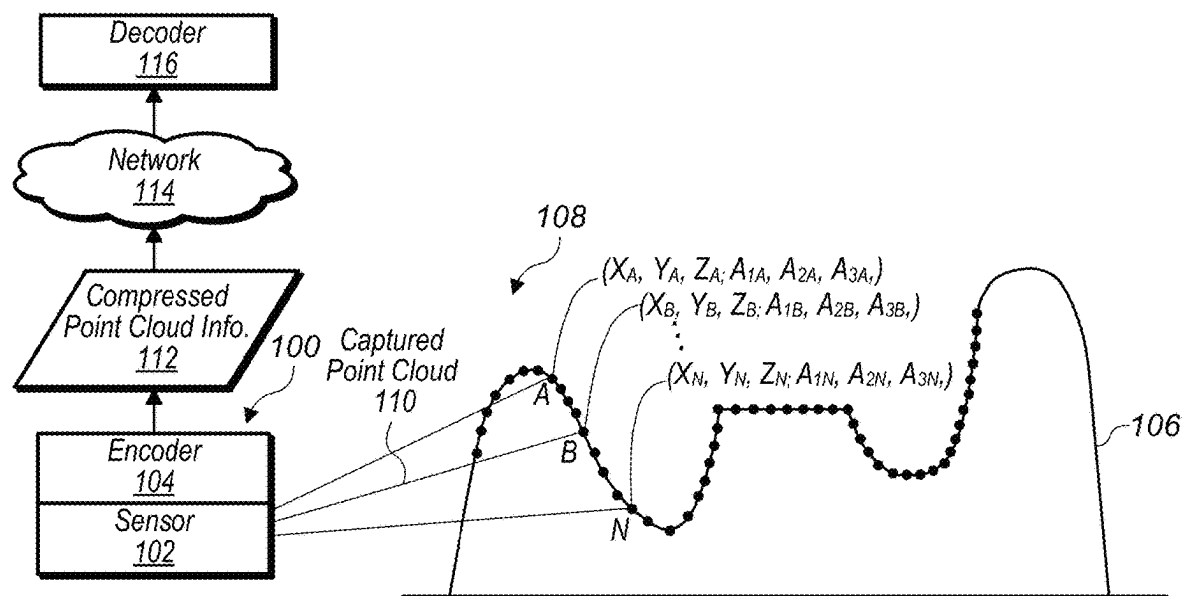


FIG. 1

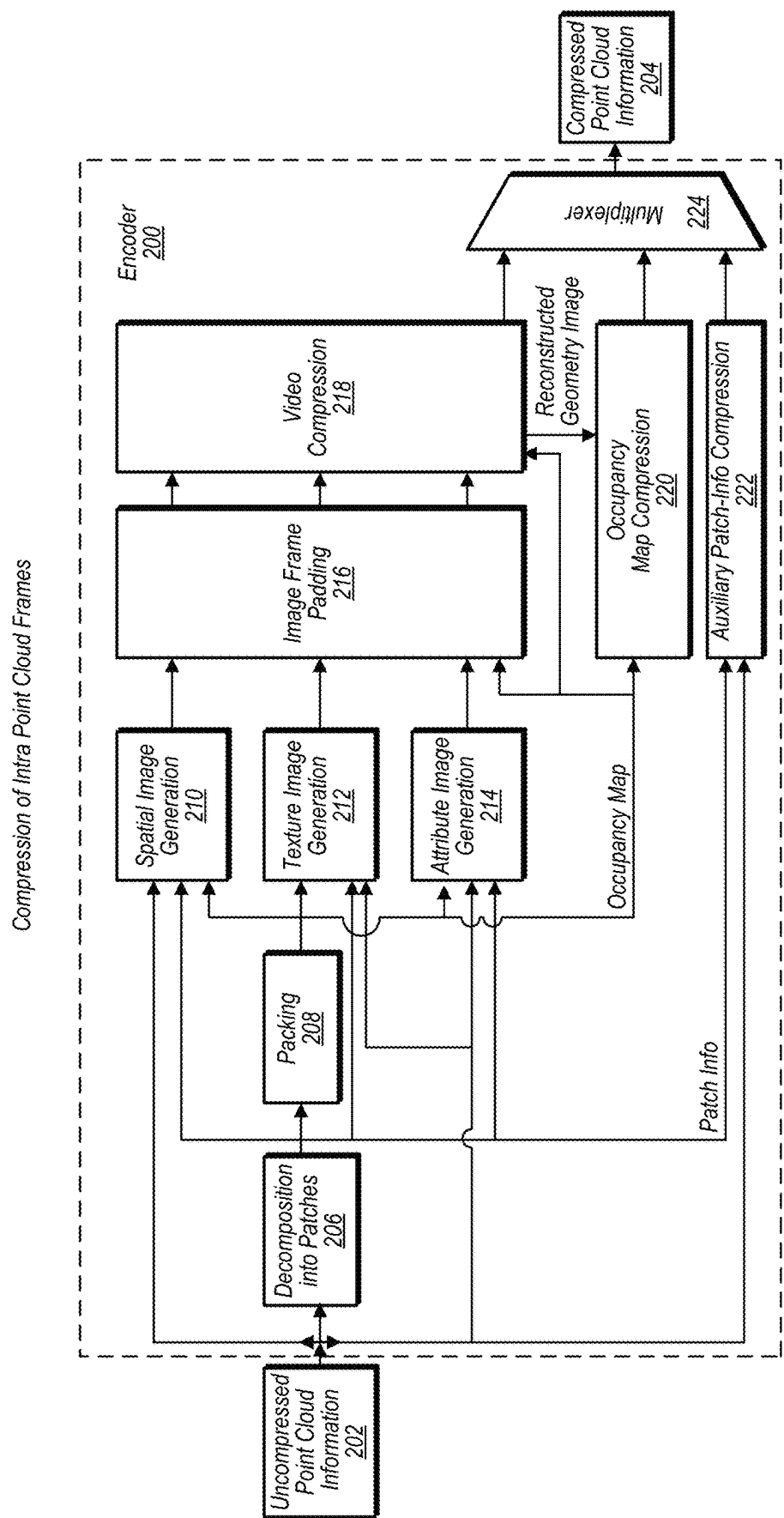


FIG. 2A

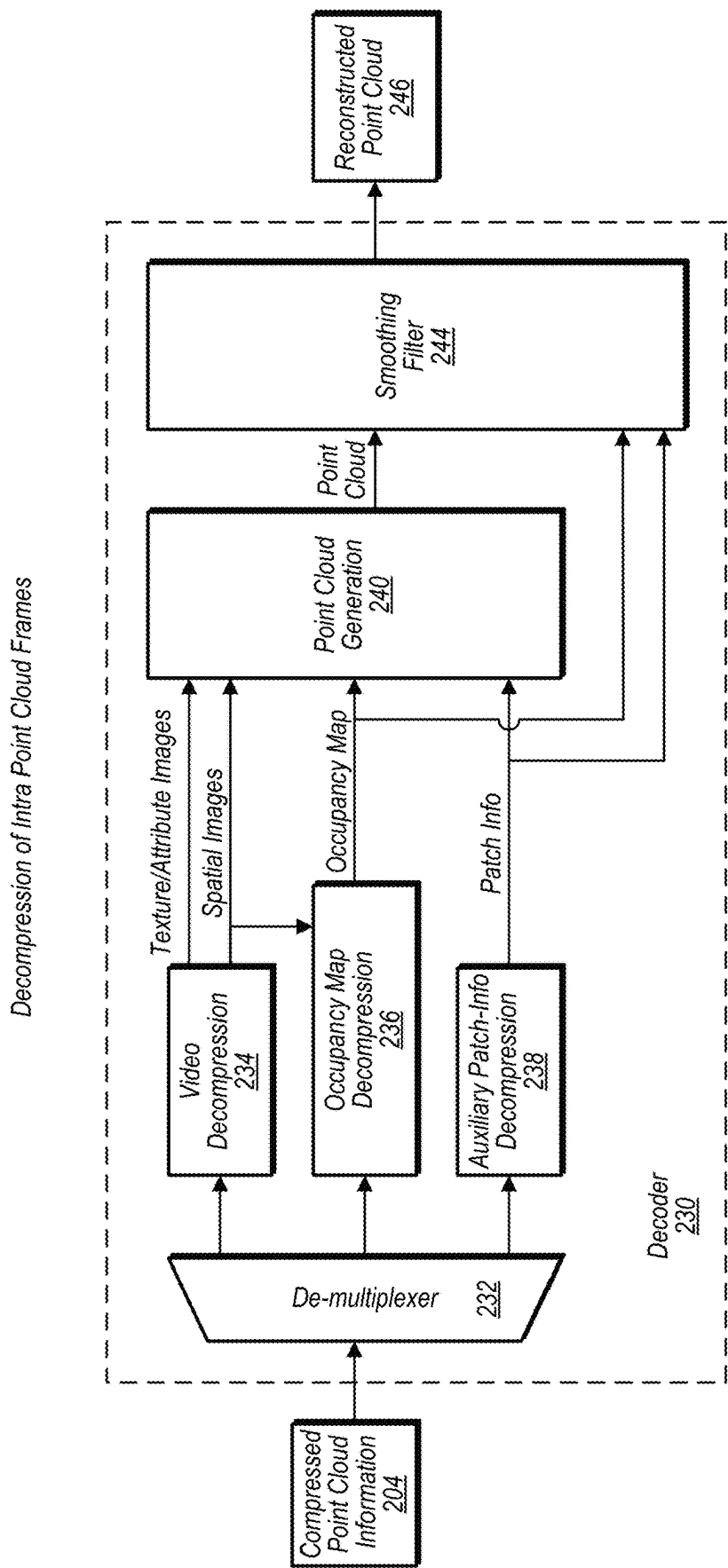


FIG. 2B

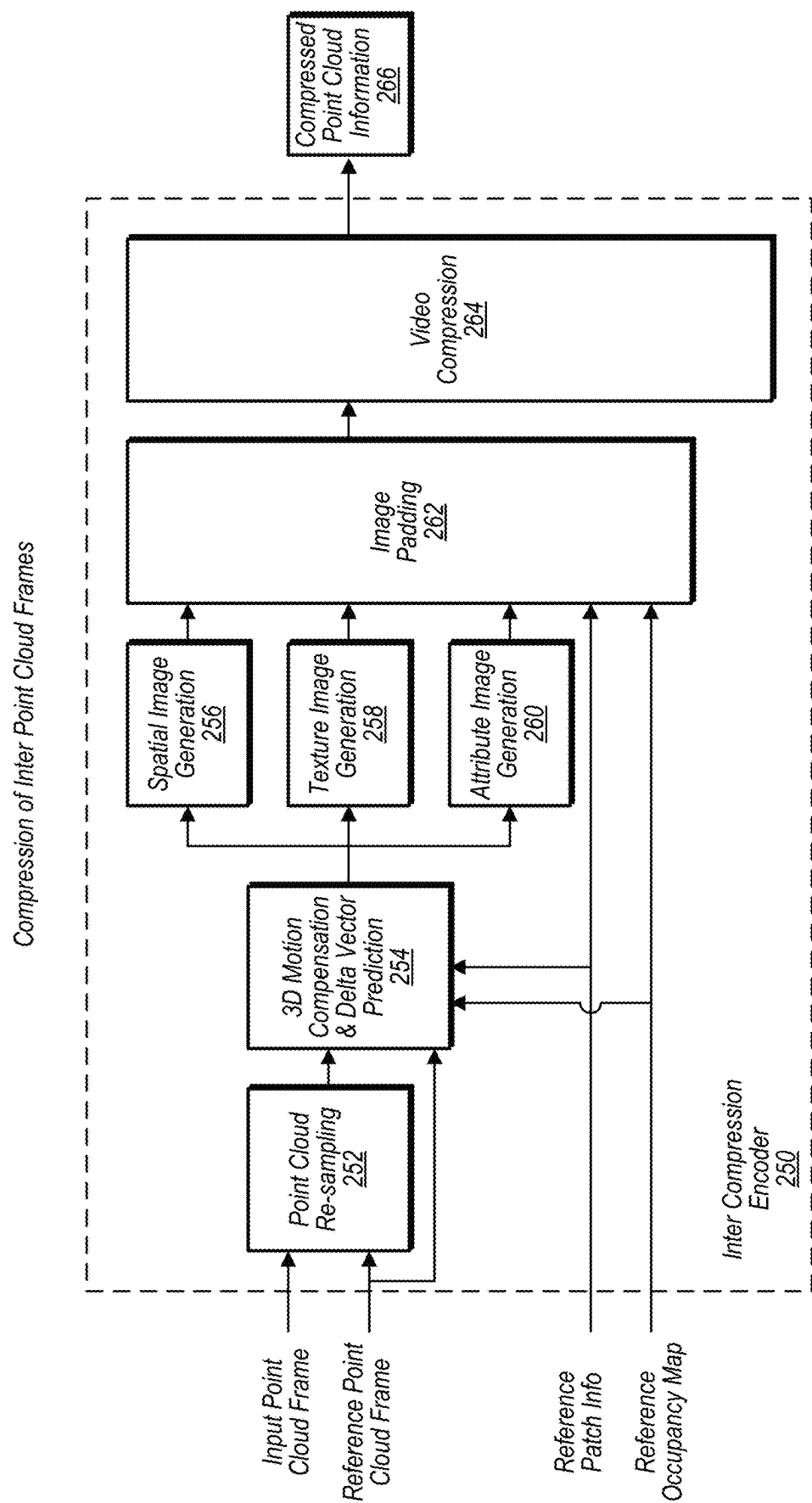


FIG. 2C

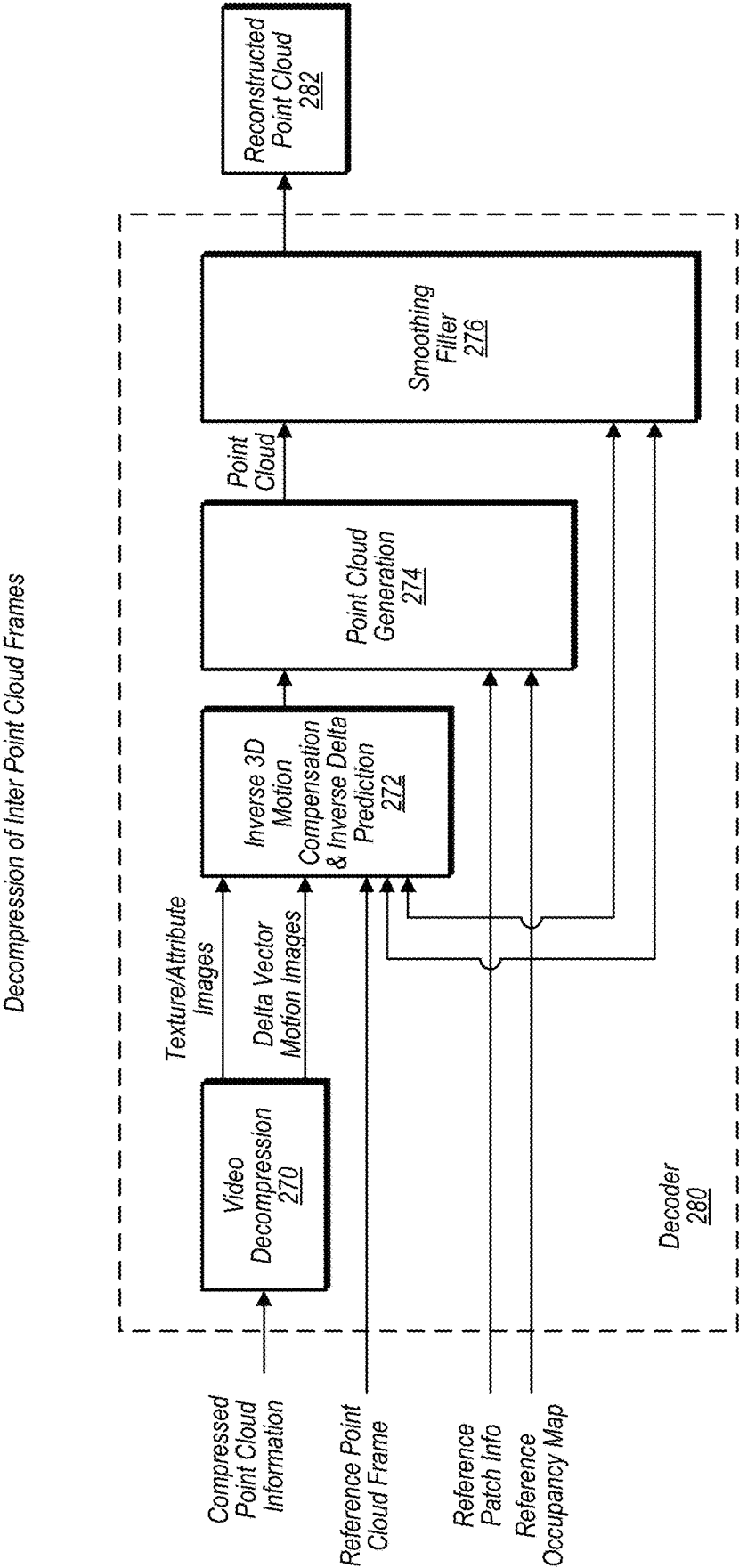


FIG. 2D

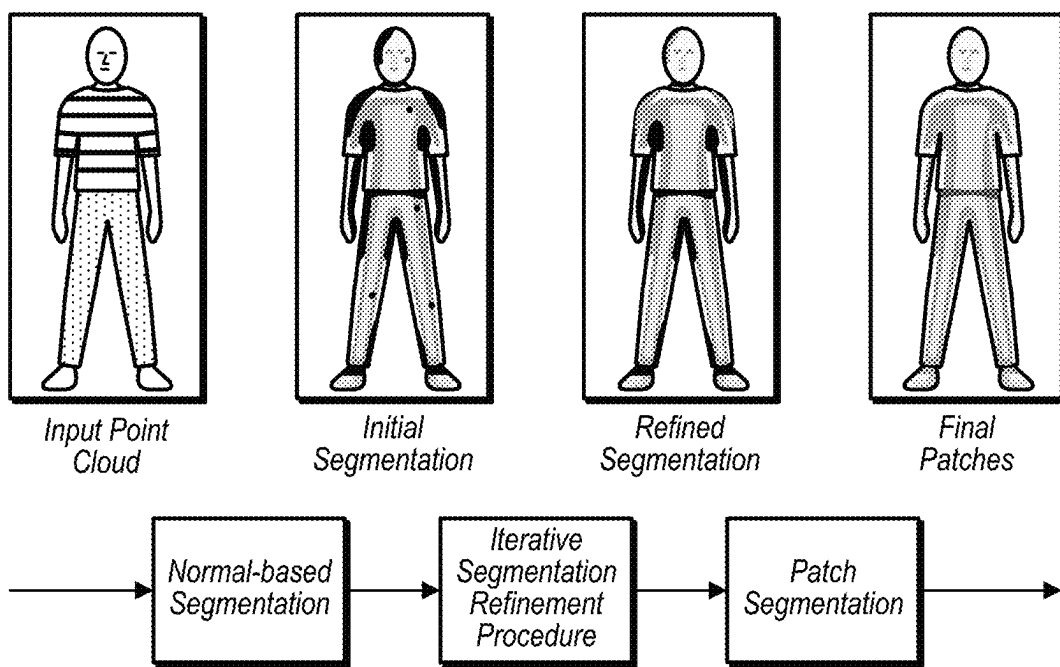


FIG. 3A

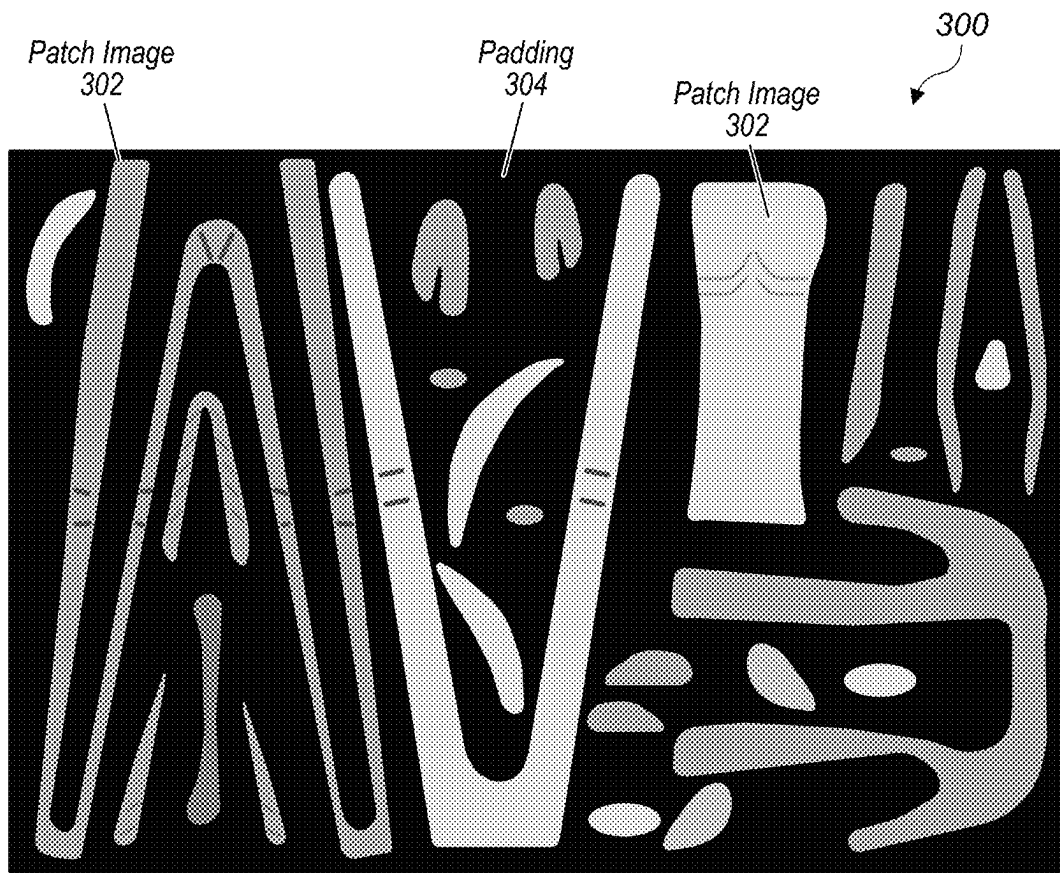


FIG. 3B

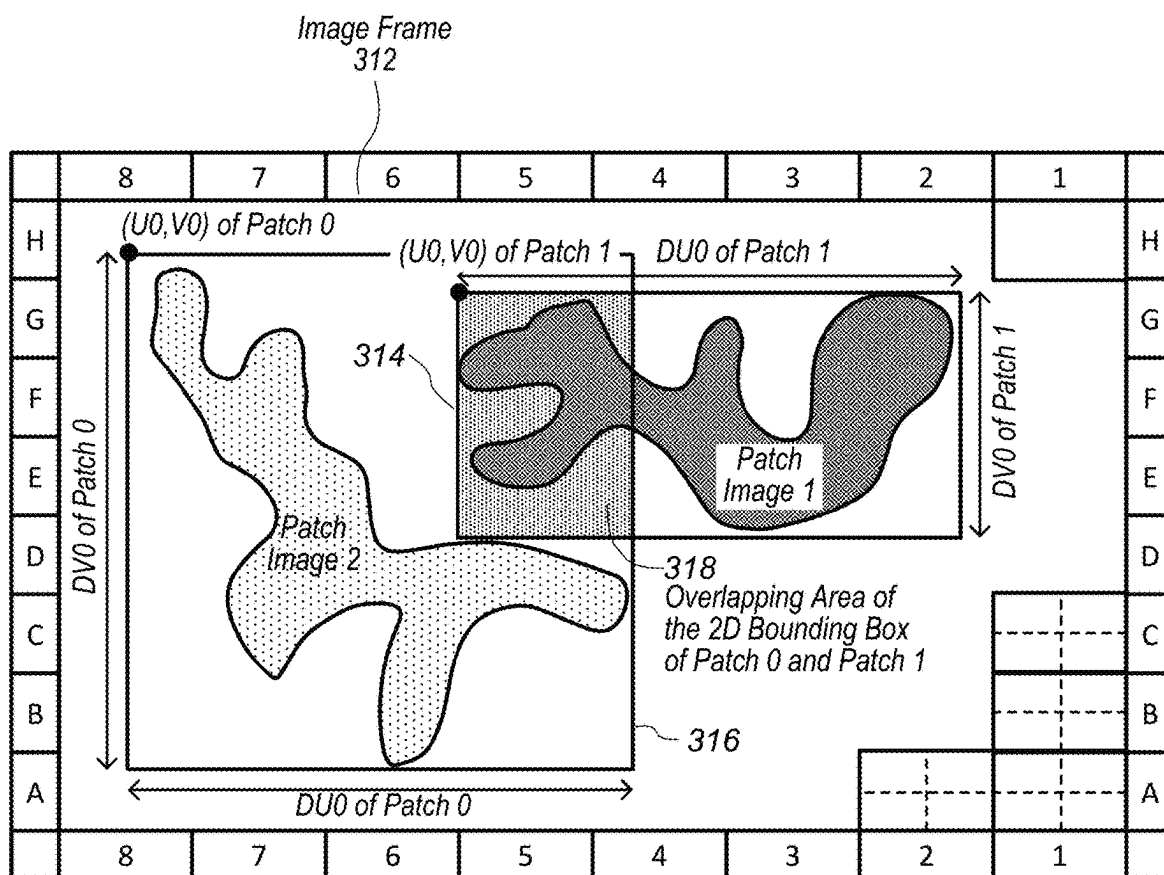


FIG. 3C

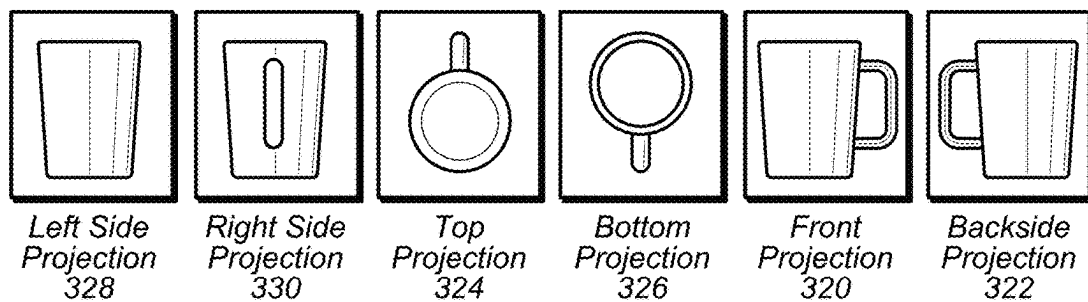


FIG. 3D

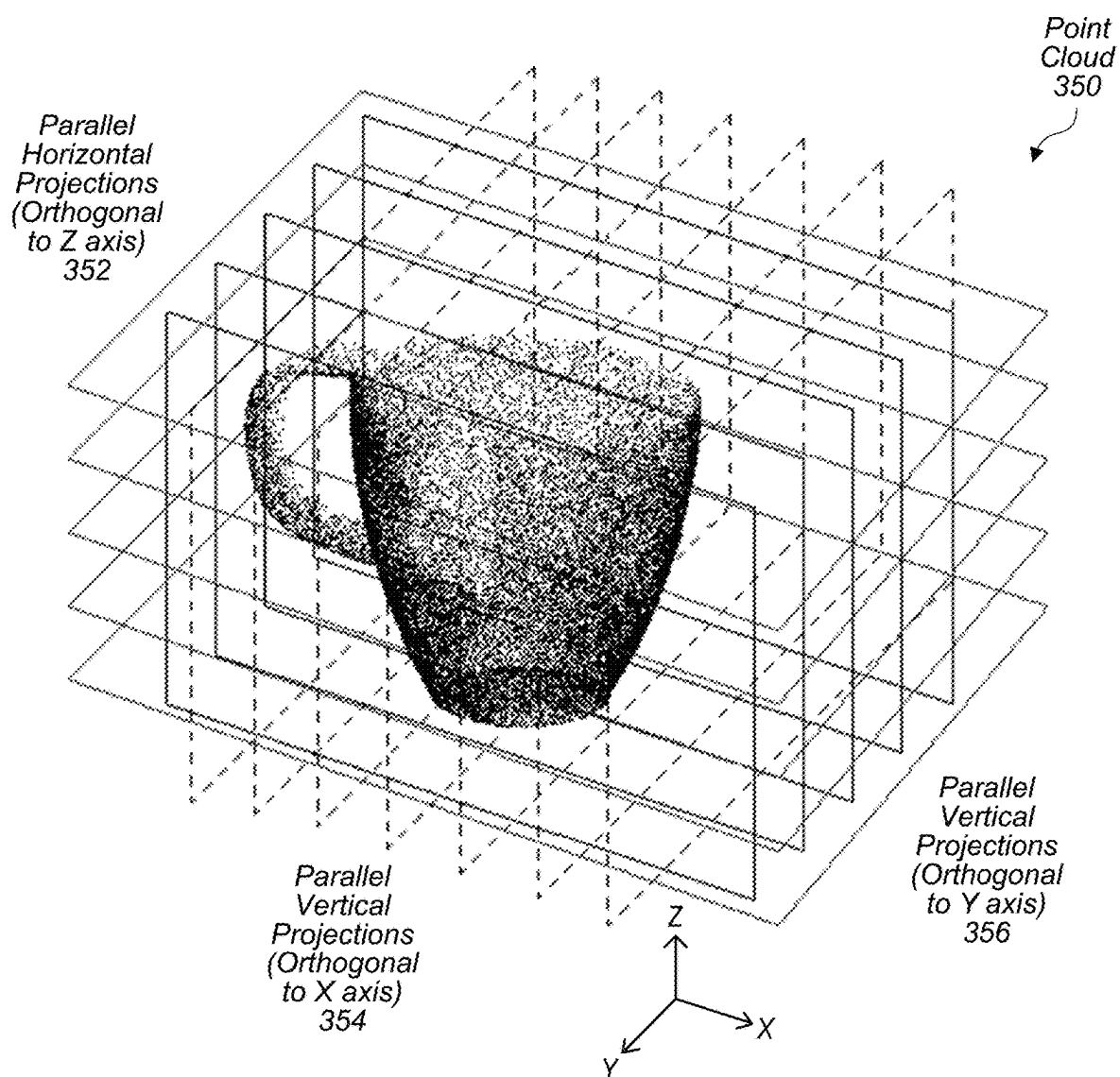


FIG. 3E

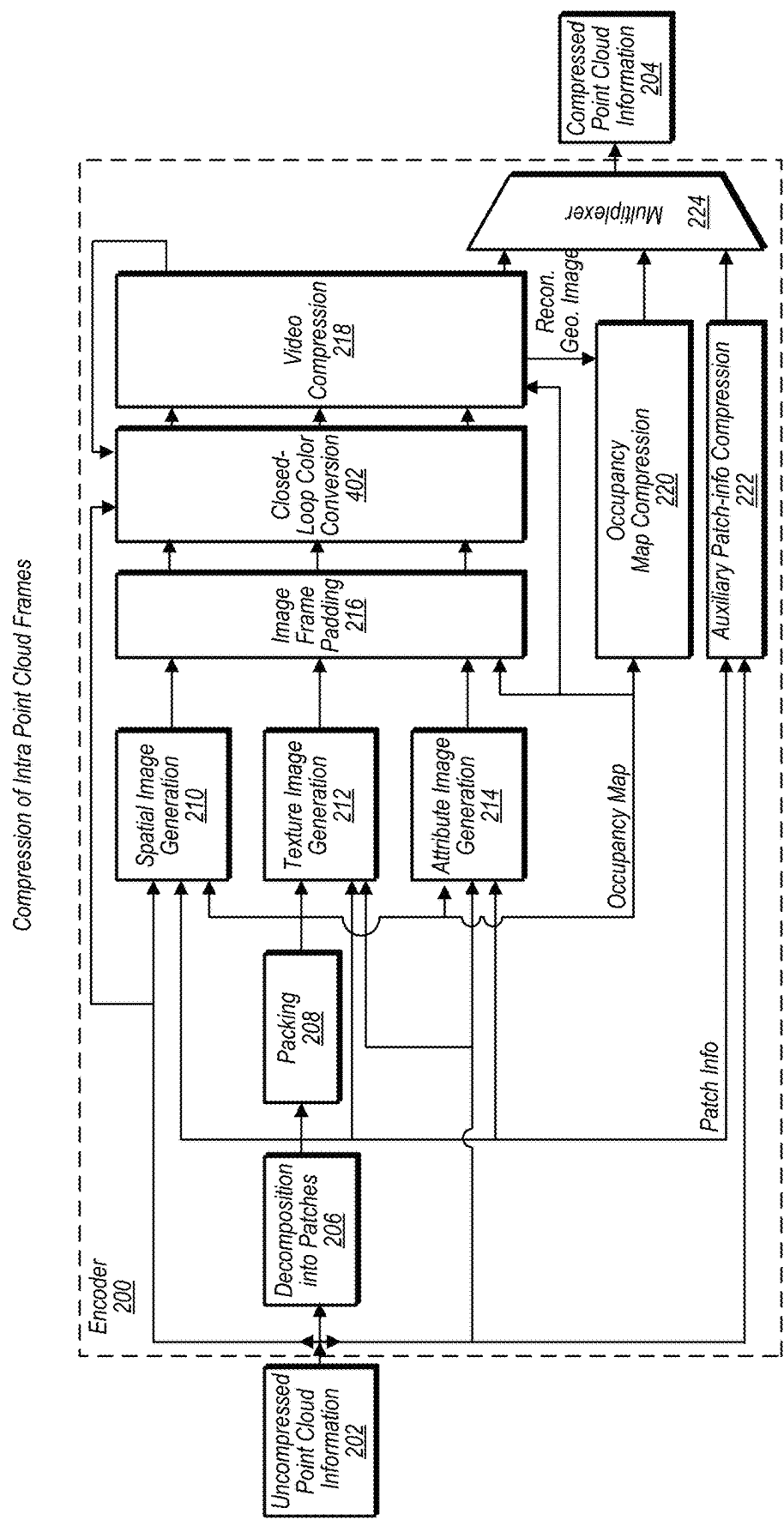


FIG. 4A

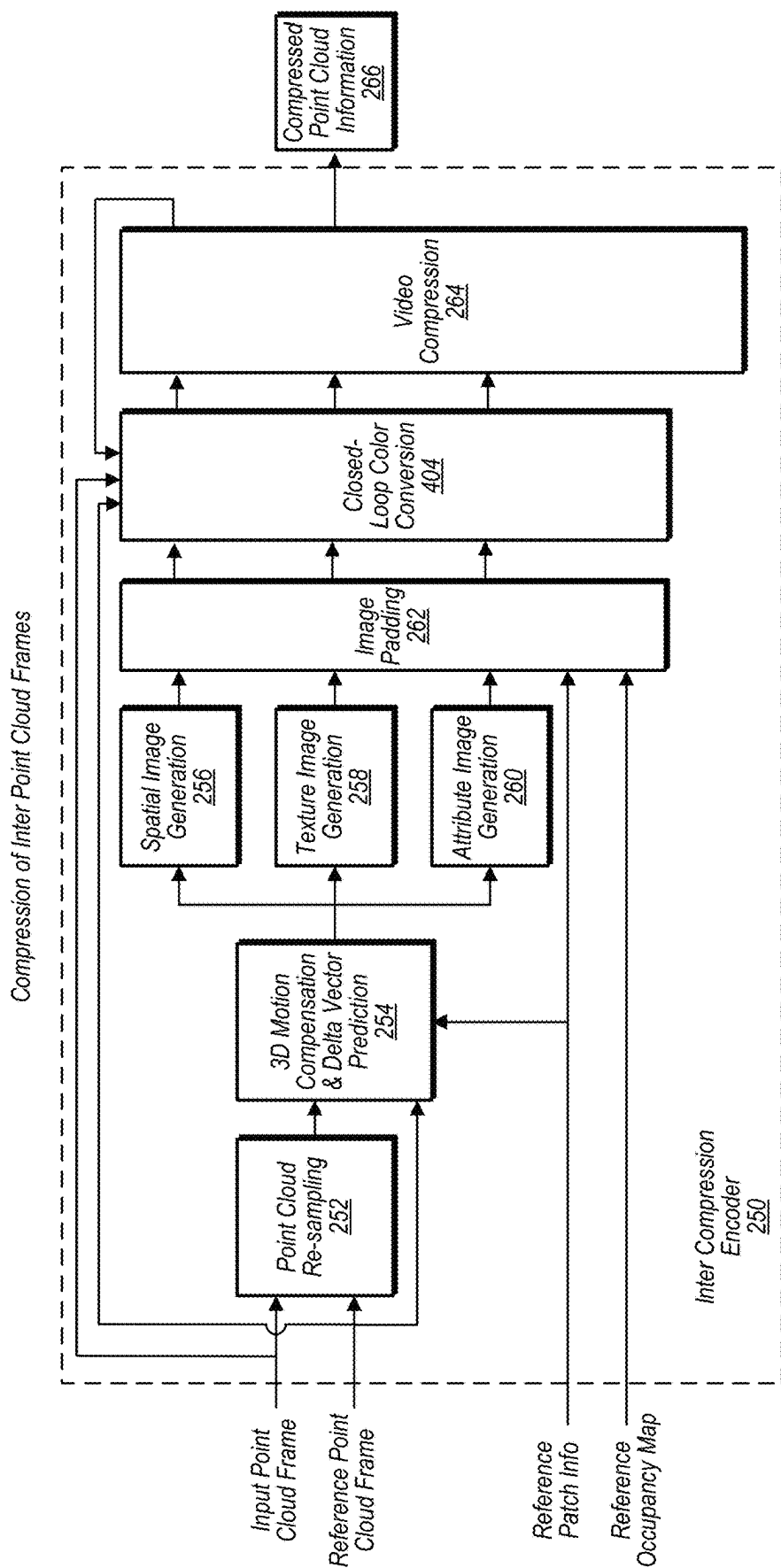


FIG. 4B

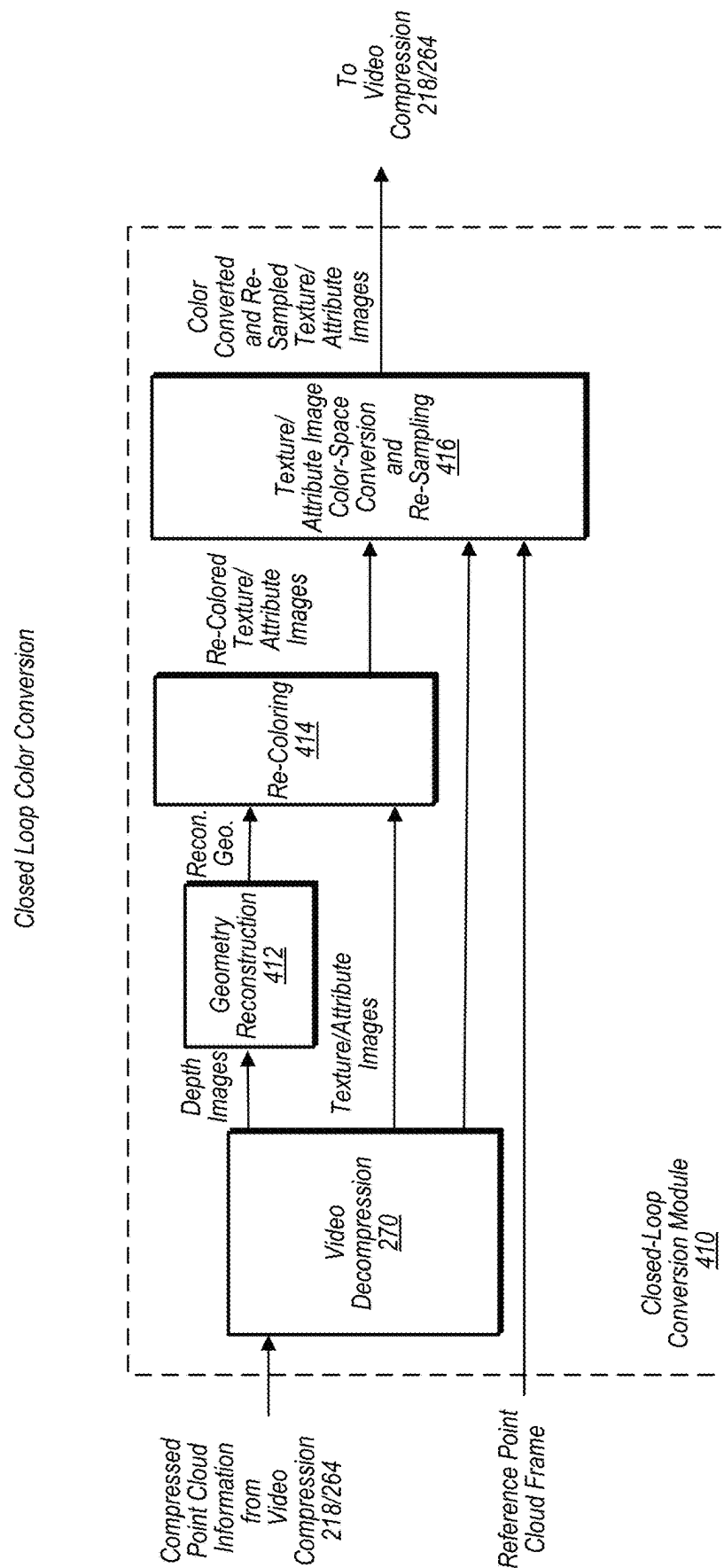
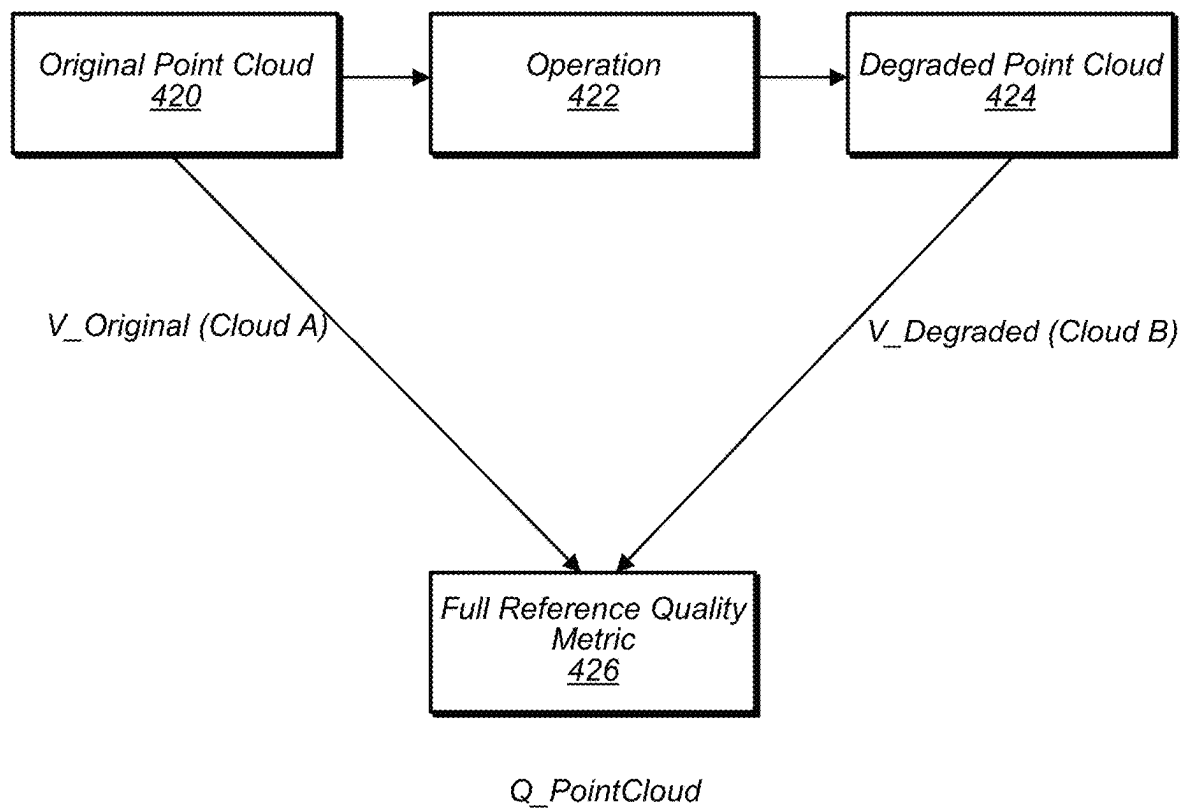


FIG. 4C

**FIG. 4D**

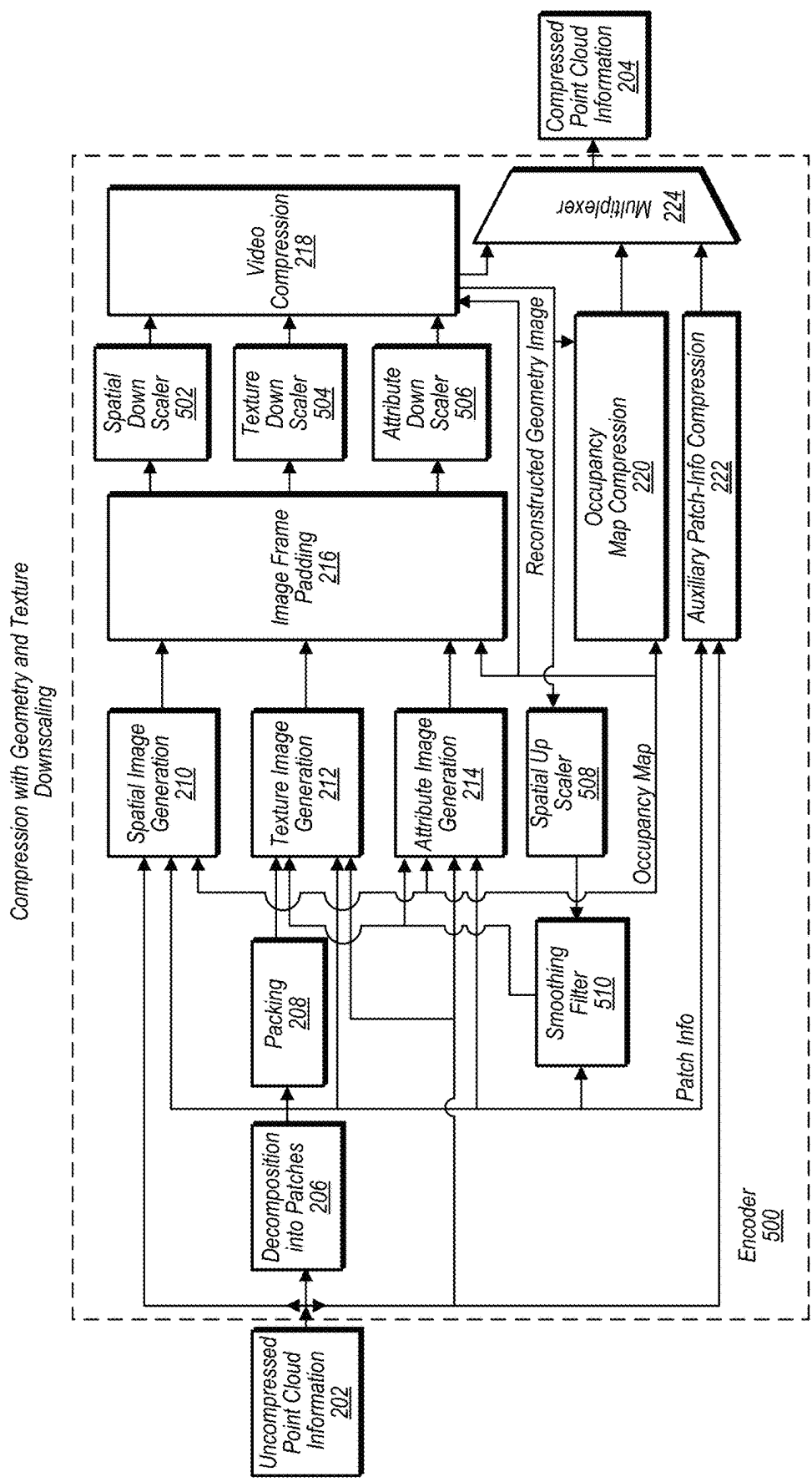


FIG. 5A

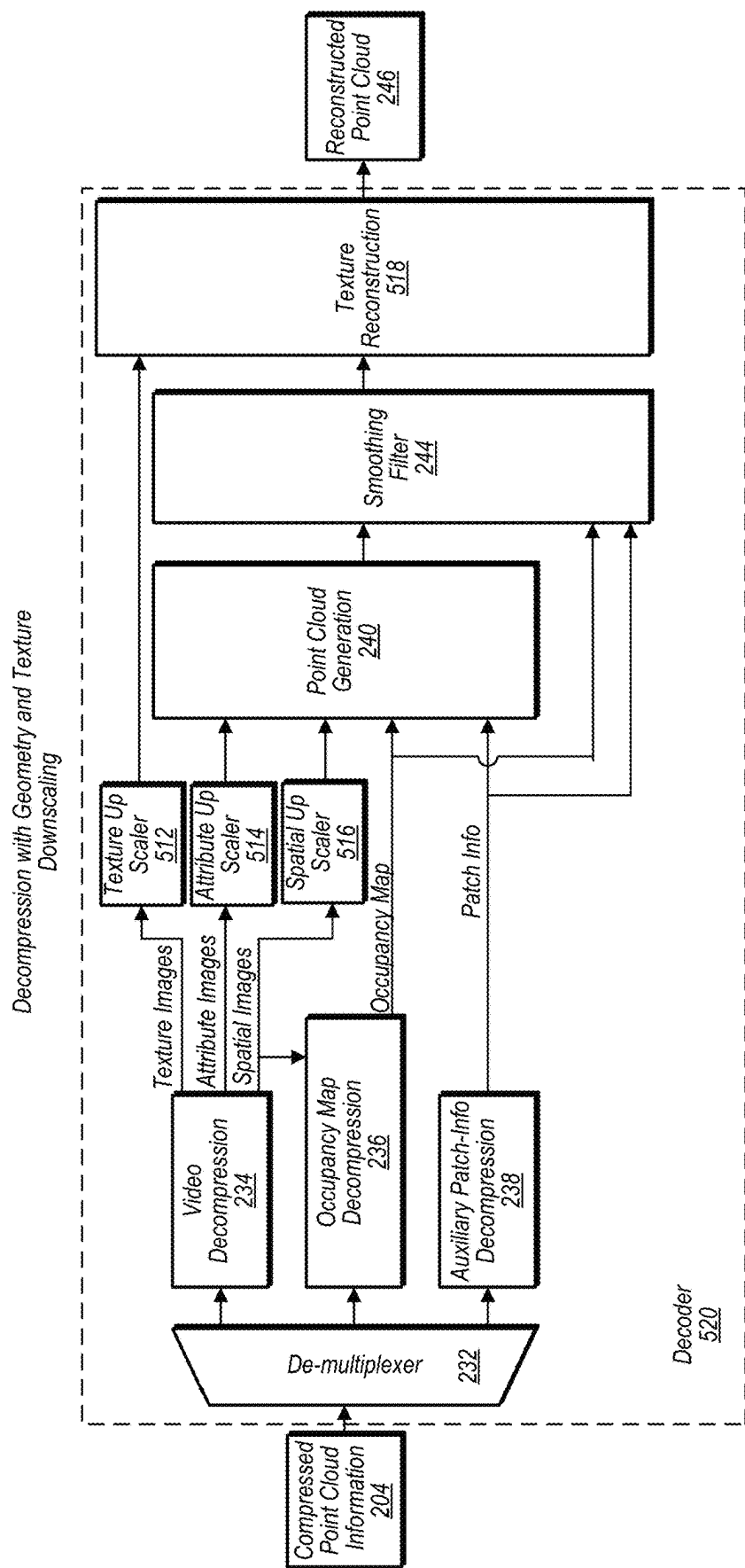


FIG. 5B

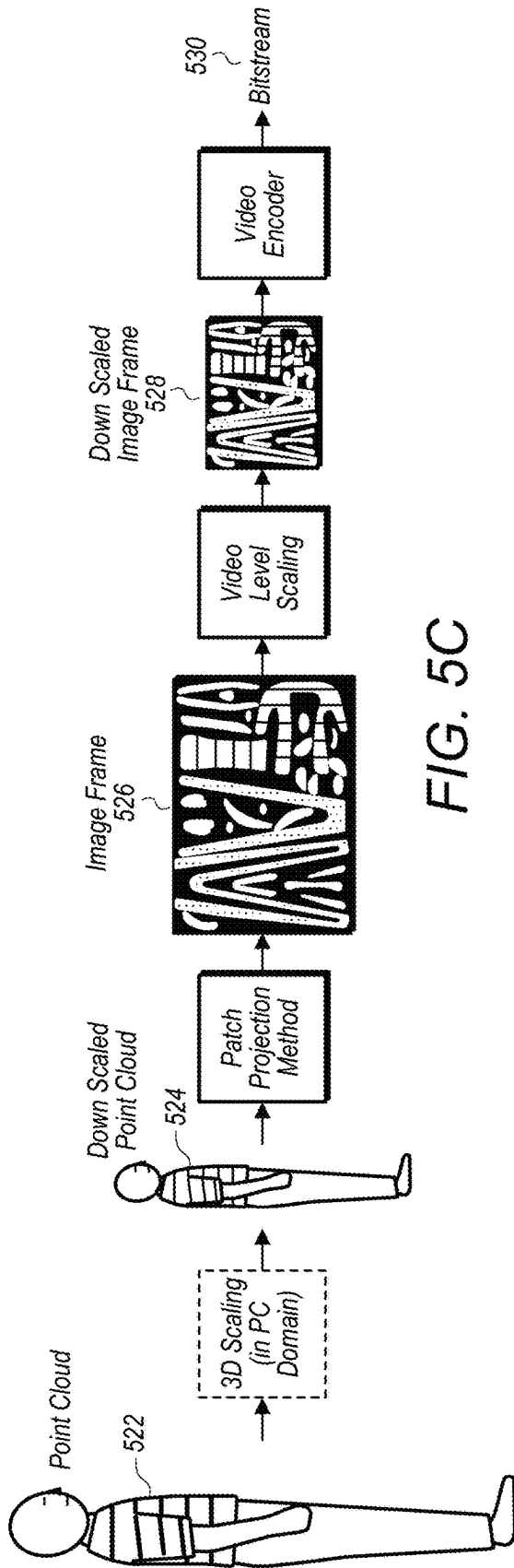


FIG. 5C

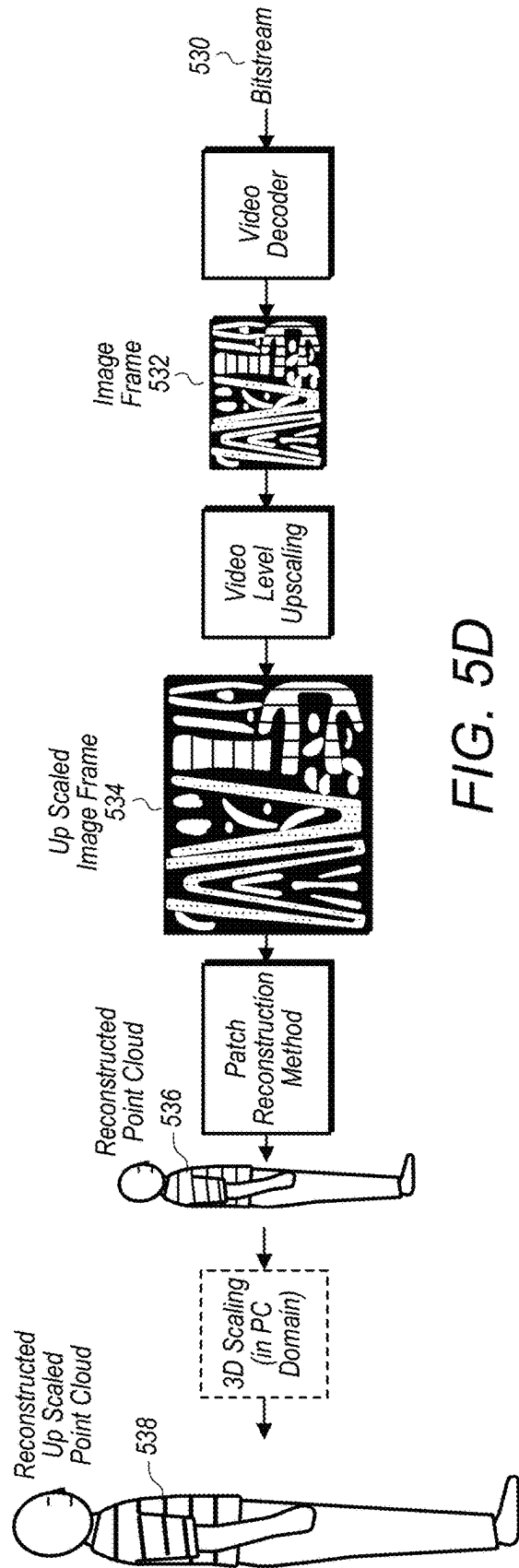


FIG. 5D

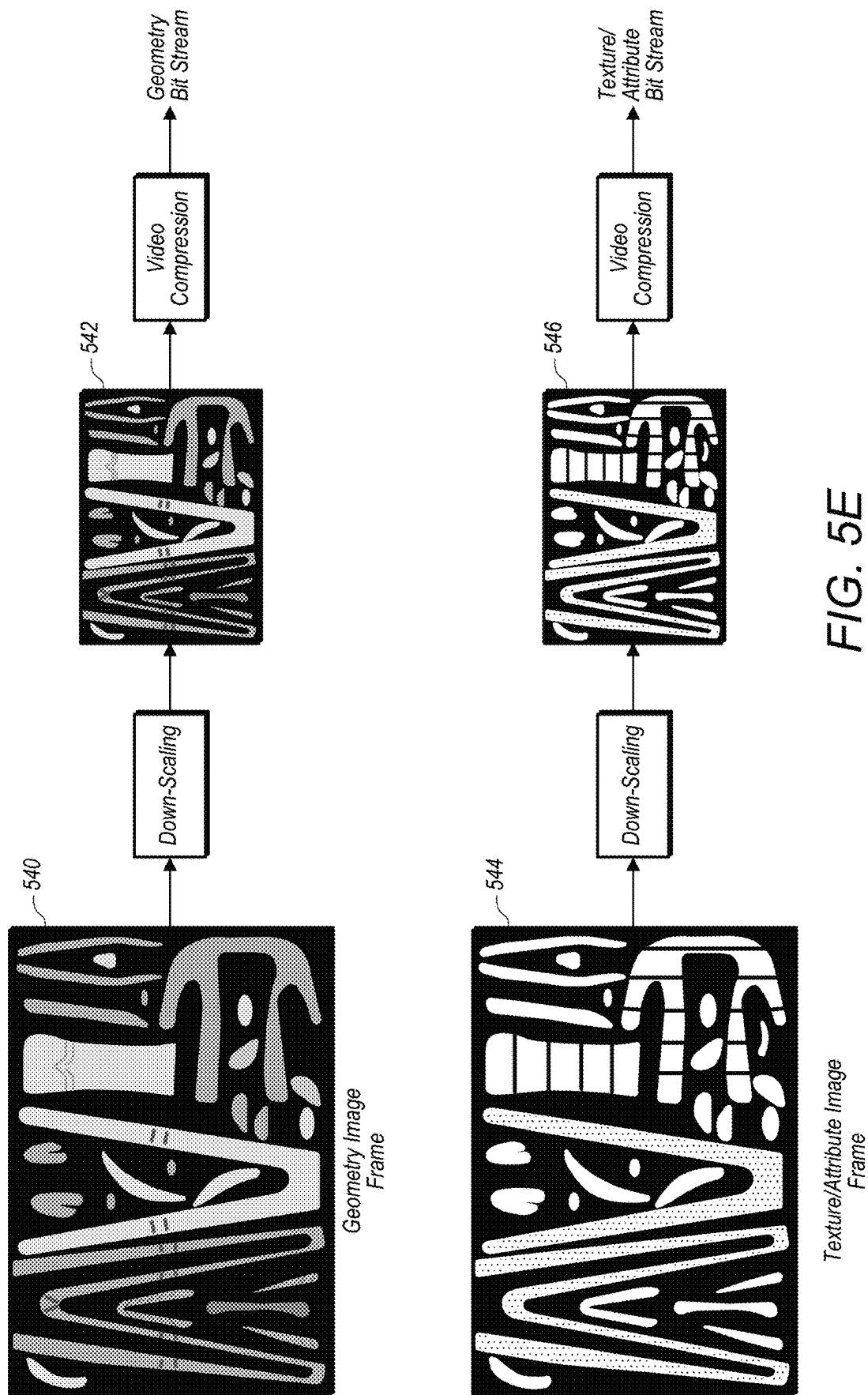


FIG. 5E

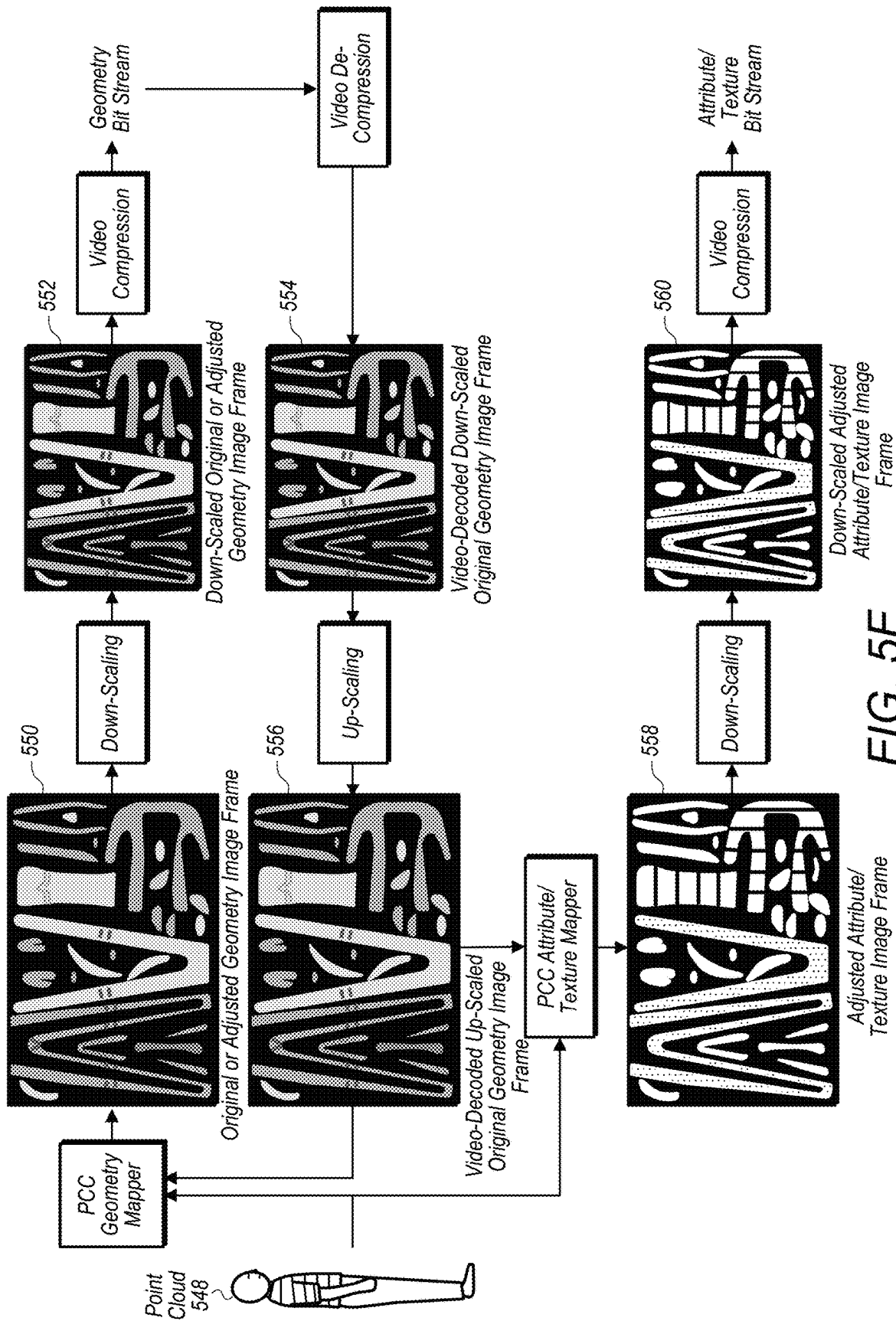


FIG. 5F

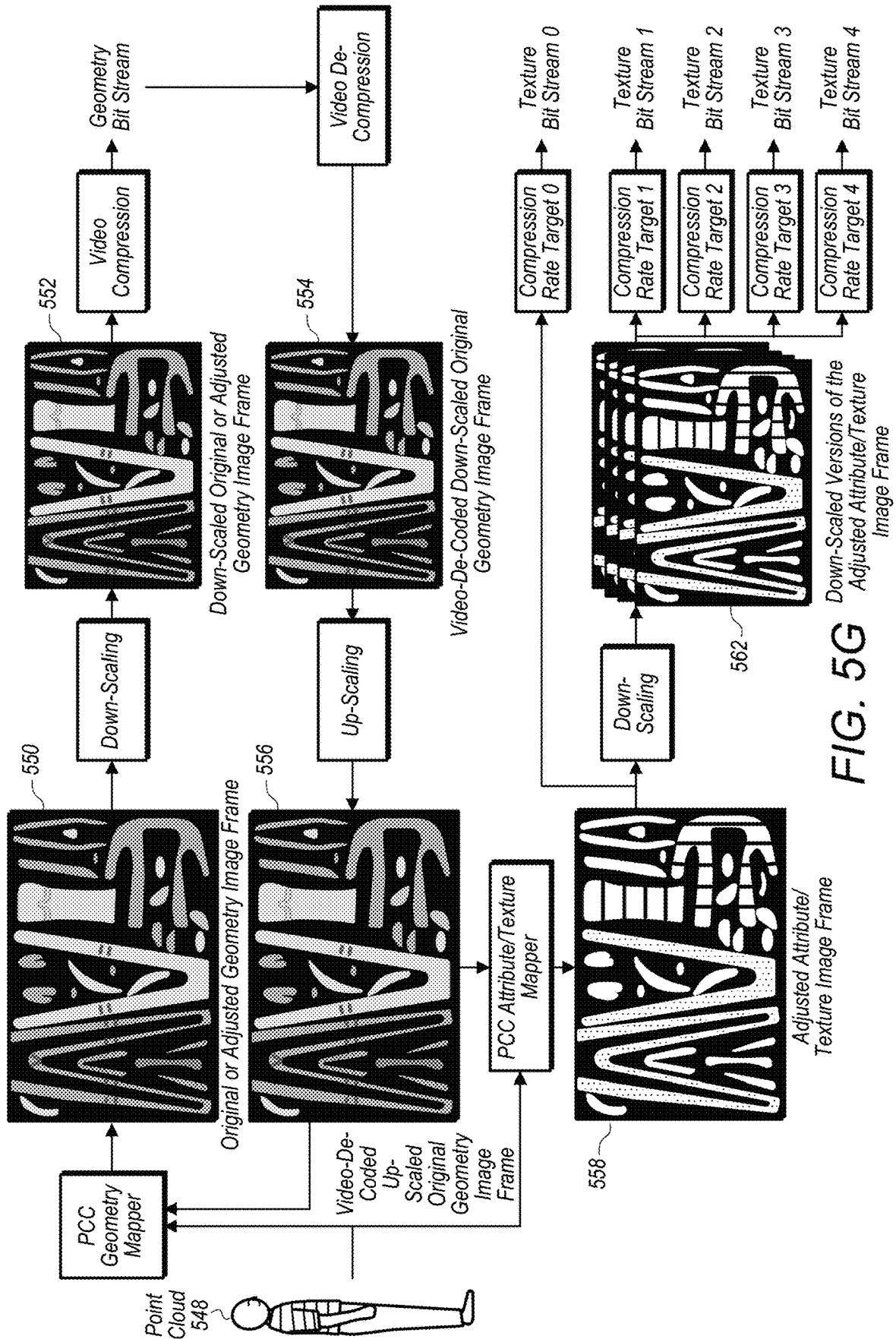


FIG. 5G

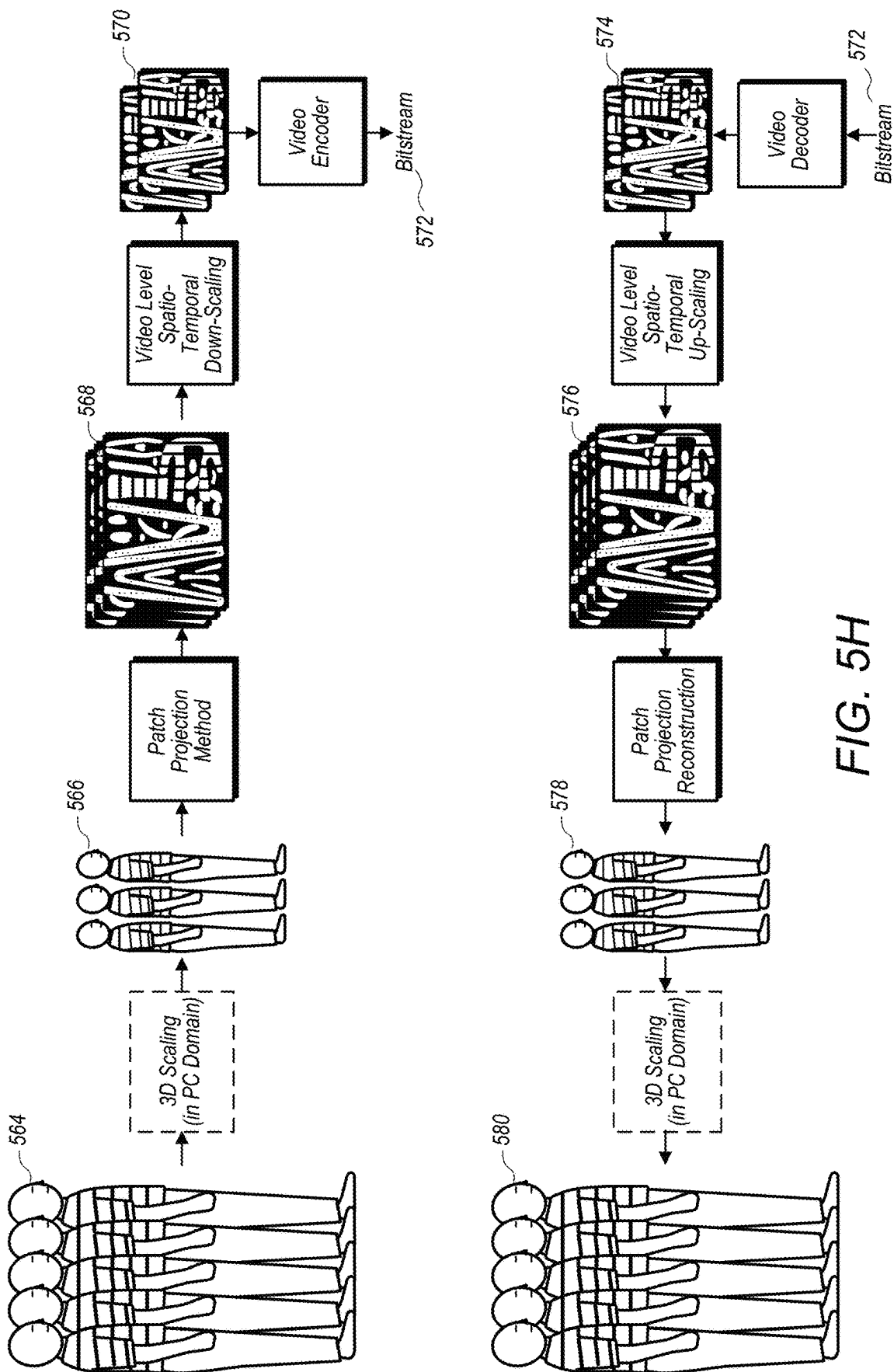


FIG. 5H

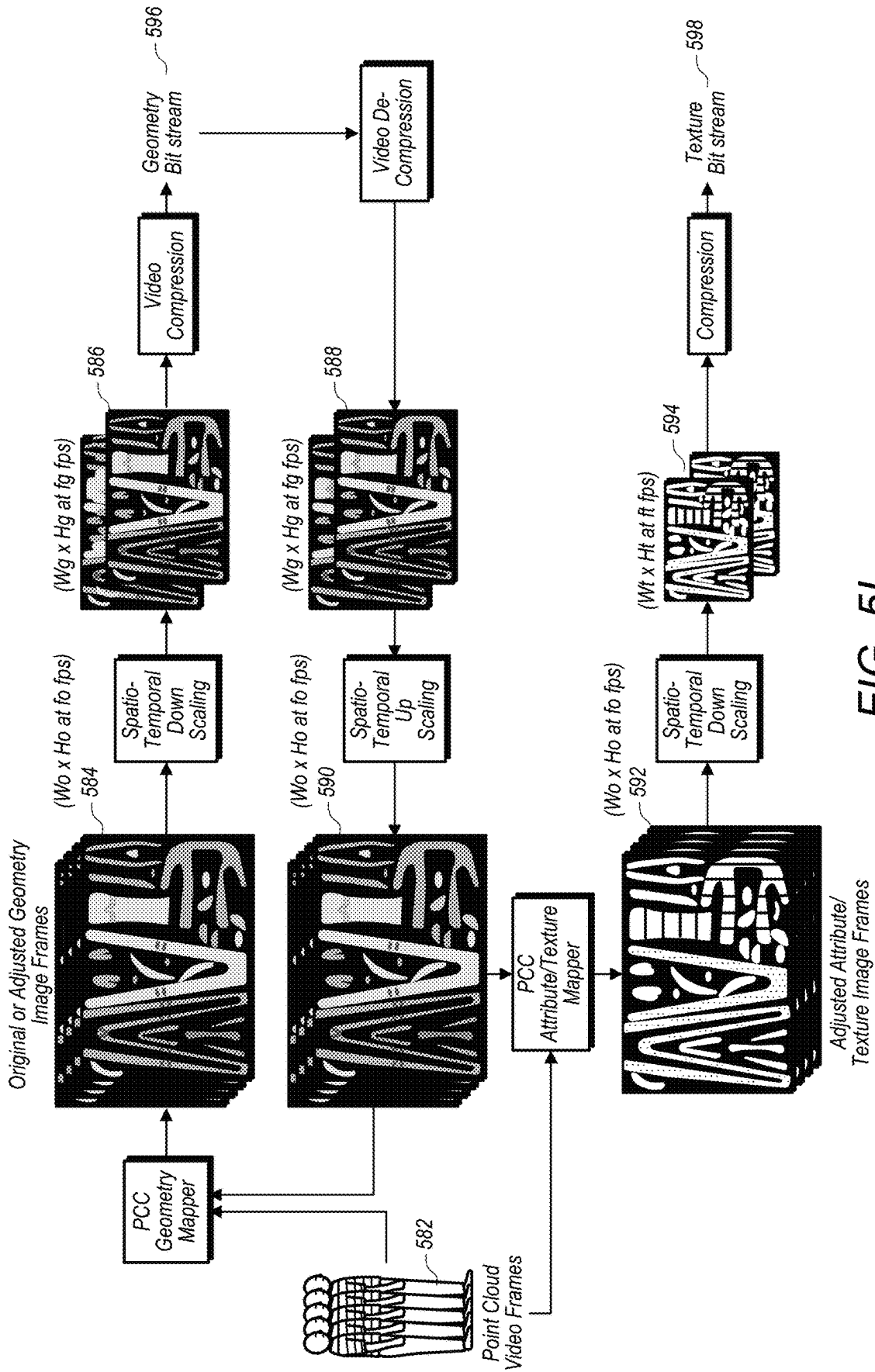
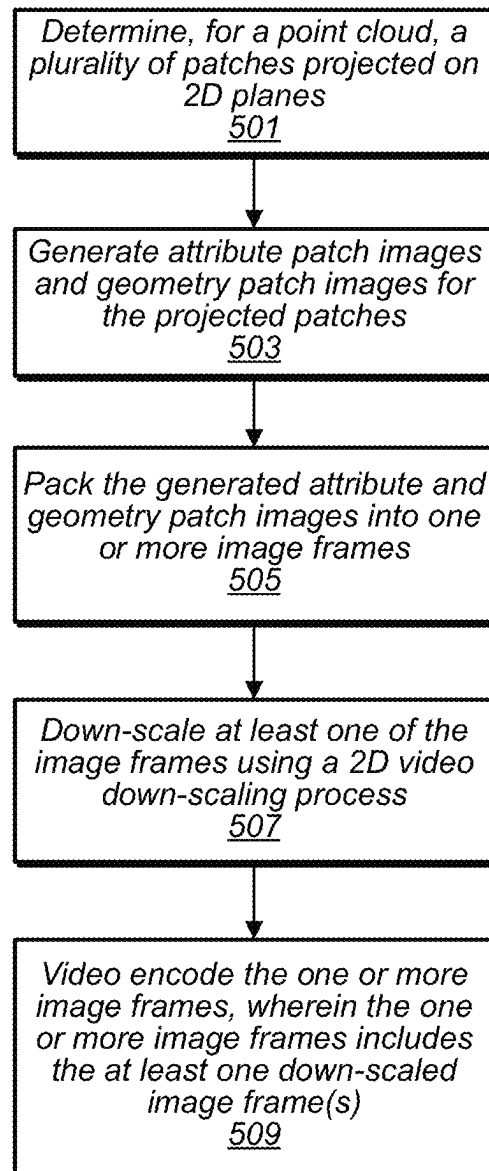


FIG. 5I

**FIG. 5J**

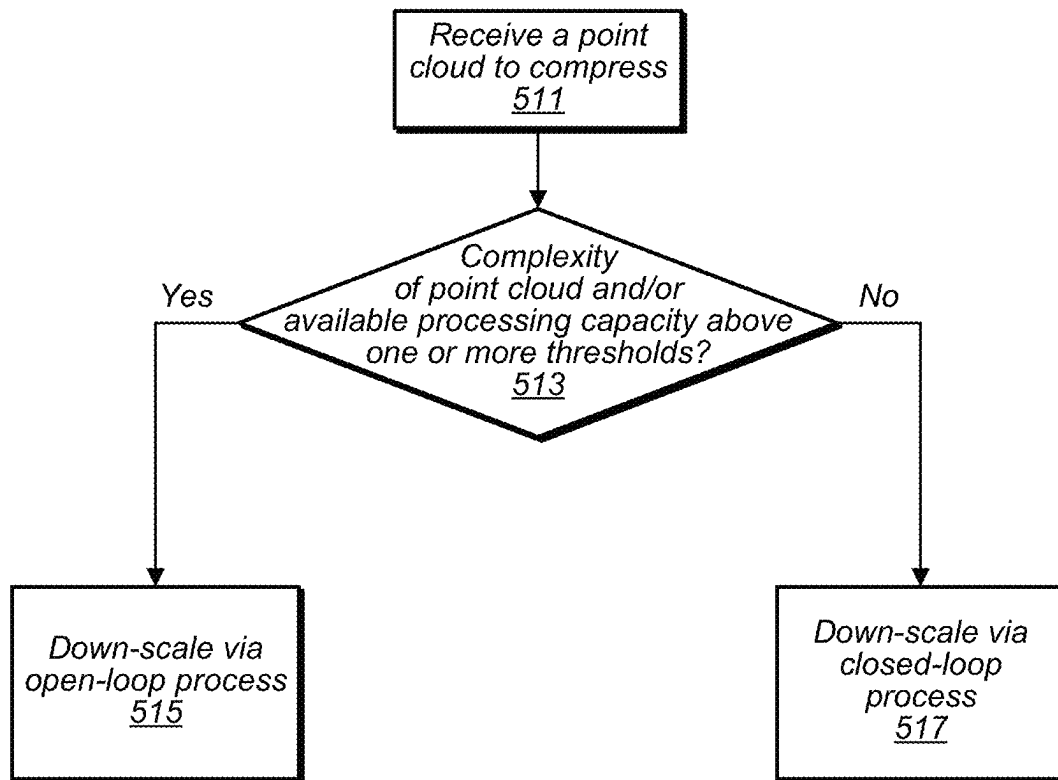
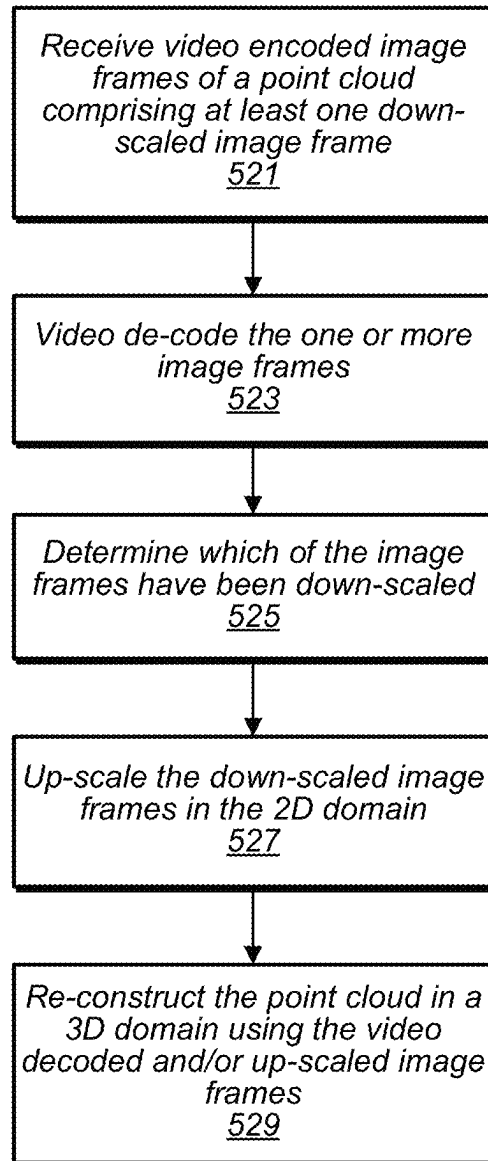


FIG. 5K

**FIG. 5L**

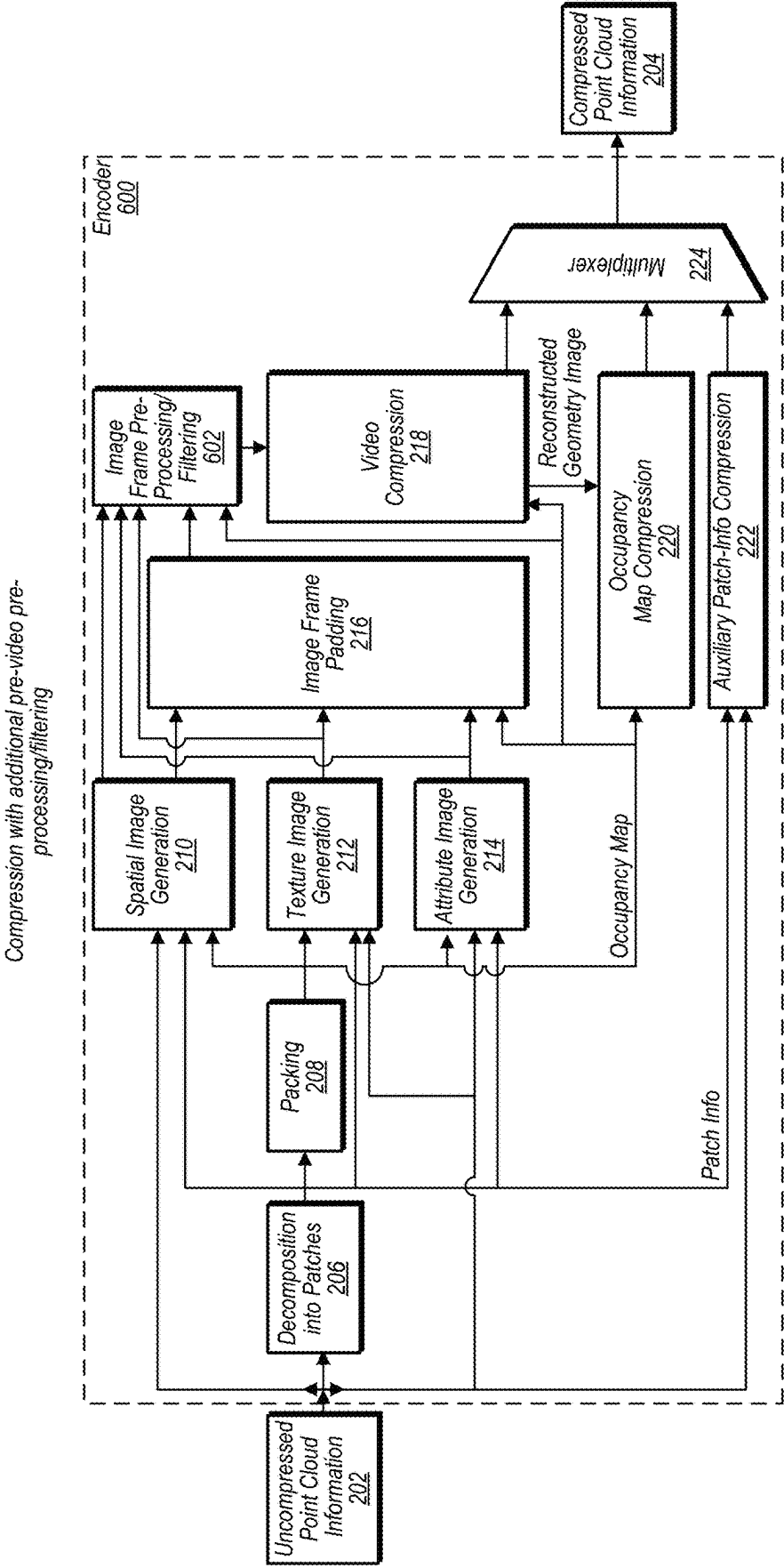


FIG.6A

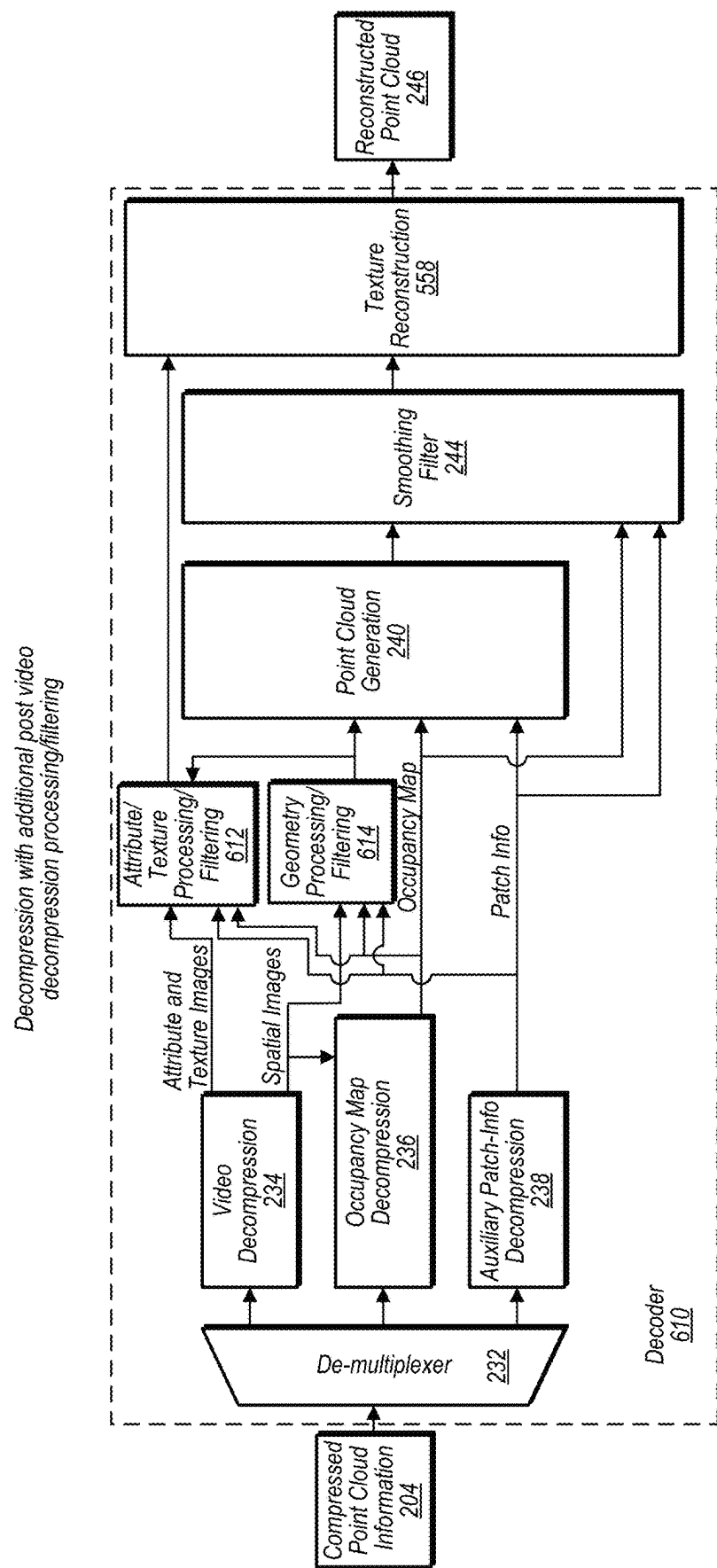


FIG. 6B

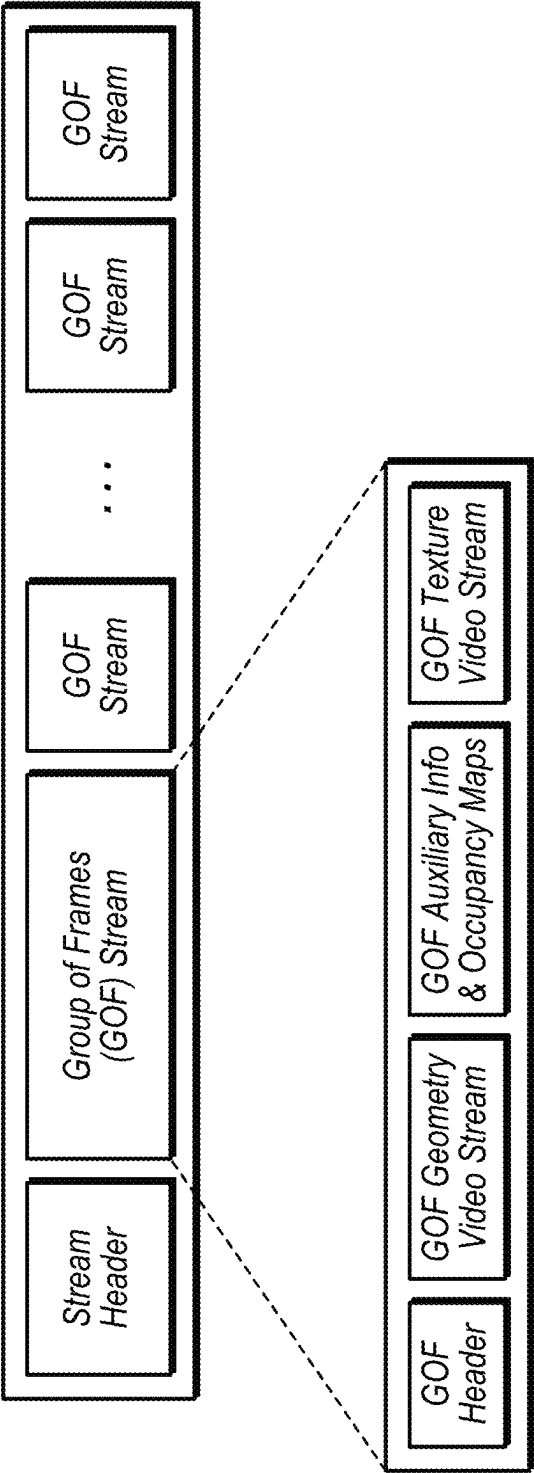
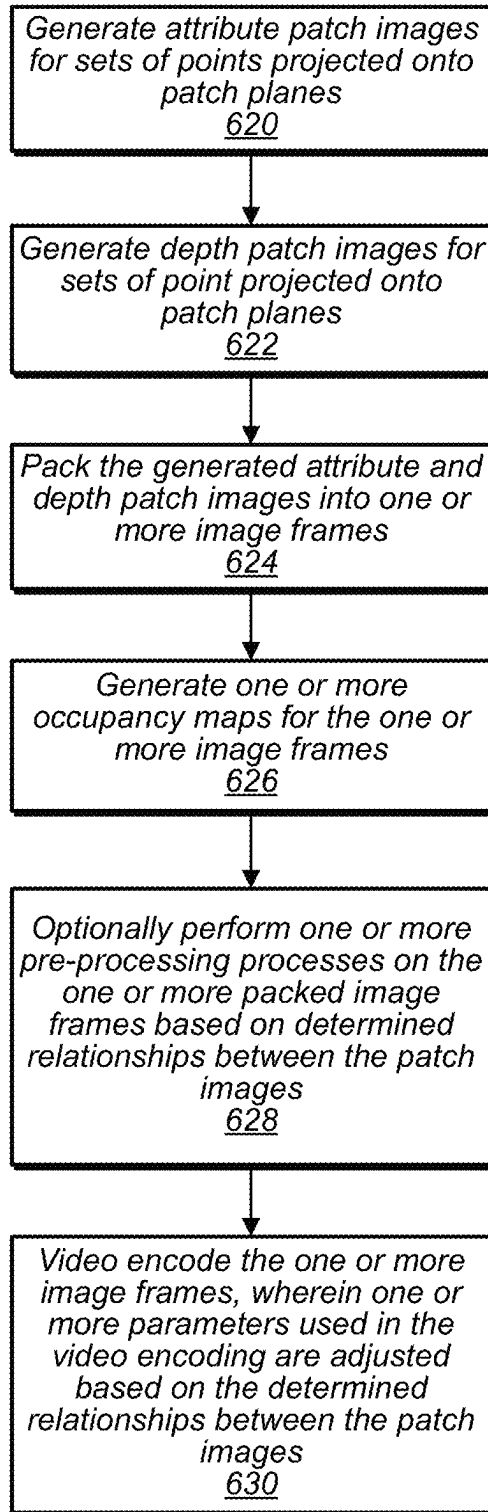


FIG. 6C

**FIG. 6D**

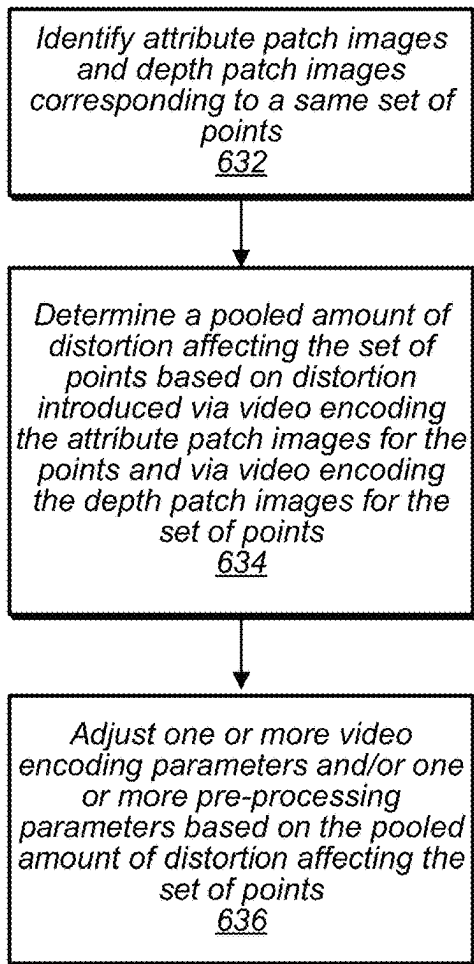


FIG. 6E

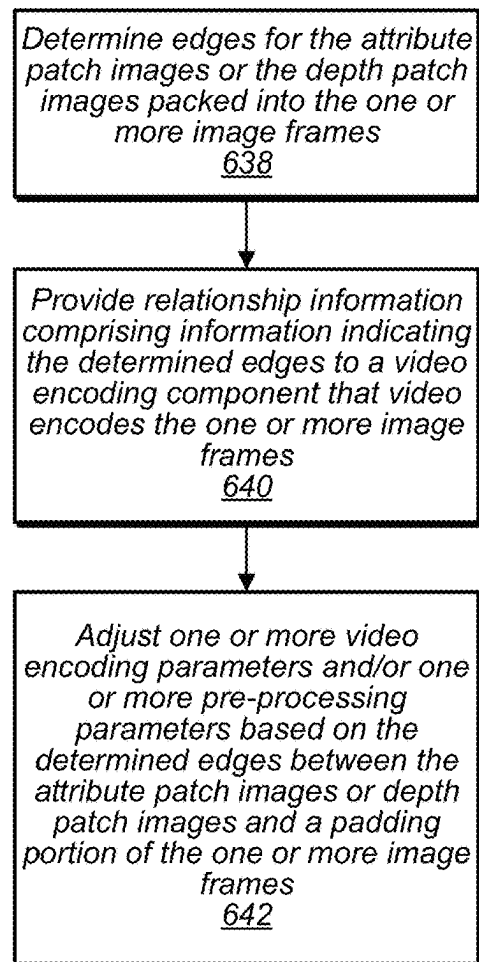


FIG. 6F

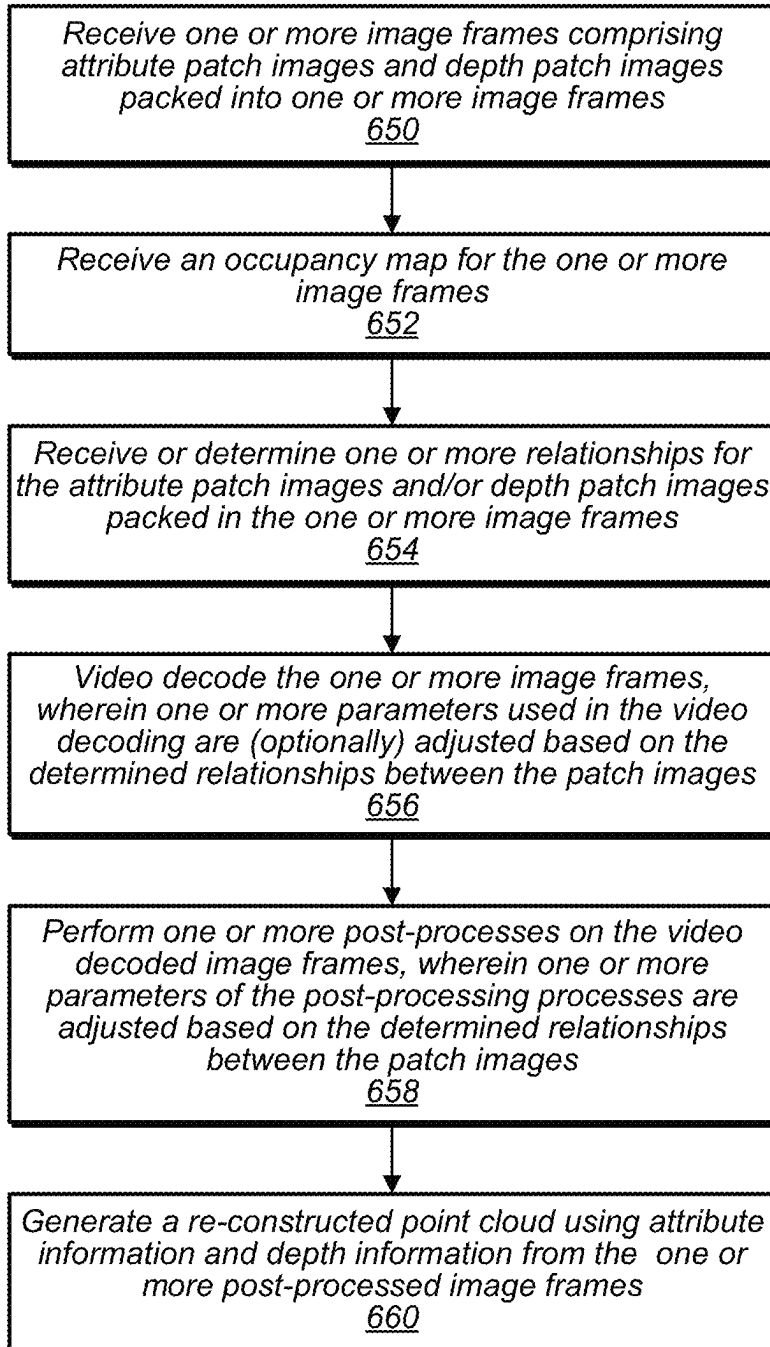
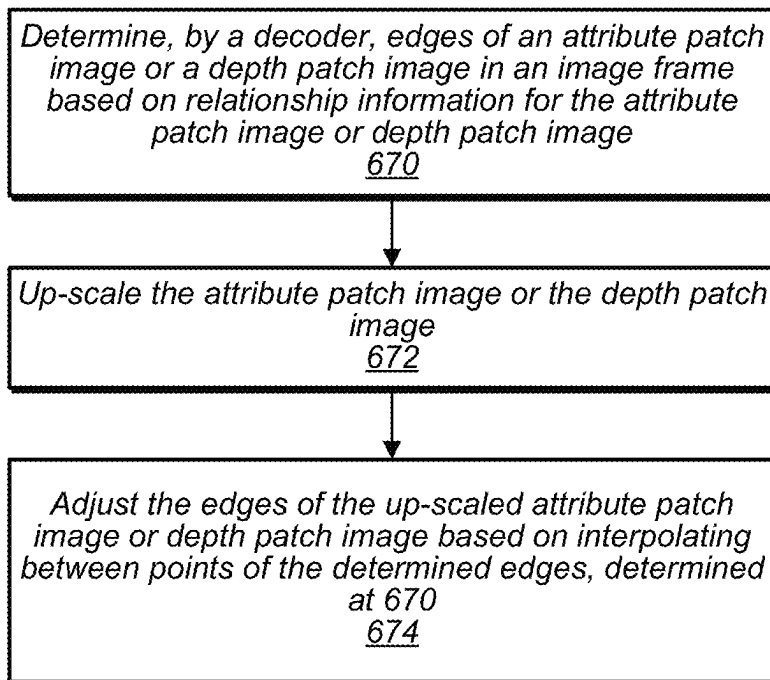
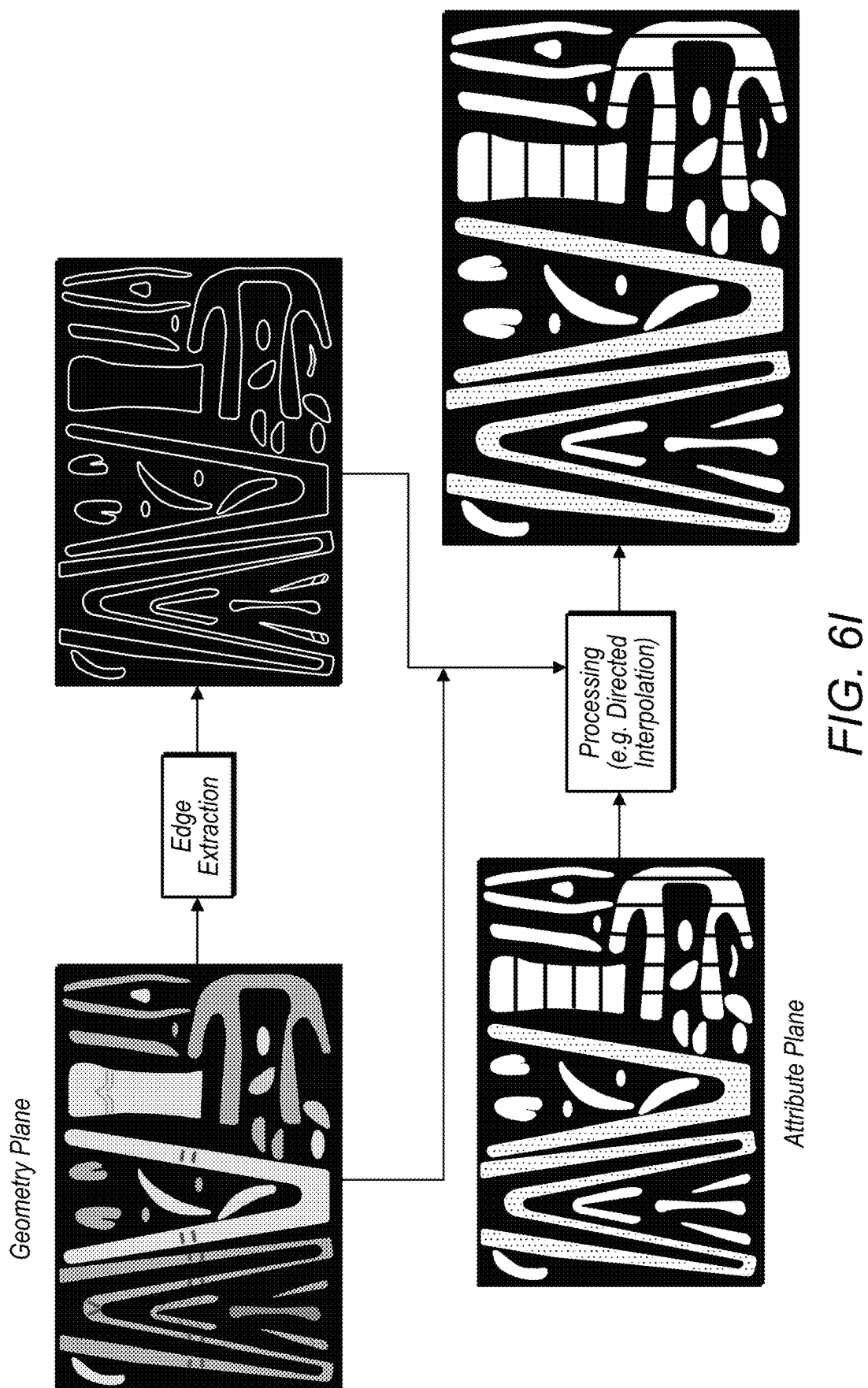


FIG. 6G

**FIG. 6H**



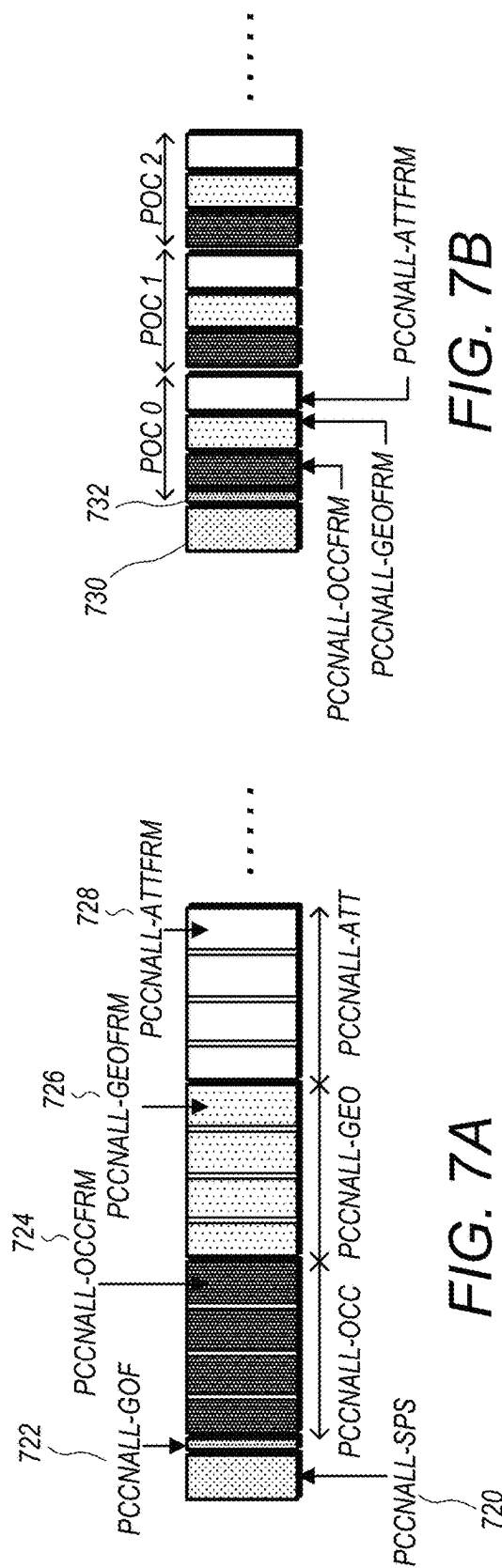


FIG. 7B

FIG. 7A

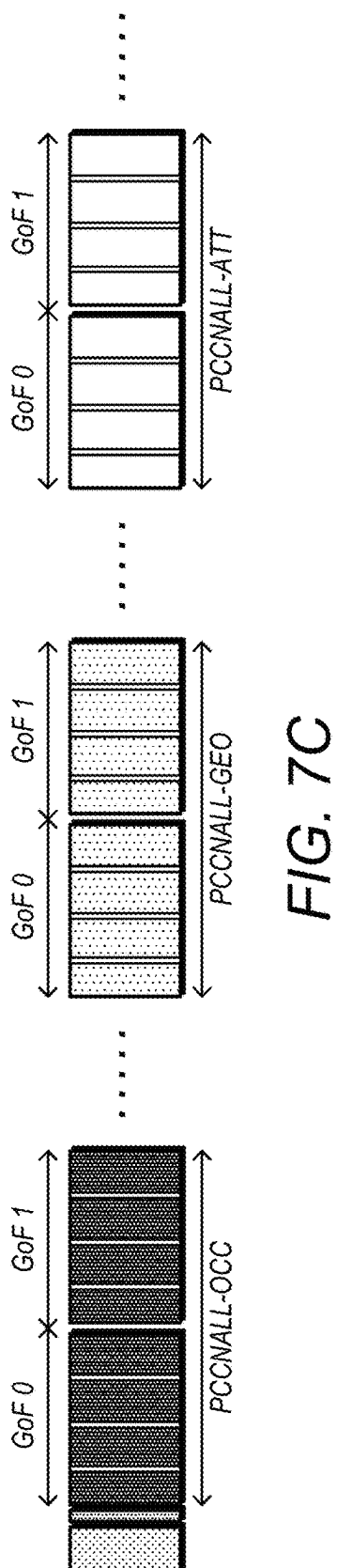


FIG. 7C

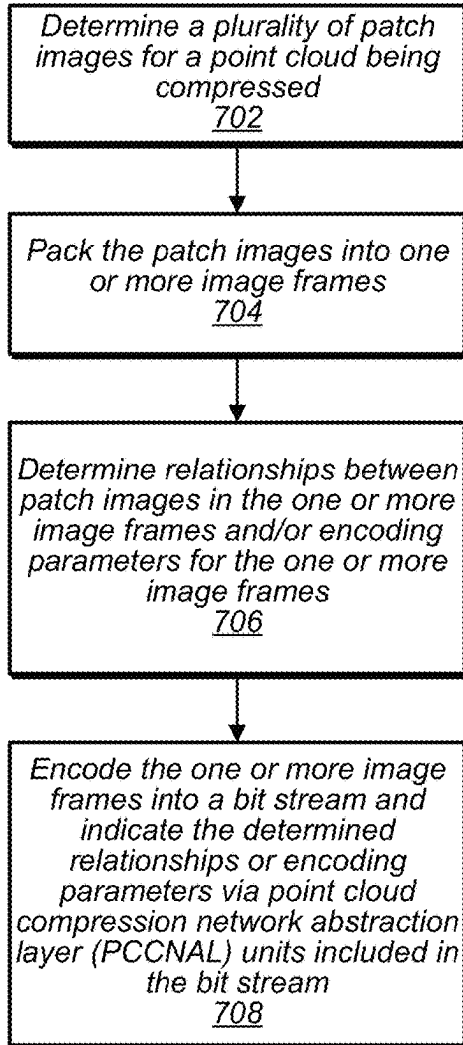


FIG. 7D

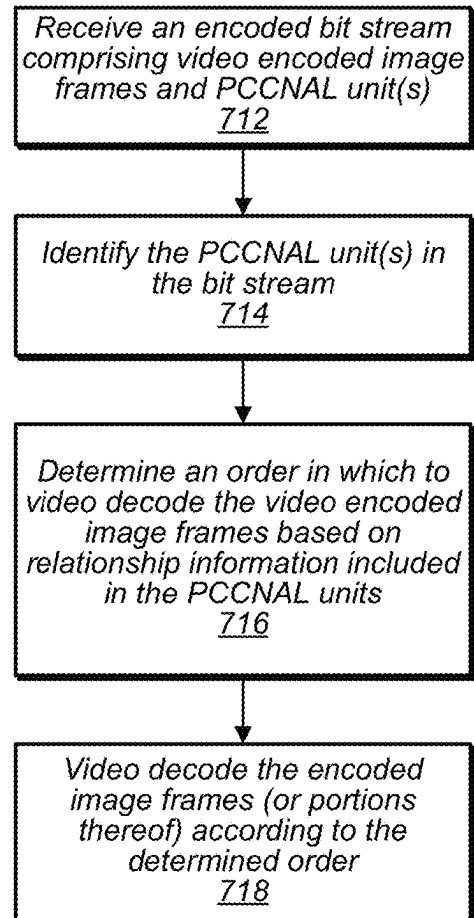


FIG. 7E

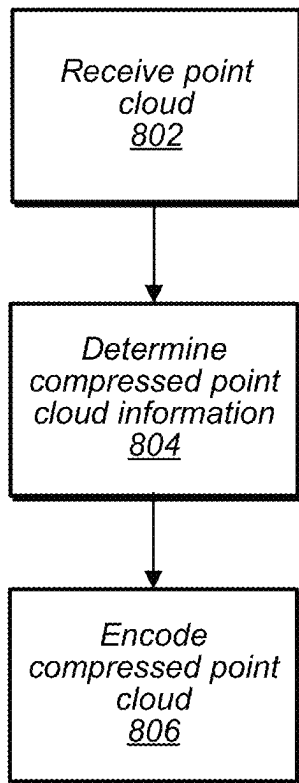


FIG. 8A

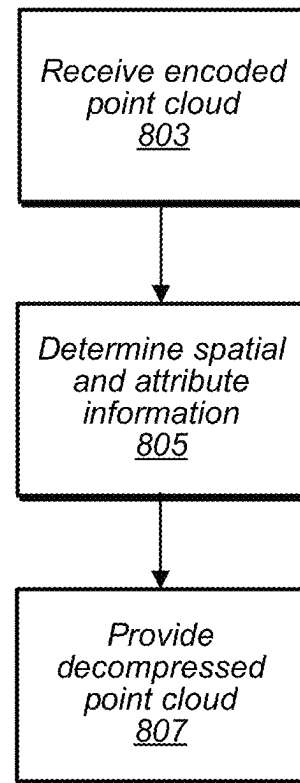


FIG. 8B

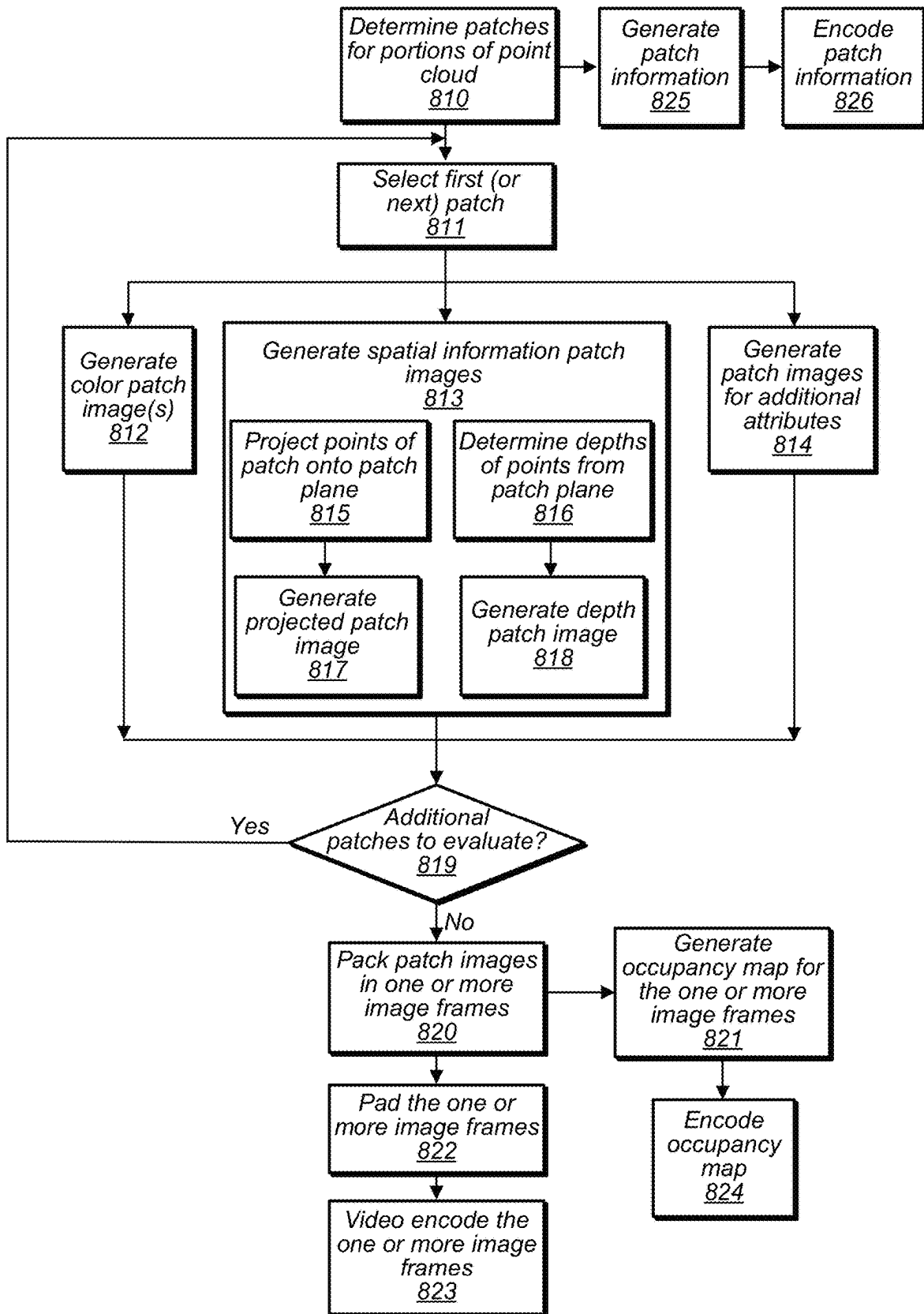


FIG. 8C

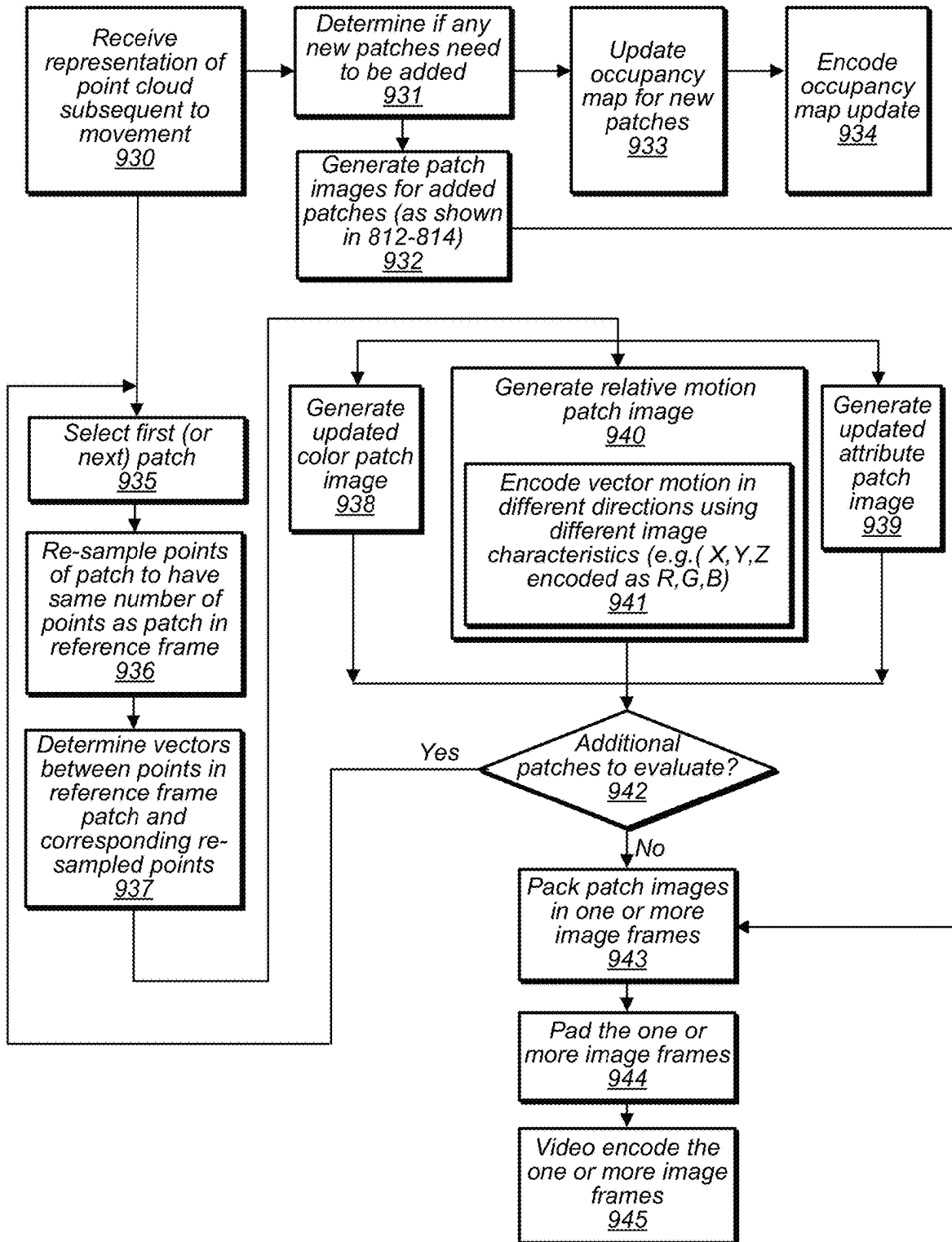


FIG. 9

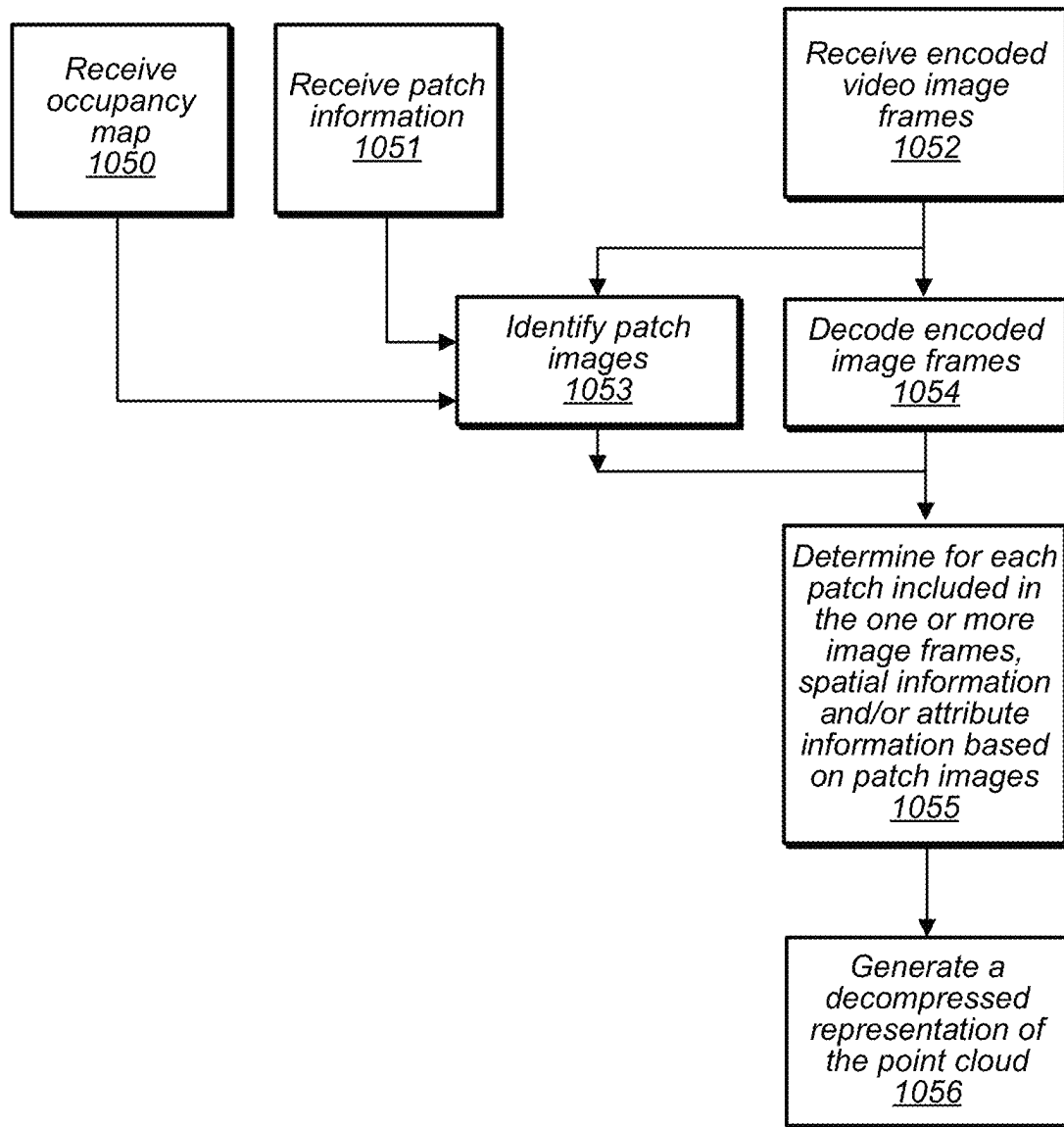


FIG. 10

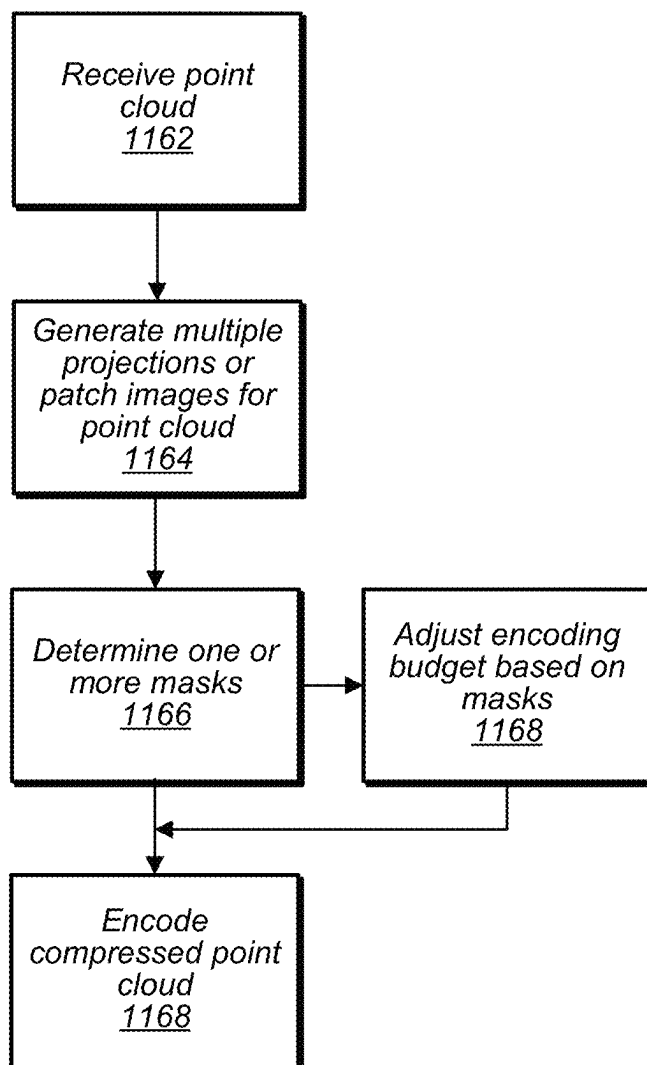


FIG. 11A

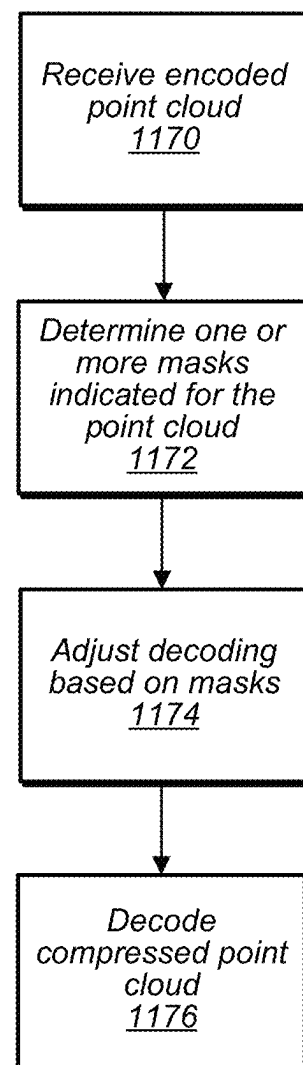


FIG. 11B

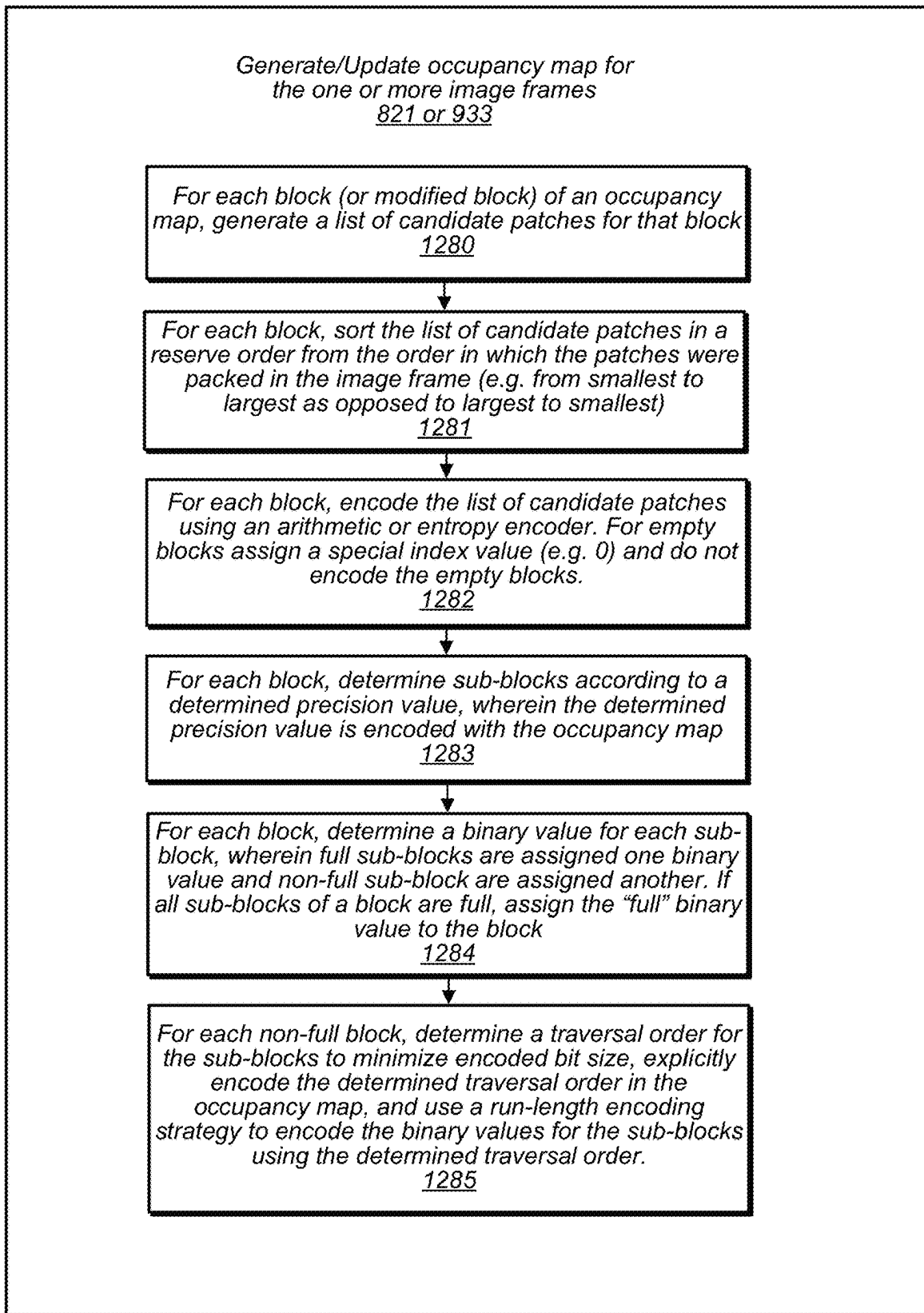
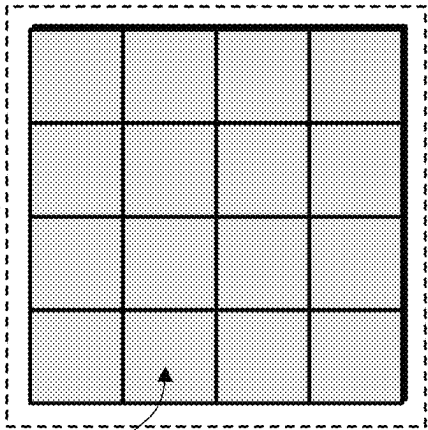


FIG. 12A

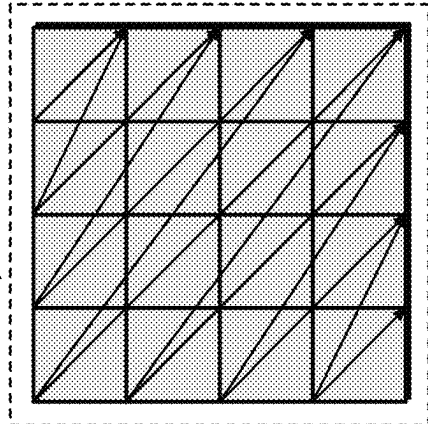
Example Block of
an Occupancy Map
1286



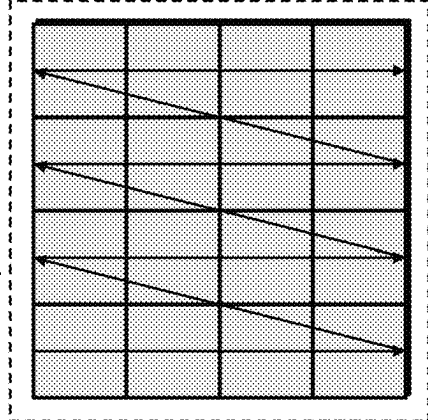
Example
Sub-Block
1287



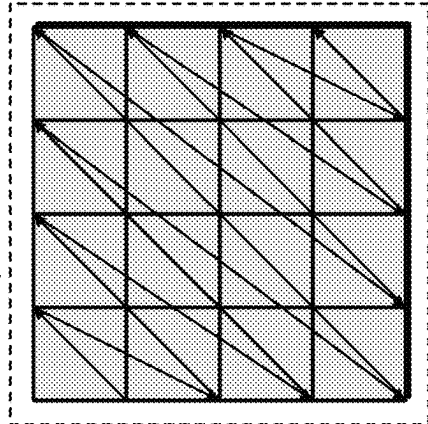
Example Traversal 1



Example Traversal 2



Example Traversal 3



Example Traversal 4

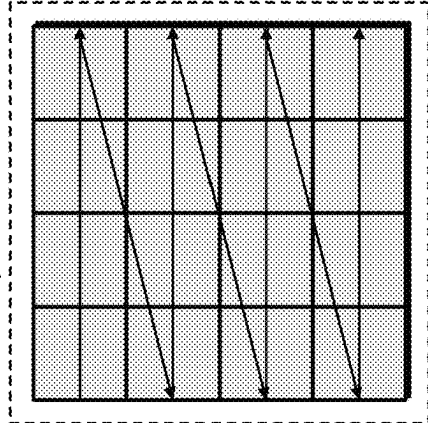


FIG. 12B

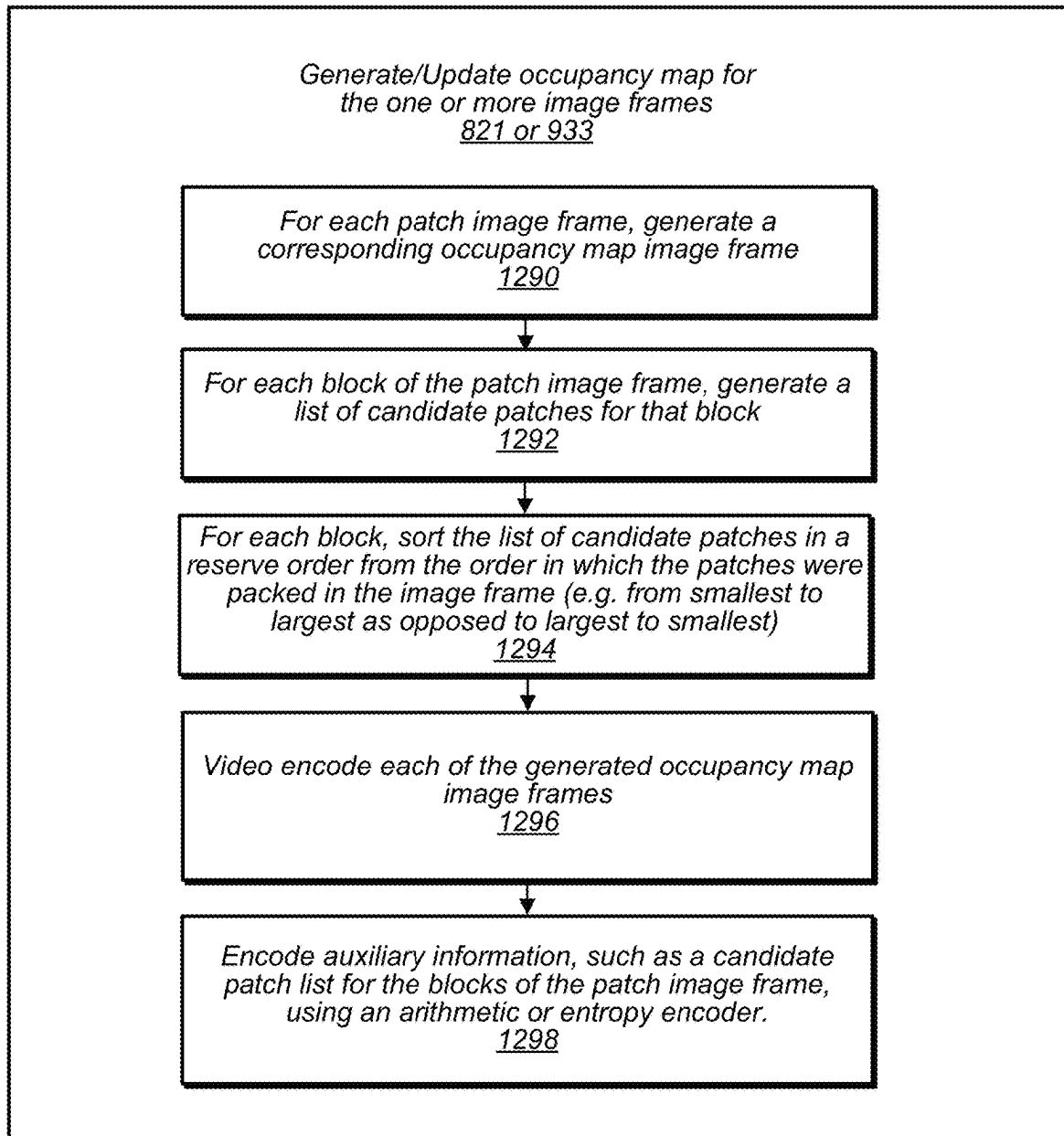
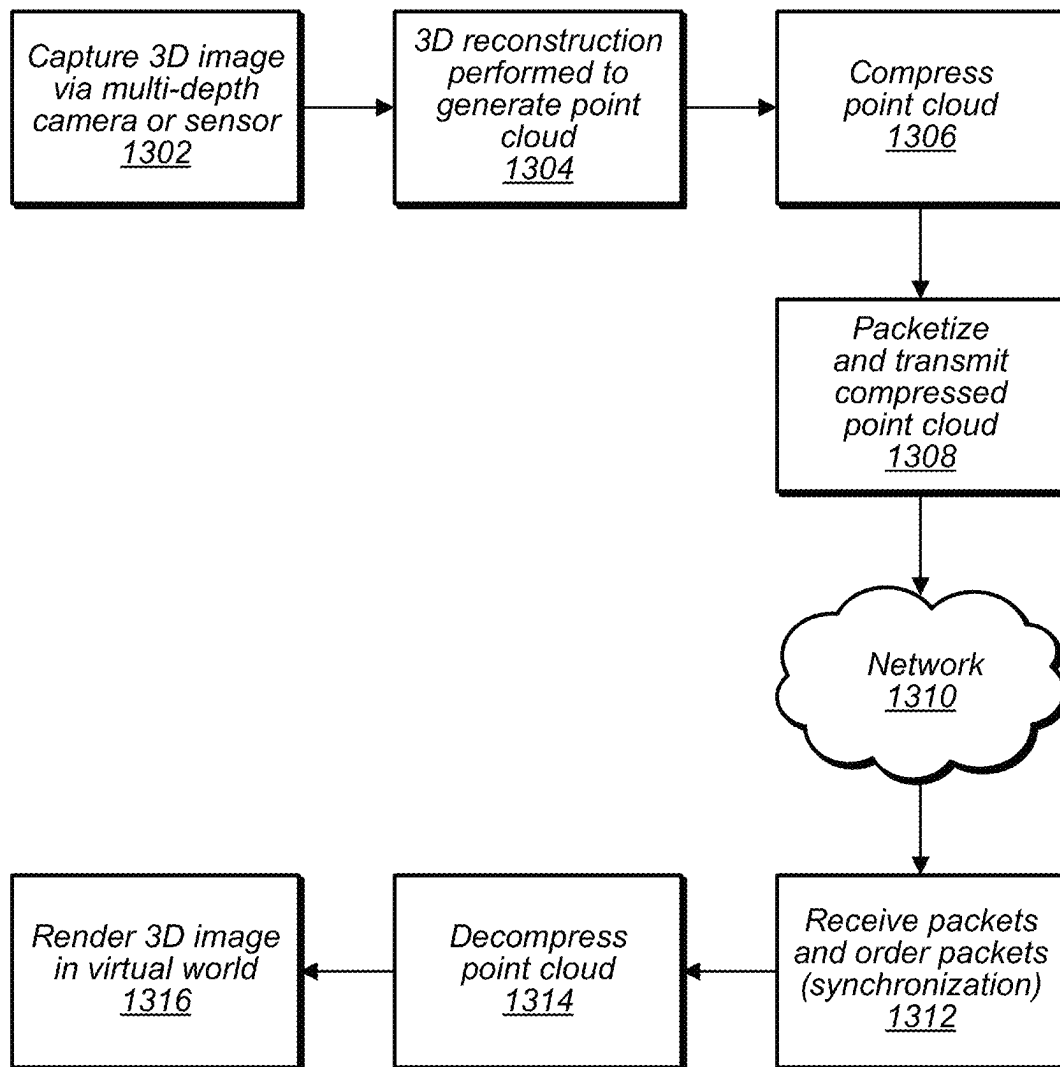


FIG. 12C

*FIG. 13*

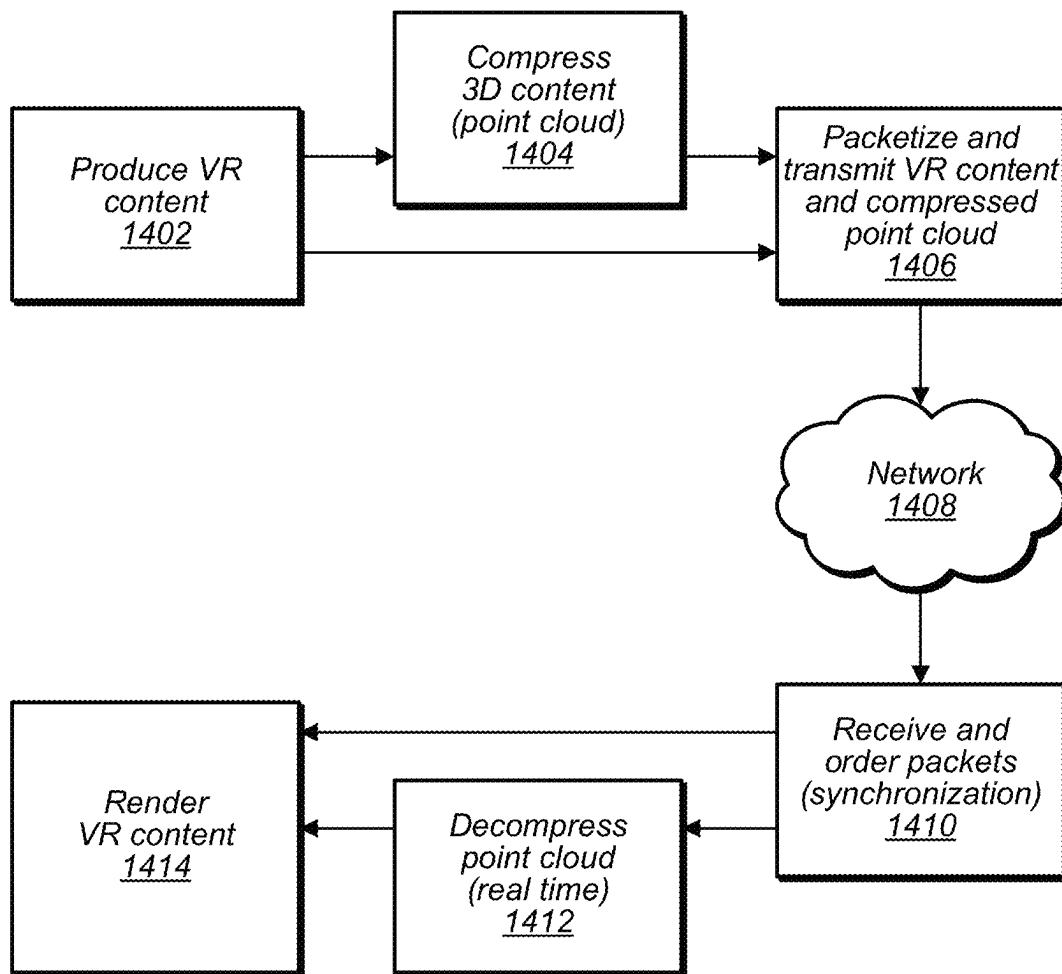
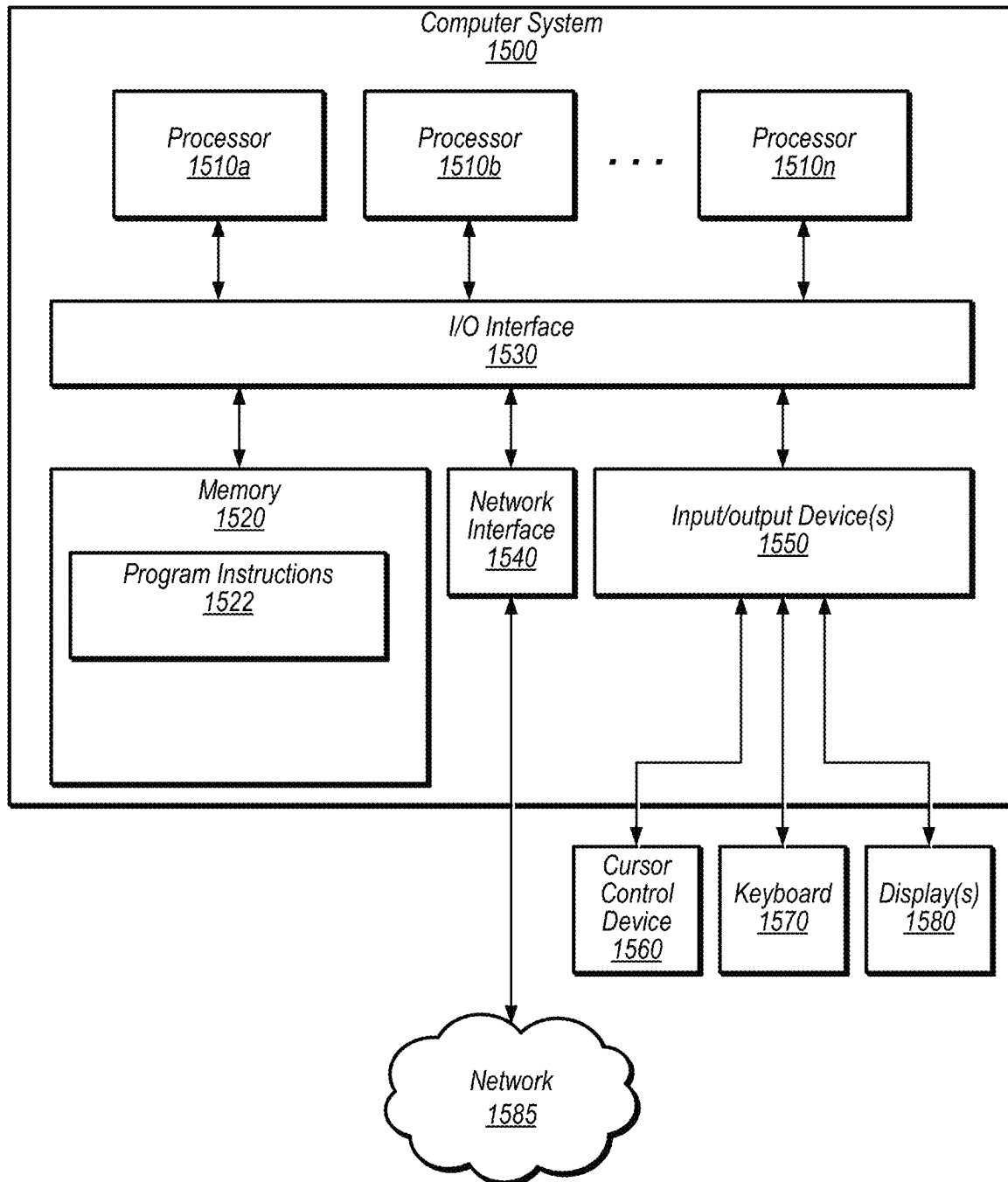


FIG. 14

**FIG. 15**

BIT STREAM STRUCTURE FOR COMPRESSED POINT CLOUD DATA

PRIORITY CLAIM

This application is a continuation of U.S. patent application Ser. No. 16/840,154, filed May 12, 2021, which is a continuation of U.S. patent application Ser. No. 16/510,706, filed Jul. 12, 2019, now U.S. Pat. No. 11,012,713, which claims benefit of priority to U.S. Provisional Application Ser. No. 62/697,369, filed Jul. 12, 2018, and which are incorporated herein by reference in their entirety.

BACKGROUND

Technical Field

This disclosure relates generally to compression and decompression of point clouds comprising a plurality of points, each having associated spatial information and attribute information.

Description of the Related Art

Various types of sensors, such as light detection and ranging (LIDAR) systems, 3-D-cameras, 3-D scanners, etc. may capture data indicating positions of points in three dimensional space, for example positions in the X, Y, and Z planes. Also, such systems may further capture attribute information in addition to spatial information for the respective points, such as color information (e.g. RGB values), texture information, intensity attributes, reflectivity attributes, motion related attributes, modality attributes, or various other attributes. In some circumstances, additional attributes may be assigned to the respective points, such as a time-stamp when the point was captured. Points captured by such sensors may make up a “point cloud” comprising a set of points each having associated spatial information and one or more associated attributes. In some circumstances, a point cloud may include thousands of points, hundreds of thousands of points, millions of points, or even more points. Also, in some circumstances, point clouds may be generated, for example in software, as opposed to being captured by one or more sensors. In either case, such point clouds may include large amounts of data and may be costly and time-consuming to store and transmit.

SUMMARY OF EMBODIMENTS

In some embodiments, a system includes one or more sensors configured to capture points that collectively make up a point cloud, wherein each of the points comprises spatial information identifying a spatial location of the respective point and attribute information defining one or more attributes associated with the respective point.

The system also includes an encoder configured to compress the attribute and/or spatial information of the points. To compress the attribute and/or spatial information, the encoder is configured to determine, for the point cloud, a plurality of patches, each corresponding to portions of the point cloud. The encoder is further configured to, for each patch, generate a patch image comprising the set of points corresponding to the patch projected onto a patch plane and generate another patch image comprising geometry information, such as depth information, for the set of points

corresponding to the patch, wherein the geometry information comprises depths of the points in a direction perpendicular to the patch plane.

For example, the geometry patch image corresponding to the patch projected onto a patch plane may depict the points of the point cloud included in the patch in two directions, such as an X and Y direction. The points of the point cloud may be projected onto a patch plane approximately perpendicular to a normal vector, normal to a surface of the point cloud at the location of the patch. Also, for example, the geometry patch image comprising depth information for the set of points included in the patch may depict depth information, such as depth distances in a Z direction. To depict the depth information, the geometry patch image may include a parameter that varies in intensity based on the depth of points in the point cloud at a particular location in the patch image. For example, the geometry patch image depicting depth information may have a same shape as the attribute patch image representing attributes of points projected onto the patch plane. However, the geometry information patch image may be an image comprising image attributes, such as one or more colors, that vary in intensity based on depth, wherein the intensity of the one or more image attributes corresponds to a depth of a corresponding point of the point cloud at a location in the geometry patch image where the image attribute is displayed in the geometry patch image depicting depth. For example, points that are closer to the patch plane may be encoded as darker values in the patch image depicting depth and points that are further away from the patch plane may be encoded as lighter values in the patch image depicting depth, for example in a monochromatic patch image depicting depth. Thus, the depth information patch image when aligned with other patch images representing attribute values for points projected onto the patch plane may indicate the relative depths of the points projected onto the patch plane, based on respective image attribute intensities at locations in the geometry patch image that correspond to locations of the points in the other patch images comprising point cloud points projected onto the patch plane.

The encoder is further configured to pack generated patch images (including a geometry patch image and one or more additional patch images for one or more other attributes such as colors, textures, reflectances, etc.) for each of the determined patches into one or more image frames. Also, the encoder is configured to provide the one or more packed image frames to a video encoding component (which may be included in the encoder or may be a separate video encoding component). Additionally, the encoder is configured to insert one or more point cloud compression network abstraction layer units (PCCNAL units) in a bit stream comprising the video encoded packed image frames.

In some embodiments, the PCCNAL units may be inserted anywhere in the bit stream and may include a sequence of leading bits, trailing bits, or both that make the PCCNAL units easily identifiable by a decoder. In some embodiments, the PCCNAL units may further include organizational information that enables a decoder to quickly identify and decode a given set of patch images packed into one or more of the video encoded image frames of the bit stream. For example, the PCCNAL units may enable a decoder to identify related patch images corresponding to a particular portion of a point cloud to enable low delay reconstruction of the particular portion of the point cloud, wherein necessary patch images needed to reconstruct the

portion of the point cloud are identified in the bit stream using the organization information included in the PCCNAL units.

In some embodiments, the encoder may utilize a video encoding component in accordance with the High Efficiency Video Coding (HEVC/H.265) standard or other suitable standards such as, the Advanced Video Coding (AVC/H.264) standard, the AOMedia Video 1 (AV1) video coding format produced by the Alliance for Open Media (AOM), etc. In some embodiments, the encoder may utilize an image encoder in accordance with a Motion Picture Experts Group (MPEG), a Joint Photography Experts Group (JPEG) standard, an International Telecommunication Union-Telecommunication standard (e.g. ITU-T standard), etc.

In some embodiments, a decoder is configured to receive one or more encoded image frames comprising patch images for a plurality of patches of a compressed point cloud, wherein, for each patch, the one or more encoded image frames comprise: a patch image comprising a set of points of the patch projected onto a patch plane and a patch image comprising depth information for the set of points of the patch, wherein the depth information indicates depths of the points of the patch in a direction perpendicular to the patch plane. In some embodiments, a depth patch image may be packed into an image frame with other attribute patch images. For example, a decoder may receive one or more image frames comprising packed patch images as generated by the encoder described above. A bit stream comprising the video encoded image frames may further included one or more point cloud compression network abstraction layer units (PCCNAL units). In some embodiments, a decoder may scan a bit stream for PCCNAL units and may further utilize organization information included in the PCCNAL units in order to determine which video encoded image frames or portions of the video encoded image frames to decoder, and in what order to decode them.

Furthermore, the decoder is further configured to video decode the one or more identified video encoded image frames comprising the patch images according to the determined order. In some embodiments, the decoder may utilize a video decoder in accordance with the High Efficiency Video Coding (HEVC) standard or other suitable standards such as, the Advanced Video Coding (AVC) standard, the AOMedia Video 1 (AV1) video coding format, etc. In some embodiments, the decoder may utilize an image decoder in accordance with a Motion Picture Experts Group (MPEG) or a Joint Photography Experts Group (JPEG) standard, etc.

The decoder is further configured to determine, for each patch, spatial information for the set of points of the patch based, at least in part, on the attribute patch image comprising the set of points of the patch projected onto the patch plane and the geometry patch image comprising the depth information for the set of points of the patch, and generate a reconstructed version of the compressed point cloud based, at least in part, on the determined spatial information for the plurality of patches and the attribute information included in the patches.

In some embodiments, a method includes receiving data for a point cloud comprising a plurality of points that make up the point cloud, wherein respective ones of the points comprise spatial information for the point and attribute information for the point. The method further includes compressing the point cloud data, wherein compressing the point cloud data comprises determining, for the point cloud, a plurality of patches each corresponding to portions of the point cloud and packing generated patch images for the determined patches into one or more image frames. Also,

compression the point cloud data comprises encoding the one or more image frames into a bit stream comprising point cloud compression network abstraction layer (PCCNAL) units, wherein the PCCNAL units indicate locations of one or more of the patch images in the bit stream.

In some embodiments, a method includes receiving an encoded bit stream comprising video encoded image frames comprising patch images packed into respective ones of the video encoded image frames and comprising one or more point cloud compression network abstraction layer (PCCNAL) units in the bit stream. The method also includes identifying one or more of the PCCNAL units in the bit stream and determining an order in which to video decode at least some of the patch images packed into the video encoded image frames based on information indicated in the one or more PCCNAL units. Additionally, the method includes video decoding a set of video encoded image frames comprising the at least some patch images according to the order determined based on the information indicated in the one or more PCCNAL units.

In some embodiments, a non-transitory computer-readable medium stores program instructions that, when executed by one or more processors, cause the one or more processors to implement an encoder as described herein to compress geometry and attribute information of a point cloud.

In some embodiments, a non-transitory computer-readable medium stores program instructions that, when executed by one or more processors, cause the one or more processors to implement a decoder as described herein to decompress geometry and attribute information of a point cloud.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 illustrates a system comprising a sensor that captures information for points of a point cloud and an encoder that compresses spatial information and attribute information of the point cloud, where the compressed spatial and attribute information is sent to a decoder, according to some embodiments.

FIG. 2A illustrates components of an encoder for encoding intra point cloud frames, according to some embodiments.

FIG. 2B illustrates components of a decoder for decoding intra point cloud frames, according to some embodiments.

FIG. 2C illustrates components of an encoder for encoding inter point cloud frames, according to some embodiments.

FIG. 2D illustrates components of a decoder for decoding inter point cloud frames, according to some embodiments.

FIG. 3A illustrates an example patch segmentation process, according to some embodiments.

FIG. 3B illustrates an example image frame comprising packed patch images and padded portions, according to some embodiments.

FIG. 3C illustrates an example image frame comprising patch portions and padded portions, according to some embodiments.

FIG. 3D illustrates a point cloud being projected onto multiple projections, according to some embodiments.

FIG. 3E illustrates a point cloud being projected onto multiple parallel projections, according to some embodiments.

FIG. 4A illustrates components of an encoder for encoding intra point cloud frames with color conversion, according to some embodiments.

5

FIG. 4B illustrates components of an encoder for encoding inter point cloud frames with color conversion, according to some embodiments.

FIG. 4C illustrates components of a closed-loop color conversion module, according to some embodiments.

FIG. 4D illustrates an example process for determining a quality metric for a point cloud upon which an operation has been performed, according to some embodiments.

FIG. 5A illustrates components of an encoder that includes geometry, texture, and/or other attribute downscaling, according to some embodiments.

FIG. 5B illustrates components of a decoder that includes geometry, texture, and/or other attribute upscaling, according to some embodiments.

FIG. 5C illustrates rescaling from the perspective of an encoder, according to some embodiments.

FIG. 5D illustrates rescaling from the perspective of a decoder, according to some embodiments.

FIG. 5E illustrates an example open loop rescaling, according to some embodiments.

FIG. 5F illustrates an example closed loop rescaling, according to some embodiments.

FIG. 5G illustrates an example closed loop rescaling with multiple attribute layers, according to some embodiments.

FIG. 5H illustrates an example of video level spatiotemporal scaling, according to some embodiments.

FIG. 5I illustrates an example closed loop rescaling with spatiotemporal scaling, according to some embodiments.

FIG. 5J illustrates a process of encoding/compressing image frames of a point cloud using down-scaling, according to some embodiments.

FIG. 5K illustrates a process of determining to down-scaling image frames using open-loop or closed-loop down-scaling, according to some embodiments.

FIG. 5L illustrates a process of decoding/decompressing image frames of a point cloud using up-scaling, according to some embodiments.

FIG. 6A illustrates components of an encoder that further includes pre-video compression texture processing and/or filtering and pre video compression geometry processing/filtering, according to some embodiments.

FIG. 6B illustrates components of a decoder that further includes post video decompression texture processing and/or filtering and post video decompression geometry processing/filtering, according to some embodiments.

FIG. 6C illustrates, a bit stream structure for a compressed point cloud, according to some embodiments.

FIG. 6D illustrates a process for generating video encoded image frames for patches of a point cloud taking into account relationship information between the patches packed into the image frames, according to some embodiments.

FIG. 6E illustrates a process for generating video encoded image frames taking into account pooled distortion for a set of patches corresponding to a same set of points, according to some embodiments.

FIG. 6F illustrates a process for generating video encoded image frames taking into account patch edges, according to some embodiments.

FIG. 6G illustrates a process for reconstructing a point cloud based on video encoded image frames comprising patches of the point cloud, wherein relationship information between the patches packed into the image frames is taken into account, according to some embodiments.

FIG. 6H illustrates a process of upscaling a patch image included in an image frame taking into account edges of the

6

patch image determined based on received or determined relationship information for the patches, according to some embodiments.

FIG. 6I illustrates an example application where an attribute plane is up-scaled using its corresponding geometry information and the geometry extracted edges, according to some embodiments.

FIG. 7A illustrates an example of a point cloud compression network abstraction layer (PCCNAL) unit based bit stream, according to some embodiments.

FIG. 7B illustrates an example of a PCCNAL units grouped by picture order count (POC), according to some embodiments.

FIG. 7C illustrates an example of a PCCNAL unit grouped by type, according to some embodiments.

FIG. 7D illustrates a process of encoding a bit stream that includes PCCNAL units, according to some embodiments.

FIG. 7E illustrates a process of decoding a bit stream that includes PCCNAL units, according to some embodiments.

FIG. 8A illustrates a process for compressing attribute and spatial information of a point cloud, according to some embodiments.

FIG. 8B illustrates a process for decompressing attribute and spatial information of a point cloud, according to some embodiments.

FIG. 8C illustrates patch images being generated and packed into an image frame to compress attribute and spatial information of a point cloud, according to some embodiments.

FIG. 9 illustrates patch images being generated and packed into an image frame to compress attribute and spatial information of a moving or changing point cloud, according to some embodiments.

FIG. 10 illustrates a decoder receiving image frames comprising patch images, patch information, and an occupancy map, and generating a decompressed representation of a point cloud, according to some embodiments.

FIG. 11A illustrates an encoder, adjusting encoding based on one or more masks for a point cloud, according to some embodiments.

FIG. 11B illustrates a decoder, adjusting decoding based on one or more masks for a point cloud, according to some embodiments.

FIG. 12A illustrates more detail regarding compression of an occupancy map, according to some embodiments.

FIG. 12B illustrates example blocks and traversal patterns for compressing an occupancy map, according to some embodiments.

FIG. 12C illustrates more detail regarding compression of an occupancy map, according to some embodiments.

FIG. 13 illustrates compressed point cloud information being used in a 3-D telepresence application, according to some embodiments.

FIG. 14 illustrates compressed point cloud information being used in a virtual reality application, according to some embodiments.

FIG. 15 illustrates an example computer system that may implement an encoder or decoder, according to some embodiments.

This specification includes references to “one embodiment” or “an embodiment.” The appearances of the phrases “in one embodiment” or “in an embodiment” do not necessarily refer to the same embodiment. Particular features, structures, or characteristics may be combined in any suitable manner consistent with this disclosure.

“Comprising.” This term is open-ended. As used in the appended claims, this term does not foreclose additional

structure or steps. Consider a claim that recites: “An apparatus comprising one or more processor units” Such a claim does not foreclose the apparatus from including additional components (e.g., a network interface unit, graphics circuitry, etc.).

“Configured To.” Various units, circuits, or other components may be described or claimed as “configured to” perform a task or tasks. In such contexts, “configured to” is used to connote structure by indicating that the units/circuits/components include structure (e.g., circuitry) that performs those task or tasks during operation. As such, the unit/circuit/component can be said to be configured to perform the task even when the specified unit/circuit/component is not currently operational (e.g., is not on). The units/circuits/components used with the “configured to” language include hardware—for example, circuits, memory storing program instructions executable to implement the operation, etc. Reciting that a unit/circuit/component is “configured to” perform one or more tasks is expressly intended not to invoke 35 U.S.C. § 112(f), for that unit/circuit/component. Additionally, “configured to” can include generic structure (e.g., generic circuitry) that is manipulated by software and/or firmware (e.g., an FPGA or a general-purpose processor executing software) to operate in manner that is capable of performing the task(s) at issue. “Configure to” may also include adapting a manufacturing process (e.g., a semiconductor fabrication facility) to fabricate devices (e.g., integrated circuits) that are adapted to implement or perform one or more tasks.

“First,” “Second,” etc. As used herein, these terms are used as labels for nouns that they precede, and do not imply any type of ordering (e.g., spatial, temporal, logical, etc.). For example, a buffer circuit may be described herein as performing write operations for “first” and “second” values. The terms “first” and “second” do not necessarily imply that the first value must be written before the second value.

“Based On.” As used herein, this term is used to describe one or more factors that affect a determination. This term does not foreclose additional factors that may affect a determination. That is, a determination may be solely based on those factors or based, at least in part, on those factors. Consider the phrase “determine A based on B.” While in this case, B is a factor that affects the determination of A, such a phrase does not foreclose the determination of A from also being based on C. In other instances, A may be determined based solely on B.

DETAILED DESCRIPTION

As data acquisition and display technologies have become more advanced, the ability to capture point clouds comprising thousands or millions of points in 2-D or 3-D space, such as via LIDAR systems, has increased. Also, the development of advanced display technologies, such as virtual reality or augmented reality systems, has increased potential uses for point clouds. However, point cloud files are often very large and may be costly and time-consuming to store and transmit. For example, communication of point clouds over private or public networks, such as the Internet, may require considerable amounts of time and/or network resources, such that some uses of point cloud data, such as real-time uses, may be limited. Also, storage requirements of point cloud files may consume a significant amount of storage capacity of devices storing the point cloud files, which may also limit potential applications for using point cloud data.

In some embodiments, an encoder may be used to generate a compressed point cloud to reduce costs and time

associated with storing and transmitting large point cloud files. In some embodiments, a system may include an encoder that compresses attribute and/or spatial information of a point cloud file such that the point cloud file may be stored and transmitted more quickly than non-compressed point clouds and in a manner that the point cloud file may occupy less storage space than non-compressed point clouds. In some embodiments, compression of attributes of points in a point cloud may enable a point cloud to be communicated over a network in real-time or in near real-time. For example, a system may include a sensor that captures attribute information about points in an environment where the sensor is located, wherein the captured points and corresponding attributes make up a point cloud. The system may also include an encoder that compresses the captured point cloud attribute information. The compressed attribute information of the point cloud may be sent over a network in real-time or near real-time to a decoder that decompresses the compressed attribute information of the point cloud. The decompressed point cloud may be further processed, for example to make a control decision based on the surrounding environment at the location of the sensor. The control decision may then be communicated back to a device at or near the location of the sensor, wherein the device receiving the control decision implements the control decision in real-time or near real-time. In some embodiments, the decoder may be associated with an augmented reality system and the decompressed attribute information may be displayed or otherwise used by the augmented reality system. In some embodiments, compressed attribute information for a point cloud may be sent with compressed spatial information for points of the point cloud. In other embodiments, spatial information and attribute information may be separately encoded and/or separately transmitted to a decoder.

In some embodiments, a system may include a decoder that receives one or more sets of point cloud data comprising compressed attribute information via a network from a remote server or other storage device that stores the one or more point cloud files. For example, a 3-D display, a holographic display, or a head-mounted display may be manipulated in real-time or near real-time to show different portions of a virtual world represented by point clouds. In order to update the 3-D display, the holographic display, or the head-mounted display, a system associated with the decoder may request point cloud data from the remote server based on user manipulations of the displays, and the point cloud data may be transmitted from the remote server to the decoder and decoded by the decoder in real-time or near real-time. The displays may then be updated with updated point cloud data responsive to the user manipulations, such as updated point attributes.

In some embodiments, a system, may include one or more LIDAR systems, 3-D cameras, 3-D scanners, etc., and such sensor devices may capture spatial information, such as X, Y, and Z coordinates for points in a view of the sensor devices. In some embodiments, the spatial information may be relative to a local coordinate system or may be relative to a global coordinate system (for example, a Cartesian coordinate system may have a fixed reference point, such as a fixed point on the earth, or may have a non-fixed local reference point, such as a sensor location).

In some embodiments, such sensors may also capture attribute information for one or more points, such as color attributes, texture attributes, reflectivity attributes, velocity attributes, acceleration attributes, time attributes, modalities, and/or various other attributes. In some embodiments, other

sensors, in addition to LIDAR systems, 3-D cameras, 3-D scanners, etc., may capture attribute information to be included in a point cloud. For example, in some embodiments, a gyroscope or accelerometer, may capture motion information to be included in a point cloud as an attribute associated with one or more points of the point cloud. For example, a vehicle equipped with a LIDAR system, a 3-D camera, or a 3-D scanner may include the vehicle's direction and speed in a point cloud captured by the LIDAR system, the 3-D camera, or the 3-D scanner. For example, when points in a view of the vehicle are captured they may be included in a point cloud, wherein the point cloud includes the captured points and associated motion information corresponding to a state of the vehicle when the points were captured.

Example System Arrangement

FIG. 1 illustrates a system comprising a sensor that captures information for points of a point cloud and an encoder that compresses attribute information of the point cloud, where the compressed attribute information is sent to a decoder, according to some embodiments.

System 100 includes sensor 102 and encoder 104. Sensor 102 captures a point cloud 110 comprising points representing structure 106 in view 108 of sensor 102. For example, in some embodiments, structure 106 may be a mountain range, a building, a sign, an environment surrounding a street, or any other type of structure. In some embodiments, a captured point cloud, such as captured point cloud 110, may include spatial and attribute information for the points included in the point cloud. For example, point A of captured point cloud 110 comprises X, Y, Z coordinates and attributes 1, 2, and 3. In some embodiments, attributes of a point may include attributes such as R, G, B color values, a velocity at the point, an acceleration at the point, a reflectance of the structure at the point, a time stamp indicating when the point was captured, a string-value indicating a modality when the point was captured, for example "walking", or other attributes. The captured point cloud 110 may be provided to encoder 104, wherein encoder 104 generates a compressed version of the point cloud (compressed attribute information 112) that is transmitted via network 114 to decoder 116. In some embodiments, a compressed version of the point cloud, such as compressed attribute information 112, may be included in a common compressed point cloud that also includes compressed spatial information for the points of the point cloud or, in some embodiments, compressed spatial information and compressed attribute information may be communicated as separate sets of data.

In some embodiments, encoder 104 may be integrated with sensor 102. For example, encoder 104 may be implemented in hardware or software included in a sensor device, such as sensor 102. In other embodiments, encoder 104 may be implemented on a separate computing device that is proximate to sensor 102.

Example Intra-Frame Encoder

FIG. 2A illustrates components of an encoder for encoding intra point cloud frames, according to some embodiments. In some embodiments, the encoder described above in regard to FIG. 1 may operate in a similar manner as encoder 200 described in FIG. 2A and encoder 250 described in FIG. 2C.

The encoder 200 receives uncompressed point cloud 202 and generates compressed point cloud information 204. In some embodiments, an encoder, such as encoder 200, may receive the uncompressed point cloud 202 from a sensor, such as sensor 102 illustrated in FIG. 1, or, in some embodiments, may receive the uncompressed point cloud

202 from another source, such as a graphics generation component that generates the uncompressed point cloud in software, as an example.

In some embodiments, an encoder, such as encoder 200, includes decomposition into patches module 206, packing module 208, spatial image generation module 210, texture image generation module 212, and attribute information generation module 214. In some embodiments, an encoder, such as encoder 200, also includes image frame padding module 216, video compression module 218 and multiplexer 224. In addition, in some embodiments an encoder, such as encoder 200, may include an occupancy map compression module, such as occupancy map compression module 220, and an auxiliary patch information compression module, such as auxiliary patch information compression module 222. In some embodiments, an encoder, such as encoder 200, converts a 3D point cloud into an image-based representation along with some meta data (e.g., occupancy map and patch info) necessary to convert the compressed point cloud back into a decompressed point cloud.

In some embodiments, the conversion process decomposes the point cloud into a set of patches (e.g., a patch is defined as a contiguous subset of the surface described by the point cloud), which may be overlapping or not, such that each patch may be described by a depth field with respect to a plane in 2D space. More details about the patch decomposition process are provided above with regard to FIGS. 3A-3C.

After or in conjunction with the patches being determined for the point cloud being compressed, a 2D sampling process is performed in planes associated with the patches. The 2D sampling process may be applied in order to approximate each patch with a uniformly sampled point cloud, which may be stored as a set of 2D patch images describing the geometry/texture/attributes of the point cloud at the patch location. The "Packing" module 208 may store the 2D patch images associated with the patches in a single (or multiple) 2D images, referred to herein as "image frames" or "video image frames." In some embodiments, a packing module, such as packing module 208, may pack the 2D patch images such that the packed 2D patch images do not overlap (even though an outer bounding box for one patch image may overlap an outer bounding box for another patch image). Also, the packing module may pack the 2D patch images in a way that minimizes non-used images pixels of the image frame.

In some embodiments, "Geometry/Texture/Attribute generation" modules, such as modules 210, 212, and 214, generate 2D patch images associated with the geometry/texture/attributes, respectively, of the point cloud at a given patch location. As noted before, a packing process, such as performed by packing module 208, may leave some empty spaces between 2D patch images packed in an image frame. Also, a padding module, such as image frame padding module 216, may fill in such areas in order to generate an image frame that may be suited for 2D video and image codes.

In some embodiments, an occupancy map (e.g., binary information describing for each pixel or block of pixels whether the pixel or block of pixels are padded or not) may be generated and compressed, for example by occupancy map compression module 220. The occupancy map may be sent to a decoder to enable the decoder to distinguish between padded and non-padded pixels of an image frame.

Note that other metadata associated with patches may also be sent to a decoder for use in the decompression process. For example, patch information indicating sizes and shapes

of patches determined for the point cloud and packed in an image frame may be generated and/or encoded by an auxiliary patch-information compression module, such as auxiliary patch-information compression module 222. In some embodiments one or more image frames may be encoded by a video encoder, such as video compression module 218. In some embodiments, a video encoder, such as video compression module 218, may operate in accordance with the High Efficiency Video Coding (HEVC) standard or other suitable video encoding standard. In some embodiments, encoded video images, encoded occupancy map information, and encoded auxiliary patch information may be multiplexed by a multiplexer, such as multiplexer 224, and provided to a recipient as compressed point cloud information, such as compressed point cloud information 204.

In some embodiments, an occupancy map may be encoded and decoded by a video compression module, such as video compression module 218. This may be done at an encoder, such as encoder 200, such that the encoder has an accurate representation of what the occupancy map will look like when decoded by a decoder. Also, variations in image frames due to lossy compression and decompression may be accounted for by an occupancy map compression module, such as occupancy map compression module 220, when determining an occupancy map for an image frame. In some embodiments, various techniques may be used to further compress an occupancy map, such as described in FIGS. 12A-12B.

Example Intra-Frame Decoder

FIG. 2B illustrates components of a decoder for decoding intra point cloud frames, according to some embodiments. Decoder 230 receives compressed point cloud information 204, which may be the same compressed point cloud information 204 generated by encoder 200. Decoder 230 generates reconstructed point cloud 246 based on receiving the compressed point cloud information 204.

In some embodiments, a decoder, such as decoder 230, includes a de-multiplexer 232, a video decompression module 234, an occupancy map decompression module 236, and an auxiliary patch-information decompression module 238. Additionally a decoder, such as decoder 230 includes a point cloud generation module 240, which reconstructs a point cloud based on patch images included in one or more image frames included in the received compressed point cloud information, such as compressed point cloud information 204. In some embodiments, a decoder, such as decoder 203, further comprises a smoothing filter, such as smoothing filter 244. In some embodiments, a smoothing filter may smooth incongruences at edges of patches, wherein data included in patch images for the patches has been used by the point cloud generation module to recreate a point cloud from the patch images for the patches. In some embodiments, a smoothing filter may be applied to the pixels located on the patch boundaries to alleviate the distortions that may be caused by the compression/decompression process.

Example Inter-Frame Encoder

FIG. 2C illustrates components of an encoder for encoding inter point cloud frames, according to some embodiments. An inter point cloud encoder, such as inter point cloud encoder 250, may encode an image frame, while considering one or more previously encoded/decoded image frames as references.

In some embodiments, an encoder for inter point cloud frames, such as encoder 250, includes a point cloud re-sampling module 252, a 3-D motion compensation and delta vector prediction module 254, a spatial image generation module 256, a texture image generation module 258, and an

attribute image generation module 260. In some embodiments, an encoder for inter point cloud frames, such as encoder 250, may also include an image padding module 262 and a video compression module 264. An encoder for inter point cloud frames, such as encoder 250, may generate compressed point cloud information, such as compressed point cloud information 266. In some embodiments, the compressed point cloud information may reference point cloud information previously encoded by the encoder, such as information from or derived from one or more reference image frames. In this way an encoder for inter point cloud frames, such as encoder 250, may generate more compact compressed point cloud information by not repeating information included in a reference image frame, and instead communicating differences between the reference frames and a current state of the point cloud.

In some embodiments, an encoder, such as encoder 250, may be combined with or share modules with an intra point cloud frame encoder, such as encoder 200. In some embodiments, a point cloud re-sampling module, such as point cloud re-sampling module 252, may resample points in an input point cloud image frame in order to determine a one-to-one mapping between points in patches of the current image frame and points in patches of a reference image frame for the point cloud. In some embodiments, a 3D motion compensation & delta vector prediction module, such as a 3D motion compensation & delta vector prediction module 254, may apply a temporal prediction to the geometry/texture/attributes of the resampled points of the patches. The prediction residuals may be stored into images, which may be padded and compressed by using video/image codecs. In regard to spatial changes for points of the patches between the reference frame and a current frame, a 3D motion compensation & delta vector prediction module 254, may determine respective vectors for each of the points indicating how the points moved from the reference frame to the current frame. A 3D motion compensation & delta vector prediction module 254, may then encode the motion vectors using different image parameters. For example, changes in the X direction for a point may be represented by an amount of red included at the point in a patch image that includes the point. In a similar manner, changes in the Y direction for a point may be represented by an amount of blue included at the point in a patch image that includes the point. Also, in a similar manner, changes in the Z direction for a point may be represented by an amount of green included at the point in a patch image that includes the point. In some embodiments, other characteristics of an image included in a patch image may be adjusted to indicate motion of points included in the patch between a reference frame for the patch and a current frame for the patch.

Example Inter-Frame Decoder

FIG. 2D illustrates components of a decoder for decoding inter point cloud frames, according to some embodiments. In some embodiments, a decoder, such as decoder 280, includes a video decompression module 270, an inverse 3D motion compensation and inverse delta prediction module 272, a point cloud generation module 274, and a smoothing filter 276. In some embodiments, a decoder, such as decoder 280 may be combined with a decoder, such as decoder 230, or may share some components with the decoder, such as a video decompression module and/or smoothing filter. In decoder 280, the video/image streams are first decoded, then an inverse motion compensation and delta prediction procedure may be applied. The obtained images are then used

13

in order to reconstruct a point cloud, which may be smoothed as described previously to generate a reconstructed point cloud **282**.

Segmentation Process

FIG. 3A illustrates an example segmentation process for determining patches for a point cloud, according to some embodiments. The segmentation process as described in FIG. 3A may be performed by a decomposition into patches module, such as decomposition into patches module **206**. A segmentation process may decompose a point cloud into a minimum number of patches (e.g., a contiguous subset of the surface described by the point cloud), while making sure that the respective patches may be represented by a depth field with respect to a patch plane. This may be done without a significant loss of shape information.

In some embodiments, a segmentation process comprises: Letting point cloud PC be the input point cloud to be partitioned into patches and $\{P(0), P(1) \dots, P(N-1)\}$ be the positions of points of point cloud PC.

In some embodiments, a fixed set $D=\{D(0), D(1), \dots, D(K-1)\}$ of K 3D orientations is pre-defined. For instance, D may be chosen as follows $D=\{(1.0, 0.0, 0.0), (0.0, 1.0, 0.0), (0.0, 0.0, 1.0), (-1.0, 0.0, 0.0), (0.0, -1.0, 0.0), (0.0, 0.0, -1.0)\}$

In some embodiments, the normal vector to the surface at every point P(i) is estimated. Any suitable algorithm may be used to determine the normal vector to the surface. For instance, a technique could include fetching the set H of the “N” nearest points of P(i), and fitting a plane $\Pi(i)$ to H(i) by using principal component analysis techniques. The normal to P(i) may be estimated by taking the normal $\nabla(i)$ to $\Pi(i)$. Note that “N” may be a user-defined parameter or may be found by applying an optimization procedure. “N” may also be fixed or adaptive. The normal values may then be oriented consistently by using a minimum-spanning tree approach.

Normal-based Segmentation: An initial segmentation S0 of the points of point cloud PC may be obtained by associating respective points with the direction D(k) which maximizes the score $\langle \nabla(i) | D(k) \rangle$, where $\langle . | . \rangle$ is the canonical dot product of R3. Pseudo code is provided below.

```

for (i = 0; i < pointCount; ++i) {
    clusterIndex = 0;
    bestScore =  $\langle \nabla(i) | D(0) \rangle$ ;
    for(j = 1; j < K; ++j) {
        score =  $\langle \nabla(i) | D(j) \rangle$ ;
        if (score > bestScore) {
            bestScore = score;
            clusterIndex = j;
        }
    }
    partition[i] = clusterIndex;
}

```

Iterative segmentation refinement: Note that segmentation S0 associates respective points with the plane $\Pi(i)$ that best preserves the geometry of its neighborhood (e.g. the neighborhood of the segment). In some circumstances, segmentation S0 may generate too many small connected components with irregular boundaries, which may result in poor compression performance. In order to avoid such issues, the following iterative segmentation refinement procedure may be applied:

14

1. An adjacency graph A may be built by associating a vertex V(i) to respective points P(i) of point cloud PC and by adding R edges $\{E(i,j(0)), \dots, E(i,j(R-1))\}$ connecting vertex V(i) to its nearest neighbors $\{V(j(0)), V(j(1)), \dots, V(j(R-1))\}$. More precisely, $\{V(j(0)), V(j(1)), \dots, V(j(R-1))\}$ may be the vertices associated with the points $\{P(j(0)), P(j(1)), \dots, P(j(R-1))\}$, which may be the nearest neighbors of P(i). Note that R may be a user-defined parameter or may be found by applying an optimization procedure. It may also be fixed or adaptive.
2. At each iteration, the points of point cloud PC may be traversed and every vertex may be associated with the direction D(k) that maximizes

$$\left(\langle \nabla(i) | D(k) \rangle + \frac{\lambda}{R} |\zeta(i)| \right),$$

where $|\zeta(i)|$ is the number of the R-nearest neighbors of V(i) belonging to the same cluster and λ is a parameter controlling the regularity of the produced patches. Note that the parameters λ and R may be defined by the user or may be determined by applying an optimization procedure. They may also be fixed or adaptive. In some embodiments, a “user” as referred to herein may be an engineer who configured a point cloud compression technique as described herein to one or more applications.

3. An example of pseudo code is provided below

```

for(l = 0; l < iterationCount; ++l) {
    for(i = 0; i < pointCount; ++i) {
        clusterIndex = partition[i];
        bestScore = 0.0;
        for(k = 0; k < K; ++k) {
            score =  $\langle \nabla(i) | D(k) \rangle$ ;
            for (j ∈ {j(0), j(1), ... j (R - 1)}) {
                if (k == partition[j]) {
                    score +=  $\frac{\lambda}{R}$ ;
                }
            }
            if (score > bestScore) {
                bestScore = score;
                clusterIndex = k;
            }
        }
        partition[i] = clusterIndex;
    }
}

```

*In some embodiments, the pseudo code shown above may further include an early termination step. For example, if a score that is a particular value is reached, or if a difference between a score that is reached and a best score only changes by a certain amount or less, the search could be terminated early. Also, the search could be terminated if after a certain number of iterations ($l=m$), the clusterindex does not change.

Patch segmentation: In some embodiments, the patch segmentation procedure further segments the clusters detected in the previous steps into patches, which may be represented with a depth field with respect to a projection plane. The approach proceeds as follows, according to some embodiments:

15

1. First, a cluster-based adjacency graph with a number of neighbors R' is built, while considering as neighbors only the points that belong to the same cluster. Note that R' may be different from the number of neighbors R used in the previous steps.
2. Next, the different connected components of the cluster-based adjacency graph are extracted. Only connected components with a number of points higher than a parameter α are considered. Let $CC = \{CC(0), CC(1), \dots, CC(M-1)\}$ be the set of the extracted connected components.
3. Respective connected component $CC(m)$ inherits the orientation $D(m)$ of the cluster it belongs to. The points of $CC(m)$ are then projected on a projection plane having as normal the orientation $D(m)$, while updating a depth map, which records for every pixel the depth of the nearest point to the projection plane.
4. An approximated version of $CC(m)$, denoted $C'(m)$, is then built by associating respective updated pixels of the depth map with a 3D point having the same depth. Let PC' be the point cloud obtained by the union of reconstructed connected components $\{C'(0), C'(1), \dots, C'(M-1)\}$.
5. Note that the projection reconstruction process may be lossy and some points may be missing. In order, to detect such points, every point $P(i)$ of point cloud PC may be checked to make sure it is within a distance lower than a parameter δ from a point of PC' . If this is not the case, then $P(i)$ may be marked as a missed point and added to a set of missed points denoted MP .
6. The steps 2-5 are then applied to the missed points MP . The process is repeated until MP is empty or CC is empty. Note that the parameters δ and α may be defined by the user or may be determined by applying an optimization procedure. They may also be fixed or adaptive.
7. A filtering procedure may be applied to the detected patches in order to make them better suited for compression. Example filter procedures may include:
 - a. A smoothing filter based on the geometry/texture/attributes of the points of the patches (e.g., median filtering), which takes into account both spatial and temporal aspects.
 - b. Discarding small and isolated patches.
 - c. User-guided filtering.
 - d. Other suitable smoothing filter techniques.

Layers

The image generation process described above consists of projecting the points belonging to each patch onto its associated projection plane to generate a patch image. This process could be generalized to handle the situation where multiple points are projected onto the same pixel as follows:

Let $H(u,v)$ be the set of points of the current patch that get projected to the same pixel (u,v) . Note that $H(u,v)$ may be empty, may have one point or multiple points.

If $H(u,v)$ is empty then the pixel is marked as unoccupied. If the $H(u,v)$ has a single element, then the pixel is filled with the associated geometry/texture/attribute value.

If $H(u,v)$, has multiple elements, then different strategies are possible:

- Keep only the nearest point $P0(u,v)$ for the pixel (u,v)
- Take the average or a linear combination of a group of points that are within a distance d from $P0(u,v)$, where d is a user-defined parameter needed only on the encoder side.

16

Store two images: one for $P0(u,v)$ and one to store the furthest point $P1(u,v)$ of $H(u,v)$ that is within a distance d from $P0(u,v)$

Store N patch images containing a subset of $H(u,v)$

The generated patch images for point clouds with points at the same patch location, but different depths may be referred to as layers herein. In some embodiments, scaling/up-sampling/down-sampling could be applied to the produced patch images/layers in order to control the number of points in the reconstructed point cloud.

Guided up-sampling strategies may be performed on the layers that were down-sampled given the full resolution image from another "primary" layer that was not down-sampled.

In some embodiments, a delta prediction between layers could be adaptively applied based on a rate-distortion optimization. This choice may be explicitly signaled in the bit stream.

In some embodiments, the generated layers may be encoded with different precisions. The precision of each layer may be adaptively controlled by using a shift+scale or a more general linear or non-linear transformation.

In some embodiments, an encoder may make decisions on a scaling strategy and parameters, which are explicitly encoded in the bit stream. The decoder may read the information from the bit stream and apply the right scaling process with the parameters signaled by the encoder.

In some embodiments, a video encoding motion estimation process may be guided by providing a motion vector map to the video encoder indicating for each block of the image frame, a 2D search center or motion vector candidates for the refinement search. Such information, may be trivial to compute since the mapping between the 3D frames and the 2D image frames is available to the point cloud encoder and a coarse mapping between the 2D image frames could be computed by using a nearest neighbor search in 3D.

The video motion estimation/mode decision/intra-prediction could be accelerated/improved by providing a search center map, which may provide guidance on where to search and which modes to choose from for each $N \times N$ pixel block.

Hidden/non-displayed pictures could be used in codecs such as AV1 and HEVC. In particular, synthesized patches could be created and encoded (but not displayed) in order to improve prediction efficiency. This could be achieved by re-using a subset of the padded pixels to store synthesized patches.

The patch re-sampling (e.g., packing and patch segmentation) process described above exploits solely the geometry information. A more comprehensive approach may take into account the distortions in terms of geometry, texture, and other attributes and may improve the quality of the re-sampled point clouds.

Instead of first deriving the geometry image and optimizing the texture image given said geometry, a joint optimization of geometry and texture could be performed. For example, the geometry patches could be selected in a manner that results in minimum distortion for both geometry and texture. This could be done by immediately associating each possible geometry patch with its corresponding texture patch and computing their corresponding distortion information. Rate-distortion optimization could also be considered if the target compression ratio is known.

In some embodiments, a point cloud resampling process described above may additionally consider texture and attributes information, instead of relying only on geometry.

Also, a projection-based transformation that maps 3D points to 2D pixels could be generalized to support arbitrary 3D to 2D mapping as follows:

Store the 3D to 2D transform parameters or the pixel coordinates associated with each point

Store X, Y, Z coordinates in the geometry images instead of or in addition to the depth information

Packing

In some embodiments, depth maps associated with patches, also referred to herein as “depth patch images” or “geometry patch images,” such as those described above, may be packed into a 2D image frame. For example, a packing module, such as packing module 208, may pack depth patch images generated by a spatial image generation module, such as spatial image generation module 210. The depth maps, or depth patch images, may be packed such that (A) no non-overlapping block of $T \times T$ pixels contains depth information from two different patches and such that (B) a size of the generated image frame is minimized.

In some embodiments, packing comprises the following steps:

- a. The patches are sorted by height and then by width. The patches are then inserted in image frame (I) one after the other in that order. At each step, the pixels of image frame (I) are traversed in raster order, while checking if the current patch could be inserted under the two conditions (A) and (B) described above. If it is not possible then the height of (I) is doubled.

- b. This process is iterated until all the patches are inserted.

In some embodiments, the packing process described above may be applied to pack a subset of the patches inside multiples tiles of an image frame or multiple image frames. This may allow patches with similar/close orientations based on visibility according to the rendering camera position to be stored in the same image frame/tile, to enable view-dependent streaming and/or decoding. This may also allow parallel encoding/decoding.

In some embodiments, the packing process can be considered a bin-packing problem and a first decreasing strategy as described above may be applied to solve the bin-packing problem. In other embodiments, other methods such as the modified first fit decreasing (MFFD) strategy may be applied in the packing process.

In some embodiments, if temporal prediction is used, such as described for inter compression encoder 250, such an optimization may be performed with temporal prediction/encoding in addition to spatial prediction/encoding. Such consideration may be made for the entire video sequence or per group of pictures (GOP). In the latter case additional constraints may be specified. For example, a constraint may be that the resolution of the image frames should not exceed a threshold amount. In some embodiments, additional temporal constraints may be imposed, even if temporal prediction is not used, for example such as that a patch corresponding to a particular object view is not moved more than x number of pixels from previous instantiations.

FIG. 3B illustrates an example image frame comprising packed patch images and padded portions, according to some embodiments. Image frame 300 includes patch images 302 packed into image frame 300 and also includes padding 304 in space of image frame 300 not occupied by patch images. In some embodiments, padding, such as padding 304, may be determined so as to minimize incongruences between a patch image and the padding. For example, in some embodiments, padding may construct new pixel blocks that are replicas of, or are to some degree similar to, pixel blocks that are on the edges of patch images. Because

an image and/or video encoder may encode based on differences between adjacent pixels, such an approach may reduce the number of bytes required to encode an image frame comprising of patch images and padding, in some embodiments.

In some embodiments, the patch information may be stored in the same order as the order used during the packing, which makes it possible to handle overlapping 2D bounding boxes of patches. Thus a decoder receiving the patch information can extract patch images from the image frame in the same order in which the patch images were packed into the image frame. Also, because the order is known by the decoder, the decoder can resolve patch image bounding boxes that overlap.

FIG. 3C illustrates an example image frame 312 with overlapping patches, according to some embodiments. FIG. 3C shows an example with two patches (patch image 1 and patch image 2) having overlapping 2D bounding boxes 314 and 316 that overlap at area 318. In order to determine to which patch the $T \times T$ blocks in the area 318 belong, the order of the patches may be considered. For example, the $T \times T$ block 314 may belong to the last decoded patch. This may be because in the case of an overlapping patch, a later placed patch is placed such that it overlaps with a previously placed patch. By knowing the placement order it can be resolved that areas of overlapping bounding boxes go with the latest placed patch. In some embodiments, the patch information is predicted and encoded (e.g., with an entropy/arithmetic encoder). Also, in some embodiments, $U0$, $V0$, $DU0$ and $DV0$ are encoded as multiples of T , where T is the block size used during the padding phase.

FIG. 3C also illustrates blocks of an image frame 312, wherein the blocks may be further divided into sub-blocks. For example block A1, B1, C1, A2, etc. may be divided into multiple sub-blocks, and, in some embodiments, the sub-blocks may be further divided into smaller blocks. In some embodiments, a video compression module of an encoder, such as video compression module 218 or video compression module 264, may determine whether a block comprises active pixels, non-active pixels, or a mix of active and non-active pixels. The video compression module may budget fewer resources to compress blocks comprising non-active pixels than an amount of resources that are budgeted for encoding blocks comprising active pixels. In some embodiments, active pixels may be pixels that include data for a patch image and non-active pixels may be pixels that include padding. In some embodiments, a video compression module may sub-divide blocks comprising both active and non-active pixels, and budget resources based on whether sub-blocks of the blocks comprise active or non-active pixels. For example, blocks A1, B1, C1, A2 may comprise non-active pixels. As another example block E3 may comprise active pixels, and block B6, as an example, may include a mix of active and non-active pixels.

In some embodiments, a patch image may be determined based on projections, such as projecting a point cloud onto a cube, cylinder, sphere, etc. In some embodiments, a patch image may comprise a projection that occupies a full image frame without padding. For example, in a cubic projection each of the six cubic faces may be a patch image that occupies a full image frame.

For example, FIG. 3D illustrates a point cloud being projected onto multiple projections, according to some embodiments.

In some embodiments, a representation of a point cloud is encoded using multiple projections. For example, instead of determining patches for a segment of the point cloud pro-

jected on a plane perpendicular to a normal to the segment, the point cloud may be projected onto multiple arbitrary planes or surfaces. For example, a point cloud may be projected onto the sides of a cube, cylinder, sphere, etc. Also multiple projections intersecting a point cloud may be used. In some embodiments, the projections may be encoded using conventional video compression methods, such as via a video compression module 218 or video compression module 264. In particular, the point cloud representation may be first projected onto a shape, such as a cube, and the different projections/faces projected onto that shape (i.e. front (320), back (322), top (324), bottom (326), left (328), right (330)) may all be packed onto a single image frame or multiple image frames. This information, as well as depth information may be encoded separately or with coding tools such as the ones provided in the 3D extension of the HEVC (3D-HEVC) standard. The information may provide a representation of the point cloud since the projection images can provide the (x,y) geometry coordinates of all projected points of the point cloud. Additionally, depth information that provides the z coordinates may be encoded. In some embodiments, the depth information may be determined by comparing different ones of the projections, slicing through the point cloud at different depths. When projecting a point cloud onto a cube, the projections might not cover all point cloud points, e.g. due to occlusions. Therefore additional information may be encoded to provide for these missing points and updates may be provided for the missing points.

In some embodiments, adjustments to a cubic projection can be performed that further improve upon such projections. For example, adjustments may be applied at the encoder only (non-normative) or applied to both the encoder and the decoder (normative).

More specifically, in some embodiments alternative projections may be used. For example, instead of using a cubic projection, a cylindrical or spherical type of a projection method may be used. Such methods may reduce, if not eliminate, redundancies that may exist in the cubic projection and reduce the number or the effect of “seams” that may exist in cubic projections. Such seams may create artifacts at object boundaries, for example. Eliminating or reducing the number or effect of such seams may result in improved compression/subjective quality as compared to cubic projection methods. For a spherical projection case, a variety of sub-projections may be used, such as the equirectangular, equiangular, and authagraph projection among others. These projections may permit the projection of a sphere onto a 2D plane. In some embodiments, the effects of seams may be de-emphasized by overlapping projections, wherein multiple projections are made of a point cloud, and the projections overlap with one another at the edges, such that there is overlapping information at the seams. A blending effect could be employed at the overlapping seams to reduce the effects of the seams, thus making them less visible.

In addition to, or instead of, considering a different projection method (such as cylindrical or spherical projections), in some embodiments multiple parallel projections may be used. The multiple parallel projections may provide additional information and may reduce a number of occluded points. The projections may be known at the decoder or signaled to the decoder. Such projections may be defined on planes or surfaces that are at different distances from a point cloud object. Also, in some embodiments the projections may be of different shapes, and may also overlap or cross through the point cloud object itself. These projections may permit capturing some characteristics of a point

cloud object that may have been occluded through a single projection method or a patch segmentation method as described above.

For example, FIG. 3E illustrates a point cloud being projected onto multiple parallel projections, according to some embodiments. Point cloud 350 which includes points representing a coffee mug is projected onto parallel horizontal projections 352 that comprise planes orthogonal to the Z axis. Point cloud 350 is also projected onto vertical projections 354 that comprise planes orthogonal to the X axis, and is projected onto vertical projections 356 that comprise planes orthogonal to the Y axis. In some embodiments, instead of planes, multiple projections may comprise projections having other shapes, such as multiple cylinders or spheres.

Generating Images Having Depth

In some embodiments, only a subset of the pixels of an image frame will be occupied and may correspond to a subset of 3D points of a point cloud. Mapping of patch images may be used to generate geometry, texture, and attribute images, by storing for each occupied pixel the depth/texture/attribute value of its associated point.

In some embodiments, spatial information may be stored with various variations, for example spatial information may:

- Store depth as a monochrome image.
- Store depth as Y and keep U and V empty (where YUV is a color space, also RGB color space may be used).
- Store depth information for different patches in different color planes Y, U and V, in order to avoid inter-patch contamination during compression and/or improve compression efficiency (e.g., have correlated patches in the same color plane). Also, hardware codec capabilities may be utilized, which may spend the same encoding\decoding time independently of the content of the frame.
- Store depth patch images on multiple images or tiles that could be encoded and decoded in parallel. One advantage is to store depth patch images with similar/close orientations or based on visibility according to the rendering camera position in the same image/tile, to enable view-dependent streaming and/or decoding.
- Store depth as Y and store a redundant version of depth in U and V.
- Store X, Y, Z coordinates in Y, U, and V
- Different bit depth (e.g., 8, 10 or 12-bit) and sampling (e.g., 420, 422, 444 . . .) may be used. Note that different bit depth may be used for the different color planes.

Padding

In some embodiments, padding may be performed to fill the non-occupied pixels with values such that the resulting image is suited for video/image compression. For example, image frame padding module 216 or image padding module 262 may perform padding as described below.

In some embodiments, padding is applied on pixels blocks, while favoring the intra-prediction modes used by existing video codecs. More precisely, for each block of size BxB to be padded, the intra prediction modes available at the video encoder side are assessed and the one that produces the lowest prediction errors on the occupied pixels is retained. This may take advantage of the fact that video/image codecs commonly operate on pixel blocks with pre-defined sizes (e.g., 64x64, 32x32, 16x16 . . .). In some embodiments, other padding techniques may include linear extrapolation, in-painting techniques, or other suitable techniques.

Video Compression

In some embodiments, a video compression module, such as video compression module 218 or video compression module 264, may perform video compression as described below.

In some embodiments, a video encoder may leverage an occupancy map, which describes for each pixel of an image whether it stores information belonging to the point cloud or padded pixels. In some embodiments, such information may permit enabling various features adaptively, such as deblocking, adaptive loop filtering (ALF), or shape adaptive offset (SAO) filtering. Also, such information may allow a rate control module to adapt and assign different, e.g. lower, quantization parameters (QPs), and in an essence a different amount of bits, to the blocks containing the occupancy map edges. Coding parameters, such as lagrangian multipliers, quantization thresholding, quantization matrices, etc. may also be adjusted according to the characteristics of the point cloud projected blocks. In some embodiments, such information may also enable rate distortion optimization (RDO) and rate control/allocation to leverage the occupancy map to consider distortions based on non-padded pixels. In a more general form, weighting of distortion may be based on the “importance” of each pixel to the point cloud geometry. Importance may be based on a variety of aspects, e.g. on proximity to other point cloud samples, directionality/orientation/position of the samples, etc. Facing forward samples, for example, may receive a higher weighting in the distortion computation than backward facing samples. Distortion may be computed using metrics such as Mean Square or Absolute Error, but different distortion metrics may also be considered, such as SSIM, VQM, VDP, Hausdorff distance, and others.

Occupancy Map Compression

In some embodiments, an occupancy map compression module, such as occupancy map compression module 220, may compress an occupancy map as described below.

Example Occupancy Map Compression Techniques

In some embodiments, an occupancy map may be encoded in a hierarchical mode. Such a process may comprise:

1. A binary information for each $B_1 \times B_2$ pixel block (e.g., a rectangle that covers the entire image frame, or smaller blocks of different sizes such as 64×64 , 64×32 , 32×32 block, etc.) being encoded indicating whether the block is empty (e.g., has only padded pixels) or non-empty (e.g., has non-padded pixels).
2. If the block is non-empty, then a second binary information may be encoded to indicate whether the block is full (e.g., all the pixels are non-padded) or not.
3. The non-empty and non-full blocks may then be refined by considering their $(B_1/2) \times (B_2/2)$ sub-blocks.
4. The steps 1-3 may be repeated until the size of the block reaches a certain block size $B_3 \times B_4$ (e.g., of size 4×4). At this level only the empty/non-empty information may be encoded.
5. An entropy-based codec may be used to encode the binary information in steps 1 and 2. For instance, context adaptive binary arithmetic encoders may be used.
6. The reconstructed geometry image may be leveraged to better encode the occupancy map. More precisely, the residual prediction errors may be used to predict whether a block is empty or not or full or not. Such an information may be incorporated by using a different

context based on the predicted case or simply by encoding the binary value XORed with the predicted value.

In some embodiments, mesh-based codecs may be an alternative to the approach described above.

Additional Example Occupancy Map Compression Technique

In some embodiments, auxiliary information and the patch encoding order may be leveraged in order to efficiently compress a mapping information indicating for each $T \times T$ block of an image frame (e.g., 16×16 block) to which patch it belongs to. This mapping may be explicitly encoded in the bit stream as follows:

A list of candidate patches is created for each $T \times T$ block of an image frame by considering all the patches that overlap with that block.

The list of candidates is sorted in the reverse order of the patches. Meaning the candidate patches are ordered from smallest patch to largest patch.

For each block, the index of the patches in this list is encoded by using an arithmetic or other form of an entropy encoder (e.g. UVLC or Huffman based).

Note that empty blocks are assigned a special index, such as zero.

The mapping information described above makes it possible to detect empty $T \times T$ blocks of an image frame (e.g., blocks that contain only padded pixels). The occupancy information is encoded only for the non-empty $T \times T$ blocks (e.g., the blocks that contain at least one non-padded pixel).

The occupancy map is encoded with a precision of a $B_0 \times B_0$ blocks. In order to achieve lossless encoding B_0 is chosen to be 1. In some embodiments $B_0=2$ or $B_0=4$, which may result in visually acceptable results, while significantly reducing the number of bits required to encode the occupancy map.

Binary values are associated with $B_0 \times B_0$ sub-blocks belonging to the same $T \times T$ block. Different strategies are possible. For instance, one could associate a value of 1 if the sub-block contains at least some non-padded pixels and 0 otherwise. If a sub-block has a value of 1 it is said to be full, otherwise it is an empty sub-block.

If all the sub-blocks of a $T \times T$ block are full (e.g., have value 1). The block is said to be full. Otherwise, the block is said to be non-full.

A binary information is encoded for each $T \times T$ block to indicate whether it is full or not. Various encoding strategies could be used. For instance, a context adaptive binary arithmetic encoder could be used.

If the block is non-full, an extra information is encoded indicating the location of the full/empty sub-blocks. More precisely, the process may proceed as follows: Different traversal orders are defined for the sub-blocks. FIG. 12B, shows some examples. The traversal orders are predetermined and known to both the encoder and decoder.

The encoder chooses one of the traversal orders and explicitly signals its index in the bit stream.

The binary values associated with the sub-blocks are encoded by using a run-length encoding strategy.

The binary value of the initial sub-block is encoded. Various encoding strategies could be used. For instance, fixed length coding or a context adaptive binary arithmetic encoder could be used.

Continuous runs of 0s and 1s are detected, while following the traversal order selected by the encoder.

23

The number of detected runs is encoded. Various encoding strategies could be used. For instance, fixed length coding or a context adaptive binary arithmetic encoder, or a universal variable length encoder (UVLC) could be used.

The length of each run, except of the last one, is then encoded. Various encoding strategies could be used. For instance, fixed length coding, a context adaptive binary arithmetic encoder, or a universal variable length encoder could be used.

Note that the symbol probabilities used during the arithmetic encoding could be initialized by using values explicitly signaled in the bit stream by the encoder in order to improve compression efficiency. Such information could be signaled at frame, slice, row(s) of blocks, or block level, or using a non-fixed interval. In that case, a system may have the ability to signal the initialization interval, or the interval adaptation could be predefined between encoder and decoder. For example, the interval could start with one block, and then increment by one block afterwards (e.g. using an adaptation positions of {1, 2, 3 . . . N-1 . . . } blocks.

The choice of the traversal order may have a direct impact on the compression efficiency. Different strategies are possible. For instance, the encoder could choose the traversal order, which would result in the lowest number of bits or the lowest number of runs. In some embodiments, hierarchical sub-blocks with variable sizes may be used.

In some embodiments, temporal prediction may be used for encoding/compressing occupancy maps as follows:

- a. The occupancy map of the current frame may be predicted from the occupancy map of a reference frame (e.g. through a difference process assuming zero motion). The prediction could be done at the frame level, but could also be done at a sub-block level, e.g. signal 1 bit whether a block will be predicted temporally, or the original map for a block will be used instead.
- b. Prediction could be enhanced by using motion compensation and by associating a motion vector with each T×T block.
- c. The values of the current block may be XOR-ed with the values of the block referenced by the motion vector or the co-located block. If no prediction is used, the current block may be coded as is.
- d. Motion vectors could be integer, integer multiples, or can have sub-pixel precision.
- e. The encoding strategy described above may be applied to the results.
- f. The motion vectors of the current block may be predicted based on the motion vectors of the previously encoded blocks. For example, a list of candidate predicted motion vectors may be computed based on the motion vectors of spatially and/or temporally neighboring blocks that have already been encoded. The index of the best candidate to be used as a predictor and the difference can be explicitly encoded in the bit stream. The process may be similar to the process used in codecs such as AVC and HEVC among others. A reduction in temporal candidates may be performed similar to what is done in HEVC to reduce memory requirements. The residual motion vector can then be encoded using a technique such as context adaptive arithmetic encoding or UVLC.
- g. A skip mode may also be supported to indicate that the predicted block matches exactly the reference block. In that case, no residual motion vector is needed.

24

h. Different block sizes could be used instead of sticking with T×T blocks.

i. The choice of the block size and the motion vectors could be achieved by minimizing the number of bits required to encode the occupancy map.

j. The process could also consider multiple references.

In some embodiments, additional techniques for encoding/compression of an occupancy map may include:

Using clues included in the video picture to help encode the occupancy map, such as:

Use high quantization parameters QPs (e.g., 51) or use skip mode for blocks composed of padded pixels only.

The arithmetic encoding contexts could be adaptively adjusted based on information extracted from the video bit streams associated with the texture/geometry/motion frames.

Group the binary values associated with pixels into 8-bit or 10-bit words and encode them with dictionary-based approaches such as the DEFLATE algorithm.

Pixels could be grouped 4×2/5×2 blocks or by leveraging a zig zag scan.

Only the pixels belonging to non-empty T×T blocks may be encoded.

The mapping information indicating for each T×T block to which patch it belongs may be encoded.

Additional Example Occupancy Map Compression Techniques

In some embodiments, a binary occupancy map is generated based on whether or not blocks of the occupancy map are occupied or un-occupied. This may be performed in a similar manner as described above. Also, the patch information (e.g. bounding box position, size, etc.) is encoded using an arithmetic encoder, in a similar manner as described above. However, instead of relying on the occupancy map to discard empty blocks that intersect with at least one patch bounding box, the empty boxes are explicitly signaled with a special value for the local index. In this approach, the block to patch information is decoded when needed.

In some embodiments, instead of using an arithmetic encoder as described above to encode block to patch information that links boxes of the occupancy map with particular patches, the block to patch information (which contains the local indexes) may be encoded using a video-based encoder. The encoded block-to patch information may then be decoded using a corresponding video-decoder.

In some embodiments, instead of generating a binary occupancy map based on whether or not blocks of the occupancy map are occupied or un-occupied, a non-binary occupancy map is generated. The non-binary occupancy map is configured such that each pixel not only indicates whether the pixel is occupied or non-occupied, but also includes an attribute value, such as a color value that is associated with a local index value of a patch with which the pixel is associated. If the pixel is non-occupied, the pixel may have a color value of zero. Also, the patch information (e.g. bounding box position, size, etc.) is encoded using an arithmetic encoder, in a similar manner as described above. The non-binary occupancy map may be encoded using a video-based encoder. A decoder can retrieve the block to patch information by decoding the non-binary occupancy map and matching each pixel value with the local index lists.

In some embodiments, instead of using a local index, a full list of patches may be used as an index. In such embodiments, there may be no need to compute a list of candidate patches for each block. The decoder can retrieve the block-to-patch information by decoding the non-binary

occupancy map directly reading the index value for the patch associated with the pixel from the value of the pixel. In such embodiments, the local index may be omitted because there are enough unique values (e.g. non-binary) values available to be associated with a block, such that each candidate patch may be assigned a unique value.

In some embodiments, during the generation of the occupancy map, the bounding boxes for the patches may be adjusted or initially packed in an image frame such that the bounding boxes do not overlap. This removes ambiguity as to whether a particular bounding box belongs to a particular patch or another patch. The patch information (with non-overlapping bounding boxes) is encoding using an arithmetic encoder. Because there is not ambiguity as to which patch goes with which bounding box, the block to patch information (such as in the local index or complete index, as described above), may be omitted.

In some embodiments, a process that uses a full list of patches (instead of a local index) may result in a high number of patches, which may exceed the max possible number of values (e.g. color values) that may be represented in the non-binary occupancy map. In some embodiments, to address such issues, an occupancy map may be decomposed into segments, with a limited number of patches per segments. Thus for each segment, the patch index is bound. For example, fewer patches may be listed as possibilities for a segment of an occupancy map, such that for each segment the list of possible patches is less than the max possible number of values (e.g. color values). In some such embodiments, bounding boxes for different patches may be allowed to overlap within a segment, but not across segments. During decoding, each segment may have its own global index list of possible patches for that segment.

In some embodiments, a binary occupancy map is generated such that when the patches are packed in the image frame, a bounding box of the patch, aligned to an occupancy resolution does not intersect any previously packed patches of size= $\text{occupancy resolution} \times \text{size occupancy resolution}$ (e.g. a 16×16 block). The patch information (e.g. bounding box position and size) for each patch is encoded using an arithmetic encoder. The order in which the patch information for each patch is encoded may create a hierarchy of patches, such that for any overlapping bounding boxes, the corresponding patch that goes with the bound box can be resolved based on the hierarchy of patch information. The decoder may reconstruct block to patch information using the arithmetically encoded patch information (without the block to patch information being explicitly encoded). For example, a patch list may be parsed in a same order at a decoder as an order in which the patch list was generated at encoding time, wherein the order indicates an order in which the patches were packed in the image frame. This is possible because the packing guarantees that the bounding box for a given patch does not cover any previously processed patch. In such embodiments, patches may be packed (and signaled) in an order such as from small to large, or vice versa. During the packing, each block may include pixels of just one patch, but some bounding boxes for multiple patches may overlap, wherein blocks of the overlapping patches include no pixels for either patch, or pixels for just one of the patches, but not pixels for more than one patch.

Auxiliary Patch-Information Compression

In some embodiments, for each patch, the following information may be encoded. For example, by auxiliary patch-info compression module 222.

Its location (U0, V0) in the packed image frame and the extent of its 2D bounding box (DU0, DV0).

Minimum/maximum/average/median depth value.

Index of the projection direction.

5 Video-Based Occupancy Map Compression

As described above, in some embodiments, the occupancy map is a binary information that indicates for each pixel in the image frame whether the pixel should be interpreted as an actual point in the point cloud or not, e.g. the pixel is a padded pixel or not. Also, as described above, the auxiliary patch-information indicates for each $T \times T$ block of an image frame to which patch it belongs. Whereas it was described above to encode an index of patches for a block of an image frame and to keep encoding information for sub-blocks of the image frame until the sub-blocks were either fully empty or fully occupied, an alternative approach is to use a video encoder to encode an additional image frame for the occupancy map. In such embodiments, the additional occupancy map image frame, indicates occupied and unoccupied pixels based on image properties such as different colors (e.g. occupied pixels may be white and non-occupied pixels may be black). In this way it is not necessary to completely subdivide the blocks of the image frame until only occupied or un-occupied sub-blocks are determined. Instead, it is only necessary to identify bounding box sizes and locations in the image frame for the respective patches. The video encoded occupancy map will mirror the image frame and the different pixel values in the occupancy map video image frame will indicate which pixels in a given bounding box of the patch video image frame are patch image pixels or are padded pixels. Thus there is not a need to create a bit stream of sub-divided blocks of the image frame and there is not a need to indicate for each sub-block whether the sub-block is full or empty. Instead the video encoded occupancy map can be used to determine which pixels included in a patch bounding box are padded pixels or patch image pixels. In some embodiments, an occupancy map may be first encoded and then used to generate an index of patches that are associated with blocks of an image frame. In some embodiments, a compression process follows the following procedure that leverages existing video codecs to compress an occupancy map.

The occupancy map could be encoded with a precision of $B_0 \times B_1$ blocks. In order to achieve lossless encoding B_0 and B_1 may be chosen to be equal to 1. In practice $B_0=B_1=2$ or $B_0=B_1=4$ may result in visually acceptable results, while significantly reducing the number of bits required to encode the occupancy map.

In some embodiments, a single binary is associated with each $B_0 \times B_1$ sub-block. Different strategies are possible. For instance, one could associate a value of 1 with the rule that the sub-block contains at least one non-padded pixel and the value of 0 if not. In order to reduce computational complexity, the binary values of multiple $B_0 \times B_1$ blocks could be grouped together in a single pixel value.

A binary video frame may be generated by storing the value of each $B_0 \times B_1$ block in a pixel. The obtained video frame could be compressed by using a lossless video codec. For example the HEVC video codec could be utilized and its main, screen context coding (scc) main or other profiles could be used.

In some embodiments, the occupancy map could be packed in a 4:4:4 or 4:2:0 chroma format, where the chroma information could contain fixed values, e.g. the values 0 or 128 for an 8 bit codec. The occupancy map could also be coded using a codec supporting a monochrome representation. The occupancy map could be replicated in all color

components and encoded using a 4:4:4 representation. Other rearrangements of the occupancy map could be used so as to fit the data in a 4:4:4, 4:2:2, or 4:2:0 representation, while preserving the lossless nature of the signal and at the same time preserving the lossless characteristics of the occupancy map. For example, the occupancy map could be segmented to even horizontal and odd horizontal position sub-maps, and those sub-maps could be embedded into a 4:4:4 signal, the odd position samples in the Y plane and the even position samples in the U plane, and then encoded. This could provide savings in complexity since a reduced resolution (by half) image would be encoded. Other such arrangements could be used.

The occupancy map is used to detect non-empty T×T blocks and only for those blocks a patch index is encoded by proceeding as follows:

- 1) A list of candidate patches is created for each T×T block by considering all the patches that contain that block.
- 2) The list of candidates is sorted in the reverse order of the patches. Meaning the index is sorted from smallest patch to largest patch, e.g. the patches with bounding boxes covering the smallest area are ordered ahead of patches with bounding boxes covering larger areas of the patch image frame.
- 3) For each block, the index of the patch in this list is encoded by using an entropy encoder, e.g. an arithmetic encoder or other suitable encoder.

Patch Alignment and Size Determination in a 2D Bounding Box of an Occupancy Map

In some embodiments, methods may be applied to remove redundant output points created by the occupancy map quantization/downsampling/upsampling process. By removing these points, the reconstruction process can result in better reconstruction. Furthermore, fewer points may need to be processed during post-processing, e.g. when performing smoothing as described below, thus reducing reconstruction complexity as well as during attribute image generation during encoding. Additionally, quality of the “removed” points in the geometry and attribute layers may be less important, therefore the characteristics of such points may be exploited during compression, such as devoting fewer resources to redundant points that will be removed.

In some embodiments, when a patch is created, the patch size information (e.g. sizeU0, sizeV0) is defined as multiples of the occupancy packing block. In other words, when patch size is N×M and the occupancy packing block resolution is 16, sizeU0 and sizeV0 will be (16*(N/16+1), 16*(M/16+1)). For example, Table 1 shows an example algorithm for determining the width and width of a 2D bounding box for a patch.

TABLE 1

Width and Height of Patch Derivation

If p is equal to 0, then:

Patch2dSizeU[frmIdx][p] = pdu_2d_delta_size_u[frmIdx][p] * ops_occupancy_packing_block_size (8-8)

Patch2dSizeV[frmIdx][p] = pdu_2d_delta_size_v[frmIdx][p] * ops_occupancy_packing_block_size (8-9)

Otherwise, if (p > 0), then:

Patch2dSizeU[frmIdx][p] = Patch2dSizeU[frmIdx][p - 1] + pdu_2d_delta_size_u[frmIdx][p] * ops_occupancy_packing_block_size (8-10)

Patch2dSizeV[frmIdx][p] = Patch2dSizeV[frmIdx][p - 1] + pdu_2d_delta_size_v[frmIdx][p] * ops_occupancy_packing_block_size (8-11)

In some embodiments, in a patch bounding box, there could be “empty” lines and/or columns maximum equal to (occupancy packing block resolution - 1).

In some embodiments, an occupancy map could be quantized/downsampled by oPrecision which can be derived from the decoded occupancy map video resolution and the nominal resolution of the decoded video frames and then dequantized/upsampled when it is used. Therefore, (oPrecision×oPrecision) pixels will share one same value (1. Occupied 0. Empty). When the (oPrecision×oPrecision) pixels were not fully filled with 1 before the quantization process, the dequantization process will mark previously empty pixels with redundant points, and it would add on the distortion and complexity of the point cloud.

A method which simply discards samples that would have otherwise created additional points may result in holes or crack during reconstruction of the point cloud. A method which moves occupied samples to reduce redundant pixels may, for irregular shapes, result in redundant pixels.

In some embodiments, to improve upon such methods and to remove redundant output points, the width, height, and placement of a patch in an occupancy map may be adjusted. Point Cloud Resampling

In some embodiments, a point cloud resampling module, such as point cloud resampling module 252, may resample a point cloud as described below.

In some embodiments, dynamic point clouds may have a different number of points from one frame to another. Efficient temporal prediction may require mapping the points of the current frame, denoted CF, to the points of a reference frame, denoted RF. Signaling such a mapping in a bit stream may require a high number of bits and thus may be inefficient. Instead, re-sampling of a current frame CF may be performed so that the current frame CF has the same number of points as reference frame RF. More precisely, the points of reference frame RF may be displaced such that its shape matches the shape of current frame CF. As a second step, the color and attributes of current frame CF may be transferred to the deformed version of reference frame RF. The obtained frame CF' may be considered as the re-sampled version of the current frame. The decision to compress the approximation CF' of CF may be made by comparing the rate distortion costs of both options (e.g., encoding CF' as inter-frame vs. encoding CF as intra-frame). In some embodiments, pre-adjusting RF may be performed in an effort to make it a better reference for future CF images. Resampling may comprise the following:

- a. First, normals of the points associated with current frame CF and reference frame RF may be estimated and oriented consistently. For every point P belonging to current frame CF (resp. Q belonging to RF), let $\alpha(P)$ (resp., $\alpha(Q)$) be its position and $V(P)$ (resp., $V(Q)$) its

29

normal. A 6D vector, denoted $v(P)$ (resp., $v(Q)$) is then associated with every point by combining its position and a weighted version of its normal in the same vector.

$$v(P) = \begin{bmatrix} \alpha(P) \\ \varepsilon \nabla(P) \end{bmatrix} \quad v(Q) = \begin{bmatrix} \alpha(Q) \\ \varepsilon \nabla(Q) \end{bmatrix},$$

where ε is a parameter controlling the importance of normal for positions. ε could be defined by the user or could be determined by applying an optimization procedure. They could also be fixed of adaptive.

- b. Two mappings from reference frame RF to current frame CF and from current frame CF to reference frame RF are computed as follows:

- i. Every point Q of reference frame RF is mapped to the point P(Q) of current frame CF that has the minimum distance to Q in the 6D space defined in the previous step.
- ii. Every point P of current frame CF is mapped to the point Q(P) of reference frame RF that has the minimum distance to P in the 6D space defined in the previous step. Let $p(Q)$ be the set of points of current frame CF that are mapped to the same point Q.

- c. At each iteration

- i. The positions of the points of reference frame RF are updated as follows:

$$\alpha'(Q) = w \cdot \alpha(P(Q)) + \frac{(1-w)}{|p(Q)|} \sum_{P \in p(Q)} \alpha(P),$$

where $|p(Q)|$ is the number of elements of $p(Q)$. The parameter w could be defined by the user or could be determined by applying an optimization procedure. It could also be fixed or adaptive.

- ii. The previous updated step results usually in an irregular repartition of the points. In order to overcome such limitations, a Laplacian-based smoothing procedure is applied. The idea is to update the positions of the points such that they stay as close as possible to $\{\alpha'(Q)\}$, while favoring a repartition as close as possible to the original point repartition in reference frame RF. More precisely, the following sparse linear system may be solved:

$$\{\alpha^*(Q)\} = \underset{\{\alpha'(Q)\}}{\operatorname{argmin}} \left\{ \sum_{Q \in RF} \|\alpha''(Q) - \alpha'(Q)\|^2 + \gamma \sum_{Q \in RF} \left\| \alpha''(Q) - \frac{1}{R} \sum_{Q' \in N(Q)} \alpha''(Q') - \alpha(Q) - \frac{1}{R} \sum_{Q' \in N(Q)} \alpha(Q') \right\|^2 \right\},$$

where $N(Q)$ is the set of the R nearest neighbors of Q in reference frame RF.

- iii. The mappings between the updated RF' point cloud and current frame CF are then updated as follows

1. Every point Q of RF' is mapped to the point P(Q) of current frame CF that has the minimum distance to Q in the 3D space of positions.
2. Every point P of current frame CF is mapped to the point Q(P) of reference frame RF that has the minimum distance to P in the 3D space of positions. Let $p(Q)$ be the set of points of current frame CF that are mapped to the same point Q.

- d. This process is iterated until a pre-defined number of iterations is reached or there is no further change.

30

- e. At this stage, the color and attribute information is transferred from current frame CF to RF' by exploiting the following formula

$$A(Q) = w(A) \cdot A(P(Q)) + \frac{(1-w(A))}{|p(Q)|} \sum_{P \in p(Q)} A(P),$$

where A stands for the texture or attribute to be transferred, $|p(Q)|$ is the number of elements of $p(Q)$. The parameter $w(A)$ could be defined by the user or could be determined by applying an optimization procedure. It could also be fixed of adaptive.

3D Motion Compensation

In some embodiments, the positions, attributes and texture information may be temporally predicted by taking the difference between the value at current resampled frame minus a corresponding value, e.g. motion compensated value, from the reference frame. These values may be fed to the image generation stage to be stored as images. For example, such techniques may be performed by 3D motion compensation and delta vector prediction module 254.

Smoothing Filter

In some embodiments, a smoothing filter of a decoder, such as smoothing filter 244 or smoothing filter 276 of decoder 230 or decoder 280, may perform smoothing as described below.

In some embodiments, a reconstructed point cloud may exhibit discontinuities at the patch boundaries, especially at very low bitrates. In order to alleviate such a problem, a smoothing filter may be applied to the reconstructed point cloud. Applying the smoothing filter may comprise:

- a. By exploiting the occupancy map, both the encoder and the decoder may be able to detect boundary points, which are defined as being points belonging to $B_0 \times B_0$ blocks encoded during the last iteration of the hierarchical occupancy map compression procedure described in previous sections above.
- b. The boundary points may have their positions/attribute/texture updated. More precisely, respective boundary points may be assigned a smoothed position based on its R nearest neighbors in the point cloud. The smoothed position may be the centroid/median of the nearest neighbors. Another option may comprise fitting a plane or any smooth surface the nearest neighbor and assigning as a smoothed position the projection of the point on that surface. The number of parameters and/or the smoothing strategy may be chosen by a user or determined by applying an optimization strategy. They may be fixed for all the points or chosen adaptively. These parameters may be signaled in the bit stream.
- c. In order to reduce the computational complexity of the smoothing stage, a subsampled version of the reconstructed point cloud may be considered when looking for the nearest neighbors. Such subsampled version could be efficiently derived by considering a subsampled version of the geometry image and the occupancy map.

Closed-Loop Color Conversion

In some embodiments, an encoder and/or decoder for a point cloud may further include a color conversion module to convert color attributes of a point cloud from a first color space to a second color space. In some embodiments, color attribute information for a point cloud may be more efficiently compressed when converted to a second color space. For example, FIGS. 4A and 4B illustrates encoders 400 and

which are similar encoders as illustrated in FIGS. 2A and 2C, but that further include color conversion modules 402 and 404, respectively. While not illustrated, decoders such as the decoders illustrated in FIGS. 2B and 2D, may further include color conversion modules to convert color attributes of a decompressed point cloud back into an original color space, in some embodiments.

FIG. 4C illustrates components of a closed-loop color conversion module, according to some embodiments. The closed-loop color conversion module 410 illustrated in FIG. 4C may be a similar closed-loop color conversion module as closed-loop color conversion modules 402 and 404 illustrated in FIGS. 4A and 4B.

In some embodiments, a closed-loop color conversion module, such as closed-loop color conversion module 410, receives a compressed point cloud from a video encoder, such as video compression module 218 illustrated in FIG. 4A or video compression module 264 illustrated in FIG. 4B. Additionally, a closed-loop color conversion module, such as closed-loop color conversion module 410, may receive attribute information about an original non-compressed point cloud, such as color values of points of the point cloud prior to being down-sampled, up-sampled, color converted, etc. Thus, a closed-loop color conversion module may receive a compressed version of a point cloud such as a decoder would receive and also a reference version of the point cloud before any distortion has been introduced into the point cloud due to sampling, compression, or color conversion.

In some embodiments, a closed-loop color conversion module, such as closed-loop color conversion module 410, may include a video decompression module, such as video decompression module 270, and a geometry reconstruction module, such as geometry reconstruction module 412. A video decompression module may decompress one or more video encoded image frames to result in decompressed image frames each comprising one or more patch images packed into the image frame. A geometry reconstruction module, such as geometry reconstruction module 412, may then generate a reconstructed point cloud geometry. A re-coloring module, such as re-coloring module 414, may then determine colors for points in the point cloud based on the determined reconstructed geometry. For example, in some embodiments, a nearest neighbor approach or other approach may be used to determine estimated color values for points of the point cloud based on sub-sampled color information, wherein a color value is not explicitly encoded for each point of the point cloud. Because there may be losses during the patching process, compression process, decompression process, and geometry reconstruction process, the geometry of the points in the reconstructed point cloud may not be identical to the geometry in the original point cloud. Due to this discrepancy, color compression techniques that rely on geometrical relationships between points to encode color values may result in colors that are slightly different when decoded and decompressed than the original colors. For example, if a color is to be determined based on color values of the nearest neighboring points, a change in geometry may cause a different nearest neighbor to be selected to determine the color value for the point at the decoder than was selected to encode a residual value at the encoder. Thus distortion may be added to the decoded decompressed point cloud.

If a color space conversion module does not account for this distortion that takes place when converting a point cloud into patches packed in an image frame and that takes place when encoding the image frames, the color space conversion

module may select less than optimum color conversion parameters, such as luma and chroma values. For example, optimum color conversion parameters that cause a packed image frame in a first color space to closely match the packed image frame converted into a second color space may be different than optimum color conversion parameters when upstream and downstream distortions are accounted for.

In order to account for such distortions, a texture/attribute image color space conversion and re-sampling module, such as module 416, may take into account a difference between the “re-created” color values from re-coloring module 416 and the original color values from the original non-compressed reference point cloud when determining color conversion parameters for converting an image frame from a first color space, such as R'G'B' 4:4:4 to YCbCr 4:2:0, for example. Thus, the color-converted and re-sampled texture/attribute images provided to video encoder 218 and 264, as shown in FIG. 4C may take into account distortion introduced at any stage of compression and decompression of a point cloud, and may utilize optimum color conversion parameters taking into account such distortion.

Such methods may result in considerably reduced distortion when reconstructing the point cloud representation, while maintaining the high compressibility characteristics of the 4:2:0 signal.

In some embodiments, conversion from 4:4:4 R'G'B' to a 4:2:0 YCbCr representation is performed using a 3×3 matrix conversion of the form:

$$\begin{bmatrix} Y' \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} a_{YR} & a_{YG} & a_{YB} \\ a_{CbR} & a_{CbG} & a_{CbB} \\ a_{CrR} & a_{CrG} & a_{CrB} \end{bmatrix} \begin{bmatrix} R' \\ G' \\ B' \end{bmatrix}$$

In the above matrix, Y' is the luma component and Cb and Cr are the chroma components. The values of R', G', and B' correspond to the red, green, and blue components respectively, after the application of a transfer function that is used to exploit the psycho-visual characteristics of the signal. The coefficients a_{YR} through a_{CrB} are selected according to the relationship of the red, green, and blue components to the CIE 1931 XYZ color space. Furthermore, the Cb and Cr components are also related to Y' in the following manner:

$$Cb = \frac{B' - Y'}{\alpha} \quad \text{with } \alpha = 2 * (1 - a_{YB})$$

$$Cr = \frac{R' - Y'}{\beta} \quad \text{with } \beta = 2 * (1 - a_{YR})$$

with also the following relationships:

$$a_{CbR} = -\frac{a_{YR}}{2 * (1 - a_{YB})}$$

$$a_{CbG} = -\frac{a_{YG}}{2 * (1 - a_{YB})}$$

$$a_{CbB} = 0.5$$

$$a_{CrR} = 0.5$$

$$a_{CrG} = -\frac{a_{YG}}{2 * (1 - a_{YR})}$$

$$a_{CrB} = -\frac{a_{YB}}{2 * (1 - a_{YR})}$$

The process described above is followed by a 2× down-sampling horizontally and vertically of the chroma components, resulting in chroma components that are 4 times smaller, in terms of overall number of samples, 2× smaller horizontally and 2× smaller vertically, compared to those of luma. Such a process can help not only with compression but also with bandwidth and processing complexity of the YCbCr 4:2:0 signals.

In using such an approach quantization for the color components, as well as the down sampling and up sampling processes for the chroma components, may introduce distortion that could impact the quality of the reconstructed signals especially in the R'G'B' but also in the XYZ (CIE 1931 domains). However, a closed loop conversion process, where the chroma and luma values are generated while taking into account such distortions, may considerably improve quality.

In a luma adjustment process, for example, the chroma components may be converted using the above formulations, additionally a down sampling and up sampling may be performed given certain reference filtering mechanisms. Afterwards, using the reconstructed chroma samples, an appropriate luma value may be computed that would result in minimal distortion for the luminance Y component in the CIE 1931 XYZ space. Such luma value may be derived through a search process instead of a direct computation method as provided above. Refinements and simplifications of this method may include interpolative techniques to derive the luma value.

Projected point cloud images can also benefit from similar strategies for 4:2:0 conversion. For example, closed loop color conversion, including luma adjustment methods may be utilized in this context. That is, instead of converting point cloud data by directly using the 3×3 matrix above and averaging all neighboring chroma values to generate the 4:2:0 chroma representation for the projected image, one may first project point cloud data/patches using the R'G'B' representation on a 4:4:4 grid. For this new image one may then convert to the YCbCr 4:2:0 representation while using a closed loop optimization such as the luma adjustment method. Assuming that the transfer characteristics function is known, e.g. BT.709, ST 2084 (PQ), or some other transfer function as well as the color primaries of the signal, e.g. BT.709 or BT.2020, an estimate of the luminance component Y may be computed before the final conversion. Then the Cb and Cr components may be computed, down sampled and up sampled using more sophisticated filters. This may then be followed with the computation of the Y' value that would result in a luminance value Y_{recon} that would be as close as possible to Y. If distortion in the RGB domain is of higher distortion, a Y' value that minimizes the distortion for R', G', and B' jointly, could be considered instead.

For point cloud data, since geometry may also be altered due to lossy compression, texture distortion may also be impacted. In particular, overall texture distortion may be computed by first determining for each point in the original and reconstructed point clouds their closest point in the reconstructed and original point clouds respectively. Then the RGB distortion may be computed for those matched points and accumulated across the entire point cloud image. This means that if the geometry was altered due to lossy compression, the texture distortion would also be impacted. Given that the texture may have been distorted, it may be desirable to consider geometry during closed loop conversion of chroma.

In some embodiments, the geometry is modified so that the relative sampling density in a given region of the point

cloud is adjusted to be similar to other regions of the point cloud. Here the relative sampling density is defined as density of original points relative to the uniform 2D sampling grid.

Because the relative sampling density can vary within a given patch, this information can be used to guide the patch decomposition process as described above in regard to occupancy maps and auxiliary information, where patch approximation is used to determine local geometry. Furthermore, this information can be used to guide encoding parameters to achieve more uniform quality after compression. If a local region has higher relative sampling density, the encoder may code that region better through a variety of means. The variety of means may include: variable block size decision, Quantization Parameters (QPs), quantization rounding, de-blocking, shape adaptive offset (SAO) filtering, etc.

In some embodiments, the geometry information is first compressed according to a target bitrate or quality, and then it is reconstructed before generating the texture projected image. Then, given the reconstructed geometry, the closest point in the reconstructed point cloud is determined that corresponds to each point in the original point cloud. The process may be repeated for all points in the reconstructed point cloud by determining their matched points in the original point cloud. It is possible that some points in the reconstructed point cloud may match multiple points in the original point cloud, which would have implications in the distortion computation. This information may be used in the closed loop/luma adjustment method so as to ensure a minimized texture distortion for the entire point cloud. That is, the distortion impact to the entire point cloud of a sample Pr at position (x,y,z) in the reconstructed point cloud can be computed (assuming the use of MSE on YCbCr data for the computation):

$$D(Pr) = D_{\text{original}}(Pr) + D_{\text{reconstructed}}(Pr)$$

$$D(Pr) = \text{Sum_matching}(((Y_{pr} - Y_{or(i)})^2 + (Cb_{pr} - Cb_{or(i)})^2 + (Cr_{pr} - Cr_{or(i)})^2) + \text{sqrt}((Y_{pr} - Y_{or})^2 + (Cb_{pr} - Cb_{or})^2 + (Cr_{pr} - Cr_{or})^2))$$

In the above equation, Y_{pr}, Cb_{pr}, and Cr_{pr} are the luma and chroma information of point Pr, Y_{or(i)}, Cb_{or(i)}, and Cr_{or(i)} correspond to the luma and chroma information of all the points that were found to match the geometry location of point Pr from the original image, and Y_{or}, Cb_{or}, and Cr_{or} is the point that matches the location of point Pr in the original as seen from the reconstructed image.

If the distortion computation in the context of closed loop conversion/luma adjustment utilizes D(Pr), then better performance may be achieved since it not only optimizes projected distortion, but also point cloud distortion. Such distortion may not only consider luma and chroma values, but may instead or additionally consider other color domain components such as R, G, or B, luminance Y, CIE 1931 x and y, CIE 1976 u' and v', YCoCg, and the ICtCp color space amongst others.

If geometry is recompressed a different optimal distortion point may be possible. In that case, it might be appropriate to redo the conversion process once again.

In some embodiments, texture distortion, as measured as described below, can be minimized as follows:

Let $(Q(i))_{i \in \{1, \dots, N\}}$ and $(P_{rec}(i))_{i \in \{1, \dots, N_{rec}\}}$ be the original and the reconstructed geometries, respectively. Let N and N_{rec} be the number of points in the original and the reconstructed point clouds, respectively.

For each point P_{rec}(i) in the reconstructed point cloud, let Q*(i) be its nearest neighbor in the original point cloud

35

and $R(Q^*(i))$, $G(Q^*(i))$, and $B(Q^*(i))$ the RGB values associated with $Q^*(i)$. For each point $P_{rec}(i)$ in the reconstructed point cloud, let $\mathbb{Q}^+(i) = (Q^+(i, h))_{h \in \{1, \dots, H(i)\}}$ be the set of point in the original point cloud that share $P_{rec}(i)$ as their nearest neighbor in the reconstructed point cloud. Note that $\mathbb{Q}^+(i)$ could be empty or could have one or multiple elements.

If $\mathbb{Q}^+(i)$ is empty, then the RGB values $R(Q^*(i))$, $G(Q^*(i))$, and $B(Q^*(i))$ are associated with the point $P_{rec}(i)$.

If $\mathbb{Q}^+(i)$ is not empty, then proceed as follows:

Virtual RGB values, denoted $R(\mathbb{Q}^+(i))$, $G(\mathbb{Q}^+(i))$, and $B(\mathbb{Q}^+(i))$, are computed as follows:

$$R(\mathbb{Q}^+(i)) = \frac{1}{H(i)} \sum_{h=1}^{H(i)} R(Q^+(i, h))$$

$$G(\mathbb{Q}^+(i)) = \frac{1}{H(i)} \sum_{h=1}^{H(i)} G(Q^+(i, h))$$

$$B(\mathbb{Q}^+(i)) = \frac{1}{H(i)} \sum_{h=1}^{H(i)} B(Q^+(i, h))$$

Note that $R(\mathbb{Q}^+(i))$, $G(\mathbb{Q}^+(i))$, and $B(\mathbb{Q}^+(i))$ correspond to the average RGB values of the points of $\mathbb{Q}^+(i)$.

The final RGB values $R(P_{rec}(i))$, $G(P_{rec}(i))$, and $B(P_{rec}(i))$ are obtained by applying the following linear interpolation:

$$R(P_{rec}(i)) = w R(\mathbb{Q}^+(i)) + (1-w) R(Q^*(i))$$

$$G(P_{rec}(i)) = w G(\mathbb{Q}^+(i)) + (1-w) G(Q^*(i))$$

$$B(P_{rec}(i)) = w B(\mathbb{Q}^+(i)) + (1-w) B(Q^*(i))$$

The interpolation parameter w is chosen such that the following cost function $C(i)$ is minimized

$$C(i) = \max \left\{ \frac{1}{N} \sum_{h=1}^{H(i)} \{ (R(P_{rec}(i)) - R(Q^+(i, h)))^2 + (G(P_{rec}(i)) - G(Q^+(i, h)))^2 + (B(P_{rec}(i)) - B(Q^+(i, h)))^2 \}, \right. \\ \left. \frac{1}{N_{rec}} \{ (R(P_{rec}(i)) - R(Q^*(i)))^2 + (G(P_{rec}(i)) - G(Q^*(i)))^2 + (B(P_{rec}(i)) - B(Q^*(i)))^2 \} \right\}$$

Note that by minimizing the cost $C(i)$, the distortion measure as described below is minimized

Different search strategies may be used to find the parameter w

Use the closed form solution described below.

No search: use $w=0.5$.

Full search: choose a discrete set of values $(w_i)_{i=1 \dots w}$ in the interval $[0,1]$ and evaluate $C(i)$ for these values in order to find the w^* , which minimizes $C(i)$.

Gradient descent search: start with $w=0.5$. Evaluate $E1(i)$, $E2(i)$ and $C(i)$. Store $C(i)$ and w as the lowest cost and its associated interpolation parameter w . If $E1(i) > E2(i)$, update w based on the gradient of $E1(i)$, else use the gradient of $E2(i)$. Re-evaluate $E1(i)$, $E2(i)$, and $C(i)$ at the new value of w . Compare the new cost $C(i)$ to the lowest cost found so far. If new cost is higher than the lowest

36

cost stop, else update the lowest cost and the associated value of w , and continue the gradient descent, where $R(P_{rec}(i))$, $G(P_{rec}(i))$, and $B(P_{rec}(i))$ are the three unknowns to be determined.

In some embodiments, the above process could be performed with other color spaces and not necessarily the RGB color space. For example, the CIE 1931 XYZ or xyY, CIE 1976 Yu'v', YCbCr, IPT, ICtCp, La*b*, or some other color model could be used instead. Furthermore, different weighting of the distortion of each component could be considered. Weighting based on illumination could also be considered, e.g. weighting distortion in dark areas more than distortion in bright areas. Other types of distortion, that include neighborhood information, could also be considered. That is, visibility of errors in a more sparse area is likely to be higher than in a more dense region, depending on the intensity of the current and neighboring samples. Such information could be considered in how the optimization is performed.

Down sampling and up sampling of chroma information may also consider geometry information, if available. That is, instead of down sampling and up sampling chroma information without consideration to geometry, the shape and characteristics of the point cloud around the neighborhood of the projected sample may be considered, and appropriately consider or exclude neighboring samples during these processes. In particular, neighboring samples for down sampling or interpolating may be considered that have a normal that is as similar as possible to the normal of the current sample. Weighting during filtering according to the normal difference as well as distance to the point may also be considered. This may help improve the performance of the down sampling and up sampling processes.

It should be noted that for some systems, up sampling of the Cb/Cr information may have to go through existing architectures, e.g. an existing color format converter, and it might not be possible to perform such guided up sampling. In those cases, only considerations for down sampling may be possible.

In some embodiments, it may be possible to indicate in the bit stream syntax the preferred method for up sampling the chroma information. A decoder (included in an encoder), in such a case, may try a variety of up sampling filters or methods, find the best performing one and indicate that in the bit stream syntax. On the decoder side, the decoder may know which up sampling method would perform best for reconstructing the full resolution YCbCr and consequently RGB data. Such method could be mandatory, but could also be optional in some architectures.

Clipping as well as other considerations for color conversion, may also apply to point cloud data and may be considered to further improve the performance of the point cloud compression system. Such methods may also apply to other color representations and not necessarily YCbCr data, such as the YCoCg and ICtCp representation. For such representations different optimization may be required due to the nature of the color transform.

Example Objective Evaluation Method

A point cloud consists of a set of points represented by (x, y, z) and various attributes of which color components (y, u, v) are of importance. First, define the point v . It has as a mandatory position in a 3D space (x, y, z) and an optional color attribute c that has components r, g, b or y, u, v and optional other attributes possibly representing normal or texture mappings.

$$\text{point } v = ((x, y, z), [c], [a_0 \dots a_A]) : x, y, z \in R, \\ [c \in (r, g, b) \vee (y, u, v), b \in N], [a_i \in [0, 1]]$$

(def. 1)

37

The point cloud is then a set of K points without a strict ordering:

Original Point Cloud $V_{or} = \{(v_i); i=0 \dots K-1\}$ (def. 2)

The point cloud comprises a set of (x,y,z) coordinates and attributes that can be attached to the points. The original point cloud Vor (420) will act as the reference for determining the quality of a second degraded point cloud Vdeg (424). Vdeg consists of N points, where N does not necessarily=K. Vdeg is a version of the point cloud with a lower quality possibly resulting from lossy encoding and decoding of Vor (e.g. operation 422). This can result in a different point count N.

Degraded Point Cloud $V_{deg} = \{(v_i); i=0 \dots N-1\}$ (def. 3)

The quality metric Q_(point cloud) is computed from Vor and Vdeg and used for assessment as shown in FIG. 4D for full reference quality metric 426.

Table 3, below, outlines the metrics used for the assessment of the quality of a point cloud, in some embodiments. The geometric distortion metrics are similar as ones used for meshes based on haussdorf (Lint) and root mean square (L2), instead of distance to surface. This approach takes the distance to the closest/most nearby point in the point cloud (see definitions 4, 5, 6, and 7) into account. Peak signal to noise ratio (PSNR) is defined as the peak signal of the geometry over the symmetric Root Mean Square (RMS/rms) distortion (def 8.). For colors, a similar metric is defined; the color of the original cloud is compared to the most nearby color in the degraded cloud and peak signal to noise ratio (PSNR) is computed per YUV/YCbCr component in the YUV color space (def. 10). An advantage of this metric is that it corresponds to peak signal to noise ratio (PSNR) in Video Coding. The quality metric is supported in the 3DG PCC software.

TABLE 3

Assessment criteria for assessment of the point cloud quality of Vdeg, Q_{point_cloud}		
d_symmetric_rms	Symmetric rms distance between the point clouds	(def. 5.)
d_symmetric_haussdorf	Symmetric haussdorf distance between the clouds	(def. 7.)
psnr_geom	Peak signal to noise ratio geometry (vertex positions)	(def. 8.)
psnr_y	Peak signal to noise ratio geometry (colors Y)	(def. 10)
psnr_u	Peak signal to noise ratio geometry (colors U)	(as def. 10 rep. y for u)
psnr_v	Peak signal to noise ratio geometry (colors V)	(as def. 10 rep. y for v)
$d_{rms}(V_{or}, V_{deg}) = \sqrt{\frac{1}{K} \sum_{v_o \in V_{or}} \ v_o - v_{d_nearest_neighbour}\ ^2}$		
(def. 4)		
$d_{symmetric_rms}(V_{or}, V_{deg}) = \max(d_{rms}(V_{or}, V_{deg}), d_{rms}(V_{deg}, V_{or}))$		
(def. 5)		
$d_{haussdorf}(V_{or}, V_{deg}) = \max_{v_o \in V_{or}} (\ v_o - v_{d_nearest_neighbour}\ _2, v_d \text{ is the point in Vdeg closest to } v_o \text{ (L2)})$		
(def. 6)		
$d_{symmetric_haussdorf}(V_{or}, V_{deg}) = \max(d_{haussdorf}(V_{or}, V_{deg}), d_{haussdorf}(V_{deg}, V_{or}))$		
(def. 7)		
$BBwidth = \max((x_{max} - x_{min}), (y_{max} - y_{min}), (z_{max} - z_{min}))$		
(def. 8)		
$psnr_{geom} = 10 \log_{10} (BBwidth \ l_2\ ^2 / (d_{symmetric_rms}(V))^2)$		
(def. 9)		
$d_y(V_{or}, V_{deg}) = \sqrt{\frac{1}{K} \sum_{v_o \in V_{or}} \ y(v_o) - y(v_{d_nearest_neighbour})\ ^2}$		
(def. 10)		
$psnr_y = 10 \log_{10} (125511^2 / (d_y(V_{or}, V_{deg}))^2)$		
(def. 11)		

38

In some embodiments, additional metrics that define the performance of a codec are outlined below in Table 4.

TABLE 4

Additional Performance Metrics	
Compressed size	Complete compressed mesh size
In point count	K, the number of vertices in Vor
Out point count	N, number of vertices in Vdeg
Bytes_geometry_layer	Number of bytes for encoding the vertex positions
Bytes_color_layer (opt)	Number of bytes for encoding the colour attributes
Bytes_att_layer (opt)	Number of bytes for encoding the other attributes
Encoder time (opt)	Encoder time in ms on commodity hardware (optional)
Decoder time (opt)	Decoder time in ms on commodity hardware (optional)

Example Closed Form Solution

For each point $P_{rec}(i)$ in the reconstructed point cloud, let $Q^*(i)$ be its nearest neighbor in the original point cloud. For each point $P_{rec}(i)$ in the reconstructed point cloud, let $(Q^+(i, h))_{h \in \{1, \dots, H(i)\}}$ be the set of point in the original point cloud that share $P_{rec}(i)$ as their nearest neighbor in the reconstructed point cloud. Let $Q^+(i)$ be the centroid of $(Q^+(i, h))_{h \in \{1, \dots, H(i)\}}$.

If $H=0$, then $C(P_{rec}(i))=C(Q^*(i))$

Denote as R-G-B vector $C(P)$ associated with a given point P. In order to compute the color for a given $P_{rec}(i)$, we have the following formulation:

argmin $\max_{C(P_{rec}(i))}$

$$\left\{ \frac{1}{N_{rec}} \|C(P_{rec}(i)) - C(Q^*(i))\|^2, \frac{1}{N} \sum_{h=1}^H \|C(P_{rec}(i)) - C(Q^+(i, h))\|^2 \right\}$$

Where $\max \left\{ \frac{1}{N_{rec}} \|C(P_{rec}(i)) - C(Q^*(i))\|^2, \right.$

$$\left. \sum_{h=1}^H \|C(P_{rec}(i)) - C(Q^+(i)) + C(Q^+(i)) - C(Q^+(i, h))\|^2 \right\} =$$

$$\max \left\{ \frac{1}{N_{rec}} \|C(P_{rec}(i)) - C(Q^*(i))\|^2, \right.$$

$$\left. \frac{H}{N} \|C(P_{rec}(i)) - C(Q^+(i))\|^2 + \frac{1}{N} \sum_{h=1}^H \|C(Q^+(i)) - C(Q^+(i, h))\|^2 + \right.$$

$$\left. \frac{2}{N} \sum_{h=1}^H \langle C(P_{rec}(i)) - C(Q^+(i)), C(Q^+(i)) - C(Q^+(i, h)) \rangle \right\} =$$

$$\max \left\{ \frac{1}{N_{rec}} \|C(P_{rec}(i)) - C(Q^*(i))\|^2, \right.$$

$$\left. \frac{H}{N} \|C(P_{rec}(i)) - C(Q^+(i))\|^2 + \frac{1}{N} \sum_{h=1}^H \|C(Q^+(i)) - C(Q^+(i, h))\|^2 \right\}$$

argmin $\max_{C(P_{rec}(i))}$

$$\left\{ \frac{1}{N_{rec}} \|C(P_{rec}(i)) - C(Q^*(i))\|^2, \frac{1}{N} \sum_{h=1}^H \|C(P_{rec}(i)) - C(Q^+(i, h))\|^2 \right\}$$

39

-continued

$$\begin{aligned}
& \text{Where } \max \left\{ \frac{1}{N_{rec}} \|C(P_{rec}(i)) - C(Q^*(i))\|^2, \right. \\
& \quad \left. \sum_{h=1}^H \|C(P_{rec}(i)) - C(Q^+(i)) + C(Q^+(i)) - C(Q^+(i, h))\|^2 \right\} = \\
& \quad \max \left\{ \frac{1}{N_{rec}} \|C(P_{rec}(i)) - C(Q^*(i))\|^2, \right. \\
& \quad \frac{H}{N} \|C(P_{rec}(i)) - C(Q^+(i))\|^2 + \frac{1}{N} \sum_{h=1}^H \|C(Q^+(i)) - C(Q^+(i, h))\|^2 + \\
& \quad \left. \frac{2}{N} \sum_{h=1}^H \langle C(P_{rec}(i)) - C(Q^+(i)), C(Q^+(i)) - C(Q^+(i, h)) \rangle \right\} = \\
& \quad \max \left\{ \frac{1}{N_{rec}} \|C(P_{rec}(i)) - C(Q^*(i))\|^2, \right. \\
& \quad \left. \frac{H}{N} \|C(P_{rec}(i)) - C(Q^+(i))\|^2 + \frac{1}{N} \sum_{h=1}^H \|C(Q^+(i, h))\|^2 \right\}
\end{aligned}$$

Now denote $D^2 = \sum_{h=1}^H \|C(Q^+(i)) - C(Q^+(i, h))\|^2$, so that

argmin max
 $C(P_{rec}(i))$

$$\left\{ \frac{1}{N_{rec}} \|C(P_{rec}(i)) - C(Q^*(i))\|^2, \frac{H}{N} \|C(P_{rec}(i)) - C(Q^+(i))\|^2 + \frac{D^2}{N} \right\}.$$

Note: if $H=1$ then $D^2=0$

Let $C^0(P_{rec}(i))$ be a solution of the previous minimization problem. It can be shown that $C^0(P_{rec}(i))$ could be expressed as:

$$C^0(P_{rec}(i)) = wC(Q^*(i)) + (1-w)C(Q^+(i))$$

Furthermore, $C^0(P_{rec}(i))$ verifies:

$$\begin{aligned}
& \frac{1}{N_{rec}} \|wC(Q^*(i)) + (1-w)C(Q^+(i)) - C(Q^*(i))\|^2 = \\
& \quad \frac{H}{N} \|wC(Q^*(i)) + (1-w)C(Q^+(i)) - C(Q^+(i))\|^2 + \frac{D^2}{N} \\
& (1-w)^2 \|C(Q^+(i)) - C(Q^*(i))\|^2 = w^2 \frac{HN_{rec}}{N} \|C(Q^*(i)) - C(Q^+(i))\|^2 + \frac{D^2 N_{rec}}{N}
\end{aligned}$$

$$\text{Let } \delta^2 = \|C(Q^*(i)) - C(Q^+(i))\|^2 \text{ and } r = \frac{N_{rec}}{N}$$

$$\text{If } \delta^2 = 0, \text{ then } C(P_{rec}(i)) = C(Q^*(i)) = C(Q^+(i))$$

$$(1-w)^2 \delta^2 = w^2 r H \delta^2 + r D^2$$

$$\delta^2 + w^2 \delta^2 - 2w\delta^2 = w^2 r H \delta^2 + r D^2$$

$$\delta^2 (1 - rH) w^2 - 2\delta^2 w + (\delta^2 - rD^2) = 0$$

$$(rH - 1)w^2 + 2w + (\alpha r - 1) = 0$$

$$\text{With } \alpha = \frac{D^2}{\delta^2}$$

$$\text{if } H = 1, \text{ then } w = \frac{1}{2}$$

$$\text{if } H > 1$$

$$\Delta = 4 - 4(rH - 1)(\alpha r - 1)$$

40

-continued

$$\Delta = 4 - 4(rH - 1) \alpha r + 4H - 4$$

$$\Delta = 4(H - (rH - 1) \alpha r)$$

$$\text{If } \Delta = 0$$

$$w = \frac{-1}{(rH - 1)}$$

$$\text{If } \Delta > 0$$

$$w1 = \frac{-1 - \sqrt{(H - (rH - 1) \alpha r)}}{(rH - 1)}$$

$$w2 = \frac{-1 + \sqrt{(H - (rH - 1) \alpha r)}}{(rH - 1)}$$

Where the cost $C(i)$ is computed for both $w1$ and $w2$ and the value that leads to the minimum cost is retained as the final solution.

Compression/Decompression Using Multiple Resolutions

FIG. 5A illustrates components of an encoder that includes geometry, texture, and/or attribute downscaling, according to some embodiments. Any of the encoders described herein may further include a spatial down-scaler component 502, a texture down-scaler component 504, and/or an attribute down-scaler component 506 as shown for encoder 500 in FIG. 5A. For example, encoder 200 illustrated in FIG. 2A may further include downscaling components as described in FIG. 5A. In some embodiments, encoder 250 may further include downscaling components as described in FIG. 5A.

In some embodiments, an encoder that includes downscaling components, such as geometry down-scaler 502, texture down-scaler 504, and/or attribute down-scaler 506, may further include a geometry up-scaler, such as spatial up-scaler 508, and a smoothing filter, such as smoothing filter 510. In some embodiments, a reconstructed geometry image is generated from compressed patch images, compressed by video compression module 218. In some embodiments an encoder may further include a geometry reconstruction module (not shown) to generate the reconstructed geometry image. The reconstructed geometry image may be used to encode and/or improve encoding of an occupancy map that indicates patch locations for patches included in one or more frame images. Additionally, the reconstructed geometry image may be provided to a geometry up-scaler, such as spatial up-scaler 508. A geometry up-scaler may scale the reconstructed geometry image up to an original resolution or a higher resolution approximating the original resolution of the geometry image, wherein the original resolution is a resolution prior to downscaling being performed at spatial down-scaler 502. In some embodiments, the up-scaled reconstructed geometry image may be provided to a smoothing filter that generates a smoothed image of the reconstructed and up-scaled geometry image. The smoothing filter, such as smoothing filter 510, may approximate smoothing that would be applied via a smoothing filter of a decoder. This information may then be provided to the spatial image generation module 210, texture image generation module 212, and/or the attribute image generation module 214. These modules may adjust generation of spatial images, texture images, and/or other attribute images based on the reconstructed geometry images. For example, if a patch shape (e.g. geometry) is slightly distorted during the downscaling, encoding, decoding, and upscaling process, these changes may be taken into account when generating spatial images, texture images, and/or other attribute images

41

to correct for the changes in patch shape (e.g. distortion). As an example, points of the point cloud represented in the reconstructed geometry image may be slightly moved as compared to the locations of the points in the original geometry image. In such circumstances, a texture image

FIG. 5B illustrates components of a decoder 520 that includes geometry, texture, and/or other attribute upscaling, according to some embodiments. For example, decoder 520 includes texture up-scaler 512, attribute up-scaler 514, and spatial up-scaler 516. Any of the decoders described herein may further include a texture up-scaler component 512, an attribute up-scaler component 514, and/or a spatial image up-scaler component 516 as shown for decoder 520 in FIG. 5B.

IG. 5C illustrates rescaling from the perspective of an encoder, according to some embodiments. In some embodiments, a point cloud may be scaled in both the point cloud domain (e.g. 3D domain prior to patch projection) and in a video level domain (e.g. by scaling 2D image frames comprising patch information). For example FIG. 5C illustrates a point cloud 522 of a person. An encoder, such as encoder 500, performs 3D scaling of the point cloud 522 in the point cloud domain to generate a downsampled point cloud 524. Patches generated based on downsampled point cloud 524 are packed into image frame 526. Additionally, downscaling is performed on the image frame 526 at the video level to reduce a resolution of the image frame. The additional downscaling results in a downsampled image frame 528 that is then encoded into a bit stream 530.

FIG. 5D illustrates rescaling from the perspective of a decoder, according to some embodiments. In some embodiments, a decoder, such as decoder 520, may receive a bit stream, such as bit stream 530. The decoder may decode the video encoded bit stream to generate one or more video image frames, such as image frame 532. The decoder may further upscale the image frame 532 to generate an up-scaled image frame 534. The decoder may then use a patch reconstruction method, as described above, to generate a reconstructed point cloud 536 from the patch information included in the up-scaled image frame 534. The decoder may also perform up-scaling in the 3D point cloud domain to scale up the reconstructed point cloud 536 to a similar size as the original point cloud. This process may result in an up-scaled reconstructed point cloud 538.

FIG. 5E illustrates an example open loop rescaling, according to some embodiments. In an open loop rescaling of an image frame, a geometry plane, and a texture or other attribute plane may be independently scaled, where geometry distortion is not taken into account when scaling the texture or other attribute information. For example, geometry image frame 540 may indicate depths of points of a point cloud relative to a projection plane and texture or attribute image frame 544 may represent respective attributes of the points of the point cloud projected on to the projection plane. As shown in FIG. 5E, in an open loop rescaling process, the geometry information and the attribute information may be independently scaled to generate downsampled geometry image frame 542 and downsampled texture or attribute image frame 546, respectively. Also, as shown in FIG. 5E, the downsampled geometry image frame 542 may be video encoded/compressed to generate a geometry bit stream and the downsampled attribute image frame 546 may be video encoded/compressed to generate a texture or attribute bit stream, such as a texture/attribute bit stream. For

42

example, spatial down-scaler 502 may downscale the geometry image frame 540 and the texture down-scaler 504 may independently downscale the texture image frame 544. In some embodiments, attribute down-scaler 506 may downscale an attribute image frame independently of spatial down-scaler 502 and texture down-scaler 504. Because different down-scalers are used to downscale different types of image frames (e.g. spatial information, texture, other attributes, etc.), different downscaling parameters may be applied to the different types of image frames to downscale geometry different than texture or attributes.

FIG. 5F illustrates an example closed loop rescaling, according to some embodiments. In some embodiments, a closed loop rescaling process may be used by an encoder such as encoder 500 to determine distortion or other changes to geometry that may occur as part of a downscaling, encoding, decoding, and/or upscaling process. In some embodiments, such distortion may be accounted for when downscaling other attributes, such as texture. An encoder, such as encoder 500, receives a point cloud 548. The encoder generates a geometry image frame for the point cloud 548, for example an image frame comprising patches representing relative depths of the points, such as an original geometry image frame 550. A point cloud compression geometry mapper, which may include a decomposition into patches module 506, a packing module 208, and a spatial image generation module 210, etc., generates the original geometry image frame 550. A geometry down-scaler, such as spatial down-scaler 502 downscales the geometry image frame to generate downsampled geometry image frame 552. Note that “geometry plane” may be used to refer to geometry patch information, which may be included in an image frame only consisting of geometry patches as shown in FIG. 5F.

The downsampled geometry image frame 552 is compressed, for example by video compression module 218, and is converted into a geometry bit stream. In a closed loop process as shown in FIG. 5F, the geometry bit stream is decompressed at the encoder to generate a reconstructed geometry plane 554. The reconstructed geometry plane is then up-scaled, at the encoder, to generate an up-scaled reconstructed geometry plane 556.

The texture points of the original geometry image frame 550 are then mapped to the points of the reconstructed up-scaled geometry plane 556. Differences in locations of the points in the original geometry image frame and the reconstructed up-scaled geometry image frame are determined. Also, the points included in the geometry image frame 550 are adjusted to take into account distortion that may be introduced during the down-scaling, video compression, video-de-compression, and up-scaling processes. Additionally, this distortion may be taken into account by a point cloud compression (PCC) attribute/texture mapper to adjust texture values for points that are distorted during the down-scaling, video-compression, video-de-compression, and up-scaling process. Additionally, attribute values may also be adjusted to take into account geometry distortion. In this way, the texture and attribute points are mapped to the same points in the same locations as the decoder will encounter when reconstructing and up-scaling the geometry plane. Then, the encoder can take into account the distortion of the geometry plane that may occur due to downscaling, encoding, decoding, and upscaling.

The texture points mapped to the points of the reconstructed up-scaled geometry plane 556 may be used to generate an adjusted attribute/texture image frame 558. The adjusted attribute/texture image frame 558 may then be down-scaled to generate a down-scaled adjusted attribute/

texture image frame **560**. The down-scaled adjusted attribute/texture image frame **560** may then be video encoded and transmitted as an attribute/texture bit stream.

FIG. **5G** illustrates an example closed loop rescaling with multiple attribute layers, according to some embodiments. In some embodiments, a similar process as described for FIG. **5F** may be followed. However, multiple degrees of down-sampling may be performed for one or more attribute image frames being down-scaled. For example texture/attribute image plane **558** may not be downscaled at all (e.g. compression rate target 0), or may be downscaled according to a plurality of compression rate targets (e.g. compression rate targets 1-4) to generate down-scaled versions of the adjusted attribute/texture image frame **562**. In such embodiments, a compression rate target may be dynamically adjusted, for example based on network conditions, processing capacity, etc.

FIG. **5H** illustrates an example of video level spatio-temporal scaling, according to some embodiments. In some embodiments, a similar process as described in FIGS. **5C** and **5D** may be performed using video level spatio-temporal downscaling and upscaling. For example, a frame rate (e.g. a number of frames generated per unit time) may be adjusted up or down in order to improve compression efficiency. In such embodiments spatial temporal adjustments may be made instead of resolution scaling and/or in addition to resolution scaling. For example, point clouds **564** at sequential movements in time may be down-scaled in resolution and/or frame rate in the 3D domain to generate down-scaled point clouds **566**. The point clouds **566** may be projected onto a patch plane and image frames **568** may be generated. Note image frames **568** may be geometry image frames or attribute image frames, or both. Additionally, video level spatio-temporal down-scaling may be applied to reduce a resolution of the image frames **568** and/or reduce a frame-rate of the image frames **568** to generate down-scaled image frames **570**. Note FIG. **5H** illustrates both frame-rate down-scaling (e.g. spatial temporal down-scaling) and resolution down-scaling. However, in some embodiments, spatio-temporal down-scaling may be performed without performing resolution down-scaling. The spatio-temporal and/or resolution down-scaled image frames **570** may then be video-encoded to generate bit stream **572**.

FIG. **5H** also illustrates an encoder receiving bit-stream **572**. A video-decoding component of the decoder may video-decode the bit stream **572** to generate down-scaled image frames **574**. The decoder may also perform video level-spatio temporal upscaling to interpolate between the down-scaled image frames to generate up-scaled image frame **576**. For example, down-scaled image frames, as an example includes two image frames per unit of time, whereas up-scaled image frames **576** includes a third image frame that has been generated in the 2D video domain by interpolating, and/or using other video-spatial intra frame compression techniques, such as motion vectors, etc. to generate the third image frame. The up-scaled image frames **576** may then be used to generate reconstructed point clouds **578**. Note that three reconstructed point clouds have been generated based on up-scaled image frames **576**. Optionally, the decoder may further upscale the reconstructed point clouds **578** either temporally or size-wise, or both, to generate up-scaled point clouds **580**.

FIG. **5I** illustrates an example closed loop rescaling with spatiotemporal scaling, according to some embodiments. For example, point clouds **582** may be used to generate original geometry image frames **584**. The original geometry frames may further be down-scaled temporally and/or size-

wise to generate down-scaled geometry image frames **586**, which may have a different frame rate, such as fg frames per second, as compared to fo frames per second for original geometry image frames **584**, where fg is less than fo. The down-scaled geometry image frames **586** may also have a smaller size than the original geometry image frames **584**, such as height and width “g” as compared to height and width “o” of original geometry image frames **584**. The down-scaled geometry image frames may further be video-encoded/compressed to generate geometry bit stream **596**. In a closed loop compression procedure, the geometry bit-stream **596** may further be video-decompressed/decoded at the encoder to generate reconstructed down-scaled geometry images **588**, which may have a similar frame rate and size as down-scaled geometry images **586**. The encoder may further apply a similar spatio-temporal and/or size-based up-scaling algorithm as would be executed at a decoder to generate up-scaled reconstructed image frames **590**, which may have a similar frame rate and size as original geometry image frames **584**. In some embodiments, the encoder may further adjust the original geometry image frames and repeat the process to reduce distortion. In some embodiments, geometry bit-stream **596** communicated out of the encoder may be based on adjusted geometry image frames **584**.

In some embodiments, a point cloud compression (PCC) texture/attribute mapper may further adjust attribute/texture values of attribute/texture image frames based on distortion introduced due to the down-scaling, video-encoding, video-decoding, and up-scaling of the geometry image frames. For example, adjusted attribute/texture image frames **592** may be generated. The adjusted attribute/texture image frames **592** may further be down-scaled at generate down-scaled adjusted attribute/texture image frames **594**, which may in turn be video-encoded to generate texture/attribute bit stream **598**. While not illustrated for spatio-temporal down-scaling, a similar process as described in FIG. **5G** may be performed, wherein a level of spatio-temporal down-scaling to be applied is determined based on an available bit rate to communicate the compressed point cloud.

As discussed above, methods of compressing point cloud video data may use conventional video codecs as well as auxiliary information that can help describe and reconstruct the point cloud information. The encoder and decoder diagrams of how that process is performed is shown in at least FIGS. **5A** and **5B**, respectively. Also, FIGS. **6A-6I** further illustrate how relationships between geometry image frames and attribute/texture image frames may be used to improve compression efficiency and reconstruction accuracy. Similar techniques may be combined with up-scaling and down-scaling techniques as described herein in FIGS. **5A-5L**.

As discussed above, the process of generating the image frames segments the point cloud into multiple 2D projected images/videos, each representing different types of information. This process is performed by segmenting the point cloud first into multiple patches that permit efficient projection of the entire 3D space data onto 2D planes. Each patch is associated with information such as geometry (also referred to herein as “spatial information”), texture, and other attributes if they are available. Such information is then copied at the co-located locations of the image frames on separate image sequences with each now containing only the geometry information, the texture information, and any other remaining attributes respectively. Auxiliary information that contains the patch information as well as an occupancy map that dictates which areas in these projected image sequences correspond to actual point cloud data and which are unoccupied, e.g. may contain no data or dummy

data, are also provided. Compression is then applied on such information using different strategies. Auxiliary information, for example, is entropy coded, while occupancy maps may be down-converted and encoded using either conventional codecs or other methods such as run length compression. The separate projected image sequences on the other hand are compressed using conventional codecs. This results in a collection of multiple sub streams, e.g. a geometry sub stream, texture and attribute sub streams, as well as occupancy and auxiliary information sub streams.

As described above, all sub streams except the occupancy map are expected to be of the same resolution. Each point in the geometry sub stream essentially corresponds to a point in the final 3D reconstructed point cloud. In some embodiments, it is permitted for the signal to be encoded at a different resolution than the original representation. Also, in some embodiments, offsetting as well as rotating the point cloud is also possible. Seeing things from the encoder perspective, this is done by signaling in the stream header additional metadata that would identify the scaling, offset, and rotation that should be applied onto the original point cloud data prior to projecting it onto the target video planes. From the decoder perspective, these parameters are used after the reconstruction of a first 3D point cloud representation and utilized to generate the final 3D point cloud representation. In such a scheme, both geometry and attribute/texture video data are signaled at the same resolution as specified in the point cloud header. Per patch metadata including scaling factors and rotation parameters are also supported in such a scheme, with scaling though now applied on each projected patch independently.

However, in some embodiments, this scheme may be further extended by providing additional resolution scaling flexibility in the encoded streams. In particular, not only may the scaling be applied in 3D space or per patch, but in some embodiments scheme scaling may be applied on the entire projected point cloud video data. This may permit use of "conventional" 2D resealing schemes and architectures, which are readily available in many architectures. Furthermore, unlike a scheme where geometry and attribute sub streams are encoded at the same resolution, this alternative scheme permits signaling of these sub streams at different resolutions. In some embodiments, this scheme could also be combined with the 3D scaling scheme described above, e.g. the specified 2D image frame scaling can follow in encoding order and precede in decoding order the 3D scaling process as described above. This can provide further flexibility in coding performance.

In particular, with the scheme described above the scaling factors, if any, that were applied to the point cloud signal in 3D space to change its resolution are known. Essentially the point cloud scene/object that is being represented would change from resolution $W_{3D} \times H_{3D} \times D_{3D}$ to $(s_x \times W_{3D}) \times (s_y \times H_{3D}) \times (s_z \times D_{3D})$. Then this resealed object would be projected using the patch approach specified above into a variety of sub videos, e.g. occupancy, geometry and attribute sub videos, each of a nominal resolution of $W_N \times H_N$. The nominal resolution may be currently specified in the group of frames header syntax of the MPEG PCC TMC2 draft (v1.2), using the syntax elements `frame_width` and `frame_height`. The scaling factors may be added into this syntax.

Group of frames header syntax

	Descriptor
5	<code>group_of_frames_header() {</code>
	<code> group_of_frames_size u(8)</code>
	<code> frame_width u(16)</code>
	<code> frame_height u(16)</code>
	<code> occupancy_resolution u(8)</code>
	<code> radius_to_smoothing u(8)</code>
10	<code> neighbor_count_smoothing u(8)</code>
	<code> radius_to_boundary_detection u(8)</code>
	<code> threshold_smoothing u(8)</code>
	<code>}</code>

With the proposed method one may also now rescale the geometry and attribute signal i further at a resolution of $W_G \times H_G$ and $W_{A(i)} \times H_{A(i)}$ respectively. There is no need of signaling the resolution of these videos in the point cloud compression PCC headers since that information already exists in the video bit streams themselves. Conventional algorithms can be used to rescale the videos from the nominal resolution of $W_N \times H_N$ to $W_G \times H_G$ or $W_{A(i)} \times H_{A(i)}$ and vice versa. These can be seen from the encoder perspective in FIG. 5C and from the decoder perspective in FIG. 5D.

FIG. 5E shows an open loop architecture for converting the geometry and attribute signals. In this architecture the geometry and attribute signals are created and converted independently. In some embodiments, the only dependency is that the geometry signal prior to downscaling and compression is used for generating the texture plane signal. However, in FIG. 5F, a closed loop architecture is considered. In this architecture, the geometry signal is scaled and coded first, then it is reconstructed and up converted to its original resolution. This new geometry signal is then used to generate the texture/attribute plane. Using this method, the texture (or attributes) generated more accurately correspond to the reconstructed geometry compared to the open loop architecture in the previous figure (FIG. 5E). It should be noted that the upscaling process of the geometry, if needed, should be matched across decoders to achieve the desired closed-loop performance. If the up-scalers do not match, there could be some difference in performance. The resolutions of the geometry and attribute signals also do not need to match in any of these systems. Conventional up-scalers, such as a separable filter based up-scaler, e.g. bicubic, lanczos, windowed cosine or sine, etc., or more complicated up-scalers, including bilateral up-scalers, edge adaptive, motion compensated, etc. could be used. Downscaling also could use similar methods, e.g. separable filters, edge preserving down-scalers, etc.

Such an approach could also be utilized by adaptive streaming solutions as well. In particular in adaptive streaming systems multiple streams may be generated at different resolutions and frame rates to better support the variability of a network. In this system apart from adjusting frame rates for the different layers, different resolutions can also be used between the texture and geometry to also augment such functionality. An example is shown in FIG. 5G where for a particular geometry signal multiple different bit streams are generated for the texture/attribute signal, each potentially having different frame rates and/or resolutions as well. An encoder may select to use the particular texture layer and then also select the appropriate corresponding sub-bit stream for the attribute/texture plane given the overall bandwidth characteristics of their network.

In a different aspect, downscaling and upscaling of the geometry plane and attribute/texture plane can be performed

by considering the characteristics of the point cloud system and how the images are constructed. In particular, in the systems described above, images are constructed using patches. These patches are available at both the encoder and decoder. A conventional system likely will not be able to consider the patches, however a more advanced system could utilize the patches to improve these two processes. In particular, better performance could be achieved by processing/filtering when upscaling or downscaling only the samples within a patch. Samples outside the patch are likely padded samples that may have no direct relationship with the samples inside the patch and if filtered/processed together could contaminate the signal. By isolating such samples this contamination can be avoided and performance can be improved. Even if not able to extract the full patch location information in a system, it could still be possible to consider other related information such as the occupancy map information. Occupancy maps, even though less accurate if they were down sampled, can still provide some improvement in performance. On the other hand, the interpolation process for the attribute signals may be augmented by also considering the characteristics, e.g. edges, in the geometry signal. In particular, edges may be extracted in the geometry signal, and they may be related to edges in the attribute signals and a guided interpolation based on that information may be performed. This is possible since edges in the two layers are highly related, especially at the boundaries of every patch. For example FIG. 6I further discusses an example guided interpolation technique.

In some embodiments, spatio-temporal rescaling may be applied as shown in FIGS. 5H and 5I. In some embodiments, on the encoder, frame dropping may be performed, while the decision to drop a frame may be based on how similar are the “neighboring” frames or the frame dropping could be done at a fixed rate (time stamps would still exist in the stream to inform the use of temporal relationships). In some embodiments “blending”/averaging of frames may be performed. That is, all frames may be scaled using different phase that is controlled “temporally”. For example, odd frames may be scaled vertically using phase 0, while even frames would be scaled vertically using phase 1. Those scaled images are called fields, which are then interleaved together to create interlace frames. This process could be used for projected point cloud data. Note that interleaving does not need to be restricted in vertical fashion, but could be done horizontally instead or as well.

For the temporal up conversion, frame repeats could be used, or more intelligent methods could be used that include motion adaptive and motion compensated strategies for the interpolation. Machine learning methods could also be used to assist with the interpolation.

In some embodiments, the temporal resolution may be only reduced in one of the planes (e.g. geometry or attribute/texture) while for the other all frames are retained (along with the occupancy and patch information). In such a case the “shape” and location of all patches in the reduced temporal resolution plane are known, but the exact value of its characteristics (depth for geometry, color for attribute/texture) is not known. However, that value may be computed by trying to locate each patch in their temporal adjacent neighbors. That can be done by searching using the shape information of the patch and/or the available values of the patch characteristics of the full resolution plane. That basically would involve a search (e.g. a motion estimation based search). When a patch is located, the information from the characteristics to interpolate from its temporal neighbors can be copied/blended and used as a predictor.

FIG. 5J illustrates a process of encoding/compressing image frames of a point cloud using down-scaling, according to some embodiments.

At 501, an encoder determines for a point cloud a plurality of patches projected on 2D planes. For example, any of the patch projection procedures described herein may be used.

At 503, the encoder generates attribute (or texture) patch images and geometry patch images for the patches. For example, any of the patch image generation techniques described herein may be used.

At 505, the encoder packs the generated attribute/texture patch images and the generated geometry patch images into one or more image frames. Note that the packing of the patch images and the generating of the patch images may occur concurrently. Any of the patch image packing techniques described herein may be used.

At 507, the encoder down-scales at least one of the image frames video down-scaling process. For example, the down-scaling process may include resolution reduction and/or frame rate reduction. Also, the down-scaling process may be open-loop or closed loop. Also the down-scaling process may be adjusted based on available bit rate, or not. Any of the down-scaling techniques described in regard to FIGS. 5A-5L may be used.

At 509, the encoder video encodes one or more bit streams based on the down-scaled image frames. Any video encoding technique as described herein may be used.

FIG. 5K illustrates a process of determining to down-scaling image frames using open-loop or closed-loop down-scaling, according to some embodiments.

At 511, the encoder receives a point cloud to compress. At 513, the encoder determines whether to apply down-scaling, and if so, whether to utilize an open-loop down-scaling process, as described herein, or a closed-loop down-scaling process, as described herein. For example, the encoder may determine whether to utilize an open-loop or closed-loop down-scaling process based on a complexity of the point cloud and an availability of processing capacity to perform the down-scaling. For example, a complicated point cloud with a larger number of patches may be down-scaled using an open-loop down-scaling process when processing capacity is limited, but may be down-scaled using a closed-loop down-scaling process when more processing capacity is available. For example, if the combination of complexity and available processing capacity are such that one or more thresholds are exceeded, the down-scaling may be performed at 515 via an open-loop process. Conversely, if the combination of complexity and available processing capacity are such that the one or more thresholds are not exceeded, the down-scaling may be performed at 517 via a closed-loop process.

FIG. 5L illustrates a process of decoding/decompressing image frames of a point cloud using up-scaling, according to some embodiments.

At 521, a decoder receives a bit stream comprising video encoded image frames of a point cloud. At least one of the video encoded image frames was video encoded from a down-scaled image frame, such as the down-scaled image frames discussed herein. The at least one down-scaled image(s) frame may have been down-scaled in size or frame-rate, or both

At 523, the decoder video de-codes the bit stream to generate one or more image frames comprising the at least one down-scaled image frame(s).

At **525**, the decoder determines which of the image frames have been down-scaled. For example any of the techniques described above may be used to identify down-scaled image frames.

At **527**, the decoder up-scales the at least one down-scaled image frame(s). The down-scaled image frame(s) may be up-scaled in size, frame-rate, or both. In some embodiments, any of the up-scaling techniques described herein may be used.

At **529**, the decoder generates a reconstructed version of the compressed point cloud based on the video decoded and/or up-scaled image frames. Any of the reconstruction procedures described herein may be used.

Pre Video Compression Pre-Processing and Post Video Decompression Filtering/Post-Processing

As discussed above, video point cloud data may be compressed using conventional video codecs. As discussed above, the process segments a point cloud frame into multiple 2D projected images/videos, each representing different types of information. This process is performed by segmenting the point cloud into multiple patches that permit one to efficiently project the 3D space data of the point cloud onto 2D planes. Each patch is associated with information such as geometry, texture, and/or other attributes. Such information is then copied at co-located locations on separate image frame sequences with each image frame containing the geometry information, the texture information, or any other remaining attributes, respectively as patch images packed into the image frame. Auxiliary information that contains the patch information as well as an occupancy map that dictates which areas in these projected image frame sequences correspond to actual point cloud data and which are unoccupied, e.g. may contain no or dummy data, are also provided. Compression is applied on such information using different strategies. Auxiliary information, for example, can be entropy coded, while occupancy maps may be down converted and encoded using either conventional video codecs or other methods such as run length compression. The separate projected image sequences on the other hand are compressed using conventional video codecs. This results in a collection of multiple sub streams, e.g. a geometry sub stream, texture and attribute sub streams, as well as occupancy and auxiliary information sub streams. All these streams are multiplexed together to generate the final point cloud bit stream as shown in FIG. 2A. In some embodiments, these streams may be provided to an image frame pre-processing/filtering element of an encoder to make intelligent pre-processing decisions and/or to intelligently adjust parameters used for video encoding based on information included in these streams.

FIG. 6A illustrates components of an encoder that further includes pre-video compression texture processing and/or filtering and pre-video decompression geometry processing/filtering, according to some embodiments. Encoder **600** includes image frame pre-processing/filtering element **602**. In some embodiments, any of the encoders described herein, such as encoder **200**, **250**, **400**, **450**, or encoder **500** may further include a texture processing/filtering element, such as texture processing/filter element **602**. In some embodiments, any of the decoders described herein, such as decoder **230** or decoder **280**, may further include a geometry processing/filtering element, such as geometry/filtering element **604**. In some embodiments, pre-processing of a packed image frame from image frame padding element **216** may be pre-processed via image frame pre-processing/filtering element **602** prior to being video encoded by video compression **218**. In some embodiments, image frame pre-process-

ing/filtering element **602** may receive occupancy map information, spatial information, for example from spatial image generation element **210**, texture information, for example from texture image generation element **212**, and attribute information, for example from attribute image generation element **214**.

In some embodiments, an image frame pre-processing/filter element, such as image frame pre-processing/filter element **602**, may determine relationship information indicating relationships between respective attribute patch images and/or depth patch images. Additionally, the relationship information may include relationships between the respective patches and an image frame, for example which portions of the image frame are occupied or unoccupied and where edges of the patches in the image frame are located.

In some embodiments, an encoder, such as encoder **600**, may utilize a received or determined occupancy map to determine the relationship information. In some embodiments, an image frame pre-processing/filtering element, such as image frame pre-processing/filtering element **602**, may cause one or more encoding parameters of a video encoding component, such as video compression **218**, to be adjusted based on the determined relationship information. For example, the image frame pre-processing/filtering element may cause the video encoding component to allocate more encoding resources to encoding portions of the image frame that are occupied with patches and may cause the video encoding component to allocate fewer encoding resources to encoding portions of the image frame that are unoccupied. Also, in some embodiments, the relationship information may indicate a plurality of patches (e.g. attribute patch images and/or depth patch images) that correspond to a same set of points projected onto a same patch plane. In some embodiments, an image frame pre-processing/filtering element may cause a video encoding component to utilize complementary encoding parameters when encoding patches corresponding to a same set of points in the point cloud.

In some embodiments, an image pre-processing/filtering element, such as image pre-processing/filtering element **602**, may determine a pooled distortion for a set of points. For example, the image frame pre-processing/filtering element may determine an amount of distortion introduced due to video compression of a depth patch image, an amount of distortion introduced due to video compression of an attribute patch image (such as colors), and an impact of overall or pooled distortion on the reconstructed point cloud based on the interactions of the distortions introduced for the separate patches. For example, depth distortion may cause additional color distortion in a reconstructed point cloud. In some embodiments, the image frame pre-processing/filtering element may adjust one or more video encoding parameters of a video encoding component, such as video compression **218**, in order to reduce the overall pooled distortion for the set of points.

In some embodiments, an image pre-processing/filtering element, such as image pre-processing/filtering element **602**, may determine edges of patches based on the relationship information. The image pre-processing/filtering element may further adjust one or more pre-processing processes such as color down-sampling of the one or more images, for example as described above in regard to closed-loop color conversion, based on the boundaries of the patches according to their determined edges.

FIG. 6B illustrates components of a decoder that further includes post video decompression texture processing and/or filtering and post video decompression geometry process-

ing/filtering, according to some embodiments. Decoder 610 includes texture processing/filtering element 612 and geometry processing/filter element 614. In some embodiments, any of the decoders described herein, such as decoder 230, 280, or 550 may further include a texture processing/filtering element, such as texture processing/filter element 612. In some embodiments, any of the decoders described herein, such as decoder 230, 280, or 550, may further include a geometry processing/filtering element, such as geometry/filtering element 614.

In some embodiments, relationship information for patch images in an image frame may be included in or derived from a bit stream for a compressed point cloud. For example, FIG. 6C illustrates a bit stream structure for a compressed point cloud, according to some embodiments. In some embodiments, the auxiliary information may include relationship information for patch images. Also, in some embodiments an occupancy map may include relationship information for patches of an image frame. For example, an occupancy map may indicate which portions of an image frame are occupied or unoccupied. Also, the auxiliary information may indicate which blocks of an image frame correspond to which patches. This information may be used to determine portions of an image frame that correspond to a same patch. Also depth information included in the geometry information (e.g. depth patch images) may be used to identify portions of image frames for points having a common depth in the point cloud. Additionally, attribute/texture information included in the texture video stream may be used to identify patches in the image frames with similar textures or attribute values. In some embodiments, an attribute texture processing/filtering element, such as attribute processing/filtering element 612, and a geometry processing/filtering element, such as geometry/filtering element 614, may utilize these data streams to determine or receive relationship information for patches included in one or more video encoded image frames, such as are included in the group of frames for the geometry video stream, and such as are included in the group of frames for the texture/attribute video stream. The attribute processing/filtering element and/or geometry processing/filtering element may further utilize these determined or received relationships to intelligently post-process geometry image frames in the geometry video stream or to post-process texture/attribute image frames in the texture video stream. Also, in some embodiments, geometry patches (e.g. depth patches) and texture or attribute patches may be packed into a same image frame. Thus, in some embodiments, a texture/attribute video stream and a geometry video stream may be combined into a common video stream comprising image frames with both geometry patches and attribute/texture patches.

One of the characteristics of this point cloud coding scheme is that the different projected image sequences can be not only compressed using “conventional” codecs but also processed with conventional processing algorithms reserved for 2D image/video data. That is, one could apply de-noising, scaling, enhancement, and/or other algorithms commonly used for processing 2D image data onto these image sequences. Such processing could have advantages, especially in terms of complexity and reuse of existing hardware implementations, versus performing such processing in the 3D domain.

One example of such processing is the conversion of the data from an RGB 4:4:4 representation to a 4:2:0 YCbCr representation and vice versa. In that scenario, for down conversion, the RGB data would be, for example, first converted to a YCbCr 4:4:4 representation, and then the

chroma planes could be filtered and downsampled to $\frac{1}{4}$ of their original resolution (half resolution horizontally and vertically). For the inverse process, the chroma planes would be up scaled to their original resolution, e.g. back to YCbCr 4:4:4, and then the signal would be converted back to RGB 4:4:4. A variety of down conversion and up conversion methods could be used, including the use of edge adaptive downscaling and upscaling, as well as techniques such as the luma adjustment method.

Although some conventional processing methods may operate “as is” on the point cloud projected image sequences, they do not fully consider the characteristics of such images and in particular the relationship that exists between different layers or the information about patches and occupancy. Consideration of such information could improve performance. For example, such methods may be improved by taking into account such characteristics and information, therefore improving performance and the final quality of the reconstructed 3D point cloud from the projected images.

In particular, conventional methods will most likely process the projected image sequences assuming that all samples inside these sequences are highly correlated and that adjoining samples likely correspond to the same or at least neighboring objects. Unfortunately, this may not be the case in such imagery. In fact, such image frames comprise samples that correspond to projected patches, as well as filler areas used to separate and distinguish these patches. Such filler areas may be left unfilled, e.g. with a default color value, or may have been filled using padding methods as described herein. Processing of individual samples, e.g. when using long filters for filtering, may result in contamination of information between different patches as well as the filler areas, which can impair quality.

Knowledge of the precise patch location and the filler areas can instead benefit performance substantially. In some embodiments, a processing engine (such as texture processing/filtering element 612 and/or geometry processing/filtering element 614) performs filtering/processing operations on such image data on a patch by patch basis. That is, a particular sample s is processed/filtered by accounting for samples that correspond to the same patch as s . Samples that may have been included in the processing using a conventional method, e.g. because of the consideration of a long filter, but which do not correspond to the same patch are excluded from the processing of s . This could be done, for example, by reducing the length of the processing filter until such samples are fully excluded, or by performing on the fly extrapolation of the data at the boundaries between patches, when processing, and using the extrapolated data in place of the available data outside a patch.

The same or similar principles could be applied when processing filler data, which can be seen as a patch on its own.

A particular system may consider the exact location and shape information for each individual patch, e.g. it may require that the patch information be fully decoded and therefore is fully available during processing. This can provide the most accurate processing and can avoid contamination across patches. In some embodiments, an approximation of the patch location and shape can be determined by looking only at the occupancy map information, which may have been coded using a conventional 2D video coding system. In this case, since occupancy information may have been subsampled, e.g. by 4 times horizontally and vertically (16 times overall), some of the samples at the boundaries of the patch may have been

1 duplicated. This may have some implications in performance, however, processing complexity can be considerably lower since there is no need to decode the full patch information.

Point cloud data are associated with geometry information as well as other attributes, e.g. texture, color, reflectance information, etc. Improved performance may be achieved by considering the relationships and characteristics across different attributes. In particular, similarity or dissimilarity of the geometry sample values in the projected plane may be accounted for when processing the corresponding samples in an attribute plane. In particular, neighboring projected samples that correspond to the same or similar depth in the geometry plane are expected to be highly correlated. However, neighboring samples that have very dissimilar depth information are less likely to be correlated. Therefore, when processing such samples, depth information could also be considered to determine how these samples should be considered.

In some embodiments, samples that are too far from a sample x in terms of depth distance, e.g. exceed a distance threshold T , may be excluded when processing sample x . Other samples may be weighted or prioritized in processing again based on their distance. Corresponding information from other attributes and how similar or dissimilar these attributes are could also be considered when processing the sample. Information, such as edges extracted from the geometry plane or from other attribute planes could also be considered when processing. In the particular example of chroma down sampling (e.g. 4:4:4 to 4:2:0), as discussed earlier, edge directed down sampling using the geometry as well as luma attribute information could be performed in the first case, while similarly for up sampling (e.g. 4:2:0 to 4:4:4) an edge directed up sampling process using geometry and luma attribute information could be performed. Such processing could again be patch/occupancy map based as described earlier, however such processing could also be performed on its own without such consideration. In another example, directed interpolation could be performed of the attribute planes from a resolution $H_o \times W_o$ to a new resolution $H_n \times W_n$, again using information from the geometry information and/or other attribute planes that may be available. For example, FIG. 6I illustrates an example application where an attribute plane is up scaled using its corresponding geometry information and the geometry extracted edges, according to some embodiments.

In some embodiments, other applications that utilize the proposed filter/processing techniques described above may include de-noising, de-banding, de-ringing, de-blocking, sharpening, edge enhancement, object extraction/segmentation, display mapping (e.g. for HDR applications), recoloring/tone mapping, among others. Such methods could also be utilized for quality evaluation, e.g. by pooling together and considering data (e.g. summing distortion values) in corresponding patches that also correspond to similar geometry information and other attributes when evaluating a particular distortion measurement. Processing may be purely spatial, e.g. only projected images that correspond to the same time stamp may be considered for such processing, however temporal/spatio-temporal processing may also be permitted, e.g. using motion compensated or motion adaptive processing strategies.

FIG. 6D illustrates a process for generating video encoded image frames for patches of a point cloud taking into account relationship information between the patches packed into the image frames, according to some embodiments.

At 620, an encoder generates attribute (or texture) image patches for sets of points projected onto patch planes. At 622, the encoder generates depth (e.g. geometry) patch images for sets of points projected onto the patch planes. At 624, the encoder packs the generated patch images into one or more image frames. In some embodiments, various ones of the patch generation and image frame packing techniques described herein may be used to generate the attribute patch images, the depth patch images, and to pack the generated patch images into one or more image frames.

At 626, the encoder generates one or more occupancy maps for the one or more packed image frames. In some embodiments, various ones of the techniques described herein to generate an occupancy map may be used.

At 628, the encoder optionally performs one or more pre-processing processes on the packed image frames taking into account the relationships between the attribute patch images and the depth patch images packed into the one or more image frames.

At 630, the encoder (or a separate video encoding component) video encodes the one or more image frames, wherein one or more parameters of the video encoding is adjusted based on the determined relationships between the patch images. For example, one or more parameters may be adjusted based on a pooled distortion for patches corresponding to the same set of points. Also, a color conversion process may be adjusted based on patch boundaries, as a few examples. In some embodiments, various other video encoding parameters may be intelligently adjusted based on knowledge not typically available to a video encoder regarding which portions of an image frame relate to patches, and which do not. For example, as explained above, default encoding algorithms for a video encoder may assume that pixels proximate to one another are highly correlated. However, in the case of packed image frames, points on either side of a patch image boundary may vary and not be correlated as would be expected in other video images. As explained above, in some embodiments, a video encoder may disregard padded portions of the image frame when selecting video encoding parameters. Also, other adjustments may be made in view of knowledge of the relationships between the patch images packed into the image frames.

FIG. 6E illustrates a process for generating video encoded image frames taking into account pooled distortion for a set of patches corresponding to a same set of points, according to some embodiments.

At 632, an encoder identifies attribute patch images and depth patch images corresponding to a same set of points projected on a same patch plane. For example, this information may be included in auxiliary information that is to be included in a compressed point cloud stream.

At 634, the encoder determines a pooled amount of distortion affecting the set of points based on distortion introduced via video encoding the attribute patch images and the depth patch images for the set of points. For example, the encoder may video encode and video decode one or more image frames to determine distortion introduced for the respective patches. Also, in some embodiments, a video encoding component may have known distortion levels associated with various settings and the encoder may be able to determine levels of distortion that will be introduced based on the selection of such settings. In some embodiments, distortion in one patch image may compound distortion in another patch image when reconstructing a representation of the point cloud. For example, depth distortion combined with attribute or texture distortion may compound

upon each other, where points are both moved in a reconstructed point cloud and also have changed colors. In some embodiments, these effects may be greater than the sum of the individual distortions. For example, depth distortion may further highlight color distortion and vice-versa.

At **636**, the encoder adjusts one or more video encoding parameters and/or one or more pre-processing parameters to reduce the overall amount of pooled distortion for the set of points.

FIG. 6F illustrates a process for generating video encoded image frames taking into account patch edges, according to some embodiments.

At **638**, an encoder determines edges for attribute patch images or depth patch images packed into one or more image frames. For example, such information may be included in an occupancy map for the image frame, and/or in auxiliary information for the image frame.

At **640**, the encoder provides the relationship information to a video encoding component that will video encode the image frame.

At **642**, the video encoding component adjusts one or more video encoding parameters and/or an image pre-processing/filtering element adjusts one or more parameters based on the determined edges. For example, a pre-processing filter may be adjusted to avoid cross patch contamination or contamination between patch portions and pad portions of the image frame.

FIG. 6G illustrates a process for reconstructing a point cloud based on video encoded image frames comprising patches of the point cloud, wherein relationship information between the patches packed into the image frames is taken into account, according to some embodiments.

At **650**, a decoder receives one or more image frames comprising attribute patch images and depth patch images packed into one or more image frames. For example, the one or more image frames may have been packed with patch images as described herein.

At **652**, the decoder receives an occupancy map and/or auxiliary information for the one or more image frames. For example, the one or more image frames, occupancy map, and/or auxiliary information may be included in a group of frames and/or point cloud data stream as shown in FIG. 6C.

At **654**, the decoder receives (e.g. via the occupancy map or auxiliary information) or determines (e.g. based on the occupancy map and/or auxiliary information) one or more relationships for the attribute patch images and/or depth patch images packed in the one or more image frames. In some embodiments, the one or more determined relationships may indicate portions of the image frame that are occupied and unoccupied. In some embodiments, the one or more determined relationships may indicate a depth patch image and one or more attribute or texture patch images that correspond to a same set of points projected on a same patch plane as the depth patch image. In some embodiments, the one or more determined relationships may indicate regions of a set of one or more attribute patch images and one or more depth patch images which correspond to a same depth of the point cloud. In some embodiments, the one or more determined relationships may indicate regions of a set of one or more attribute patch images which share similar attribute values. In some embodiments, the one or more determined relationships may indicate regions of a set of one or more attribute patch images and one or more depth patch images which correspond to patches comprising points of the point cloud with depth gradients that deviate from one another by less than a threshold amount. In some embodiments, other

relationships may be determined based on the occupancy map and/or auxiliary information.

At **656**, the decoder video decodes the one or more packed image frames. In some embodiments, the decoder adjusts decoding parameters based on the determined relationships.

At **658**, the decoder performs one or more post-processing processes on the video decoded image frames before using the depth patch images and attribute or texture patch images to reconstruct a reconstructed representation of the point cloud. As explained above, it may be more efficient to perform post-processing on a 2-D representation via a video post-processing process than to perform post-processing in a 3-D space comprising points of a point cloud reconstructed from the patch images included in the 2-D image frames. The decoder may adjust one or more parameters of the post-processing based on the determined relationships. For example, a long filter may be adjusted to avoid cross-contamination across patch images, between patches and padding, or between patch images. In some embodiments, various post-processing processes may be adjusted, such as: denoising, debanding, deringing, deblocking, sharpening, object extraction or segmentation, display mapping, wherein a range of the one or more image frames is mapped to a range to be used to display a reconstructed representation of the point cloud, a color space conversion of the one or more image frames, a filtering process of the one or more image frames, color or tone adjustment processes applied to the one or more image frames, etc.

At **660**, the decoder generates a reconstructed representation of the compressed point cloud based on attribute information and depth information extracted from the patch images in the one or more image frames that have undergone the post-processing processes at **658**.

FIG. 6H illustrates a process of upscaling a patch image included in an image frame taking into account edges of the patch image determined based on received or determined relationship information for the patches, according to some embodiments.

At **670**, a decoder determines edges of an attribute patch image or a depth patch image in an image frame based on the determined relationships (e.g. determined from an occupancy map and/or auxiliary information). For example, FIG. 6I illustrates an edge extraction process being applied to a geometry plane **680** to determine edges **682** of patch images in the geometry image frame.

At **670**, an attribute patch image or depth patch image is up-scaled. For example, any of the up-scaling and down-scaling processes described herein may take advantage of determined relationships as described for an encoder or decoder in regards to FIGS. 6A-6I. For example, FIG. 6I also illustrates an attribute patch image **684** being up-scaled.

At **674**, edges of the up-scaled attribute or depth patch image are updated based on an interpolation between points of the extracted edges determined at **670**. For example, FIG. 6I illustrates a directed interpolation step included in the processing of attribute plane **684** to result in up-scaled attribute plane **686**.

Bit Stream Structure for Compressed Point Cloud Data

As discussed above and in more detail in regard to FIGS. **13** and **14**, there is considerable interest in augmented and virtual reality applications and in the use and compression of 3 dimensional data to support them. One such form of data includes point cloud representations, where objects are specified as a series of points that are described in terms of 3D geometry and a set of attributes per point that may include information such as color, reflectance, time, or other types of information. Compression of such information is

highly desirable given the amount of space and bandwidth such data would require if not compressed.

One method that has been proposed for compressing point cloud data is described above in regard to packing patches into video frames and may be further extended to achieve near lossless or lossless performance by further encoding points that are “missed” or not included in the patches.

The encoder and decoder diagrams as shown in FIGS. 5A and 5B show how that process is performed. In some embodiments, the point cloud data is first segmented into multiple 2D projected images/videos, each representing different types of information. Segmentation is performed by dividing the point cloud into multiple patches that permit one to efficiently project the entire 3D space data onto 2D planes. Each patch is associated with information such as geometry, texture, and other attributes if they are available. Such information is then copied at the co-located locations on separate image sequences with each now containing only the geometry information, the texture information, and any other remaining attributes respectively. Auxiliary information that contains the patch information as well as an occupancy map that dictates which areas in these projected image sequences correspond to actual point cloud data and which are unoccupied, e.g. may contain no or dummy data (e.g. padded data), are also provided. Compression is then applied on such information using different strategies. Auxiliary information, for example, can be entropy coded, while occupancy maps may be down-converted and encoded using either conventional image/video codecs or other methods such as run length compression. The separate projected image sequences may be compressed using conventional codecs. This results in a collection of multiple sub streams, e.g. a geometry sub stream, texture and attribute sub streams, as well as occupancy and auxiliary information sub streams. All these streams are multiplexed together to generate the final point cloud bit stream as shown in the bit stream structure illustrated in FIG. 6C.

Point Cloud Compression Network Abstraction Layer Units (PCCNAL Units)

In some embodiments, the structure specified in FIG. 6C may be quite rigid and inflexible, and may not account for certain applications, especially low delay applications, that would require all information corresponding to a single point cloud frame in time to be efficiently signaled and decoded within a constrained time frame. The bit stream architecture illustrated in FIG. 6C may also impose considerable penalties in terms of memory and delay. In some embodiments, a point cloud video sequence is segmented into multiple groups of point cloud frames (GOFs). Group of Frames or GOFs may consist of multiple layers of information, with each one representing different types of data, such as geometry and texture information among others. In some embodiments, a point cloud compression PCC decoder is required to first decode and store the entire geometry video stream for each GOF, as well as any associated information with it, followed by the related texture video stream before starting to reconstruct each frame within a GOF (for example point cloud reconstruction can follow the decoding order of the texture video stream). However, the memory requirements may be reduced by scanning the bit stream and finding the appropriate location of each sub-bit stream (e.g. geometry, occupancy/auxiliary info, texture) and decoding them in parallel. Also, geometry, texture, occupancy/auxiliary information for a first patch or set of patches may be decoded and used to reconstruct the first patch or set of patches without requiring all of the geometry, texture and occupancy/auxiliary information for the full bit stream to be

decoded and stored in memory. However, this assumes that such streams are restricted in using the same coding order and structures.

When all the data is sequentially signaled without any markers to indicate the positions of different sub streams, there may be a significant disadvantage of time delay. For example, one frame cannot be reconstructed until all the group of frame GOF information is decoded. Also, the bit stream cannot be decoded in parallel unless every data has information of its own size. To resolve this issue, in some embodiments the concept of a coding unit, which may be referred to herein as a PCCNAL (Point Cloud Compression Network Abstraction Layer) unit for convenience, that contains information on one or more types of data and its related header information may be used. Encapsulated data comprising a PCCNAL unit can be placed in any location within a bit stream and can be decoded and reconstructed in parallel.

In some embodiments, a PCCNAL unit may further include one or more encoding parameters that instruct a decoder as to how to decode video encoded image frames comprising image frames packed with patch images. In some embodiments, signaling methods of the encoding parameters may not be defined or limited. The names of the encoding parameters may not be limited as long as the parameters serve the same purpose at the encoder and the decoder. The actual value or code words of each parameter may not be limited as long as each function of the parameter is identified by the numbers used as code words for the encoding parameter, wherein the encoder and the decoder use the same numbers for the same code words for the same parameters.

For example, bit stream structures for compressed point cloud data that may be more flexible than the structure illustrated in FIG. 6C, are shown in FIGS. 7A, 7B, and 7C. Such bit stream structures may permit the delivery of point cloud data for low delay applications or random access applications. The bit stream structures, as shown in FIGS. 7A, 7B, and 7C, may also enable other features such as unequal error protection, reordering, and reduced memory footprint, among others. Furthermore, parameters and/or component units which are used to identify different encoding/decoding methods and definitions may be applied over a slice of a frame, a frame, a group of frames (GOF), or a sequence of groups of frames. An example of a component unit that may be defined and used within a point cloud compression PCC bit stream is the Point Cloud Compression Network Abstraction Layer (PCCNAL) unit. In particular, a PCCNAL unit may be defined as a set of data that contains one or more types of information and that can be placed anywhere in the bit stream. However, placement may be limited within a particular period.

FIG. 7D illustrates a process of encoding a bit stream that includes PCCNAL units, according to some embodiments.

At 702, an encoder, such as any of the encoders described herein, determines a plurality of patch images for a point cloud being compressed. At 704, the encoder packs the patch images into one or more image frames.

At 706, the encoder determines one or more relationships and/or encoding parameters for the patch images packed into the one or more image frames.

For example, the encoder may determine a relationship between a first patch image in a first image frame and a second patch image in a second image frame. In some embodiments, the determined relationship may be that the first patch image and the second patch image are patch images that correspond to the same set of points of the point

cloud projected on the same patch plane. For example, the first patch image may be a geometry patch image for the set of points projected onto the patch plane and the second patch image may be an attribute or texture patch image for the same set of points projected onto the same patch plane. Also, in some embodiments, the relationship may be that the first patch image and the second patch image are a same or similar type of patch image, such a geometry patch images, attribute patch images, texture patch images, etc.

At 708, the encoder encodes a bit stream comprising video encoded versions of the image frames packed with patch images and one or more PCCNAL units that indicate the determined relationships, encoding parameters, and/or locations of respective patch images in the bit stream.

For example, in some embodiments the first patch image and the second patch image may both be geometry patch images or may both be attribute or texture patch images. For example, FIG. 7C illustrates groups of frames that are organized based on patch image type (e.g. occupancy, geometry, or attribute/texture).

In some embodiments, a bit stream as shown in FIG. 7A may include a PCCNAL unit 720 that indicates an encoding/decoding parameter for a sequence of image frames and/or PCCNAL units. Additionally, the bit stream may include a PCCNAL unit that indicates a group of frames in the bit stream. For example, PCCNAL unit 722 may indicate that the group of frames includes a set of occupancy frames 724, a set of geometry frames 726, and a set of attribute or texture frames 728. In some embodiments, the bit stream may further include additional PCCNAL units that indicate patch images included in a group of occupancy frames, geometry frames, or attribute frames, etc. In some embodiments, a group of frames PCCNAL unit, such as PCCNAL unit 722, may indicate one or more encoding parameters that supplement or override encoding parameters included in a sequence parameter set PCCNAL unit, such as PCCNAL unit 720. For example, PCCNAL unit 720 may define global encoding parameters for a sequence of image frames and other PCCNAL units may define regional or local encoding parameters that apply to a particular group of image frames in the sequence, a particular image frame in a group of frames, or a particular slice (e.g. patch image) in a particular image frame.

In some embodiments, different types of image frames may be interleaved in a group of frames as shown in FIG. 7B. For example, sequence parameter set PCCNAL unit 730 may define encoding parameters for a set of frames. Additionally, group of frames (GOF) PCCNAL unit 732 may define a picture order count for frames in a group of frames corresponding to the GOF PCCNAL unit 732, wherein the picture order count repeats for the frames of the group of frames. For example, as shown in FIG. 7B, a first image frame may be an occupancy image frame, a next image frame may be a geometry image frame, and a subsequent image frame may be an attribute or texture image frame. In some embodiments, the occupancy image frame, the geometry image frame, and the attribute/texture image frame may include related patch images that correspond to the same set or sets of points projected on to the same patch plane or planes. Such an organization of image frames in a bit stream may reduce delay times because a decoder can quickly find all related patch images for a given set of points and start reconstructing a point cloud representation of the set of points before fully video decoding the whole bit stream.

In some embodiments, common types of image frames (e.g. occupancy, geometry, attribute/texture, etc.) may be grouped together in a bit stream as shown in FIG. 7C. In

some embodiments, this may provide better compression efficiency and/or reduce computational load at a decoder, but may require more memory and incur greater delay times than interleaving as shown in FIG. 7B.

In some embodiments, PCCNAL units may be inserted in a bit stream to indicate hierarchical relationships. For example a first PCCNAL unit may indicate a relationship between patch images of image frames in a first set of image frames and a second PCCNAL unit may indicate a relationship between patch images of image frames in a second set of image frames. Additionally, a third PCCNAL unit may indicate a relationship between the first set and the second set of image frames. For example, the third PCCNAL unit may indicate that the patch images of the first set and second set of image frames belong to a region of the point cloud, and the first PCCNAL unit may indicate that the patch images in the first set of image frames belong to a sub-region within the region, while the second PCCNAL unit may indicate that the patch images of the second set of image frames belong to a different sub-region of the same region of the point cloud. Additionally, PCCNAL units may be inserted into a bit stream to encode hierarchical encoding parameters.

In some embodiments, encoding parameters that may be signaled via one or more PCCNAL units may include a frame height or a frame width, wherein an image frame packed with patch images occupies less than a full area of a video frame, and wherein the frame height or the frame width indicates the height or width of the image frame packed with patch images in the larger video frame.

In some embodiments, a parameter signaled via one or more PCCNAL units may indicate a re-sizing parameter to be used to re-size an image frame packed into a video frame in order to use the image frame to reconstruct a point cloud. For example, the image frame may have been down-scaled at an encoder and may need to be up-scaled at a decoder prior to being used to reconstruct the compressed point cloud.

In some embodiments, a PCCNAL unit may indicate a number of image frames for which one or more particular encoding parameters are to be applied. Additionally, a color encoding format for a set of image frames may be signaled via one or more PCCNAL units.

In some embodiments, PCCNAL units may be used to signal bit-depths. For example, a PCCNAL unit may signal a bit depth of encoded image frames and a bit-depth to be used in a reconstructed point cloud. Additionally, a PCCNAL unit may indicate an intermediate bit depth to be used in intermediate calculations subsequent to video decoding the one or more image frames but prior to presenting a final reconstructed version of the point cloud.

In some embodiments, different groups of frames or sequences may be encoded using different video encoding standards or formats and a PCCNAL unit may indicate a particular video decoding standard or format to be used to video decode a set of image frames associated with the PCCNAL unit.

In some embodiments, a point cloud may include layered points, for example different points at a same X and Y location but at different Z depths, as an example. In some embodiments, a PCCNAL unit may indicate the presence of layered points and/or a number of layers included for a set of points represented in a group of image frames.

FIG. 7E illustrates a process of decoding a bit stream that includes PCCNAL units, according to some embodiments.

At 712, a decoder receives an encoded bit stream comprising video encoded image frames comprising patch

61

images packed into respective ones of the video encoded image frames and comprising one or more point cloud compression network abstraction layer (PCCNAL) units in the bit stream. For example, the encoder may receive any of the bit streams illustrated in FIG. 7A, 7B, or 7C, which may include packed image frames that have been generated by any of the encoders as described herein.

At 714, the decoder identifies PCCNAL units in the bit stream. In some embodiments, the PCCNAL units may be encoded with a pre-amble bit sequence that is known by the decoder, such that the decoder may scan the bit stream for the known pre-amble bit sequence to identify the PCCNAL units. Also in some embodiments, PCCNAL units may include data indicating a size of the PCCNAL unit and/or trailing bits that indicate an end of the PCCNAL unit. These also may enable a decoder to quickly locate PCCNAL units in a bit stream.

At 716, the decoder determines an order in which to decode the video encoded image frame based on relationship information included in the PCCNAL units. For example, the decoder may be requested to render a particular region of a point cloud and may quickly identify image frames comprising patch information for the requested region based on information included in PCCNAL units. For example, in the bit stream illustrated in FIG. 7B, group of frames (GOF) PCCNAL unit 732 may include information indicating a region of the point cloud with which the group of frames is related. In such embodiments, instead of reconstructing the whole point cloud and then identifying the region of interest, the decoder may utilize information included in PCCNAL units to locate a sub-set of the image frames of the bit stream to decode that are needed to reconstruct the requested region of interest. In a similar manner, when a bit stream includes multiple representations of a point cloud at multiple points in time, a decoder may locate image frames for a particular point in time using PCCNAL units.

At 718, the decoder then decodes the image frames in the determined order and reconstructs the point cloud or portion thereof using any of the decoding techniques described herein.

In some embodiments, properties of the PCCNAL unit may include:

PCCNAL header: sequence of bits that indicates the start of the unit and/or the type of the unit. Such a header may contain a "start code" indicator that is a unique sequence that should not be present anywhere else within the PCCNAL, and can help in identifying such a unit. Start code emulation prevention methods could be used to avoid the presence of equivalent signatures within the stream.

PCCNAL index: index to identify different PCCNAL units

PCCNAL size: size of the PCCNAL unit

PCCNAL trailing bits: Such information is optional, and similar to the start code, this is a unique signature that can help in identifying the end of a PCCNAL unit

PCCNAL GoF index: Corresponding GoF index to the PCCNAL units

PCCNAL POC: An indexing parameter for such a unit.

This index can be used to classify and/or identify each PCCNAL unit and permit grouping of different PCCNAL units based on its value. For example, a geometry and an attribute frame that correspond to the same point cloud frame can be given the same index, which helps identify their relationship later during decoding and reconstruction of the point cloud repre-

62

sentation. This information may limit placement of PCCNAL units within the bit stream.

Each coded block or set of coded blocks may be identified as a PCCNAL unit. Such blocks can include sequence parameter sets, picture parameter sets, geometry video data, occupancy data, texture video data, geometry frame, occupancy frame and texture frame amongst others. For example, Geometry video stream in FIG. 7A can correspond to geometry video data PCCNAL(PCCNAL-GEO), auxiliary info & occupancy maps can correspond to PCCNAL-OCC and Texture video stream can correspond to PCCNAL-ATT. In an alternative embodiment, all of the geometry video data, occupancy data and texture video data can comprise one PCCNAL unit.

Examples of PCCNAL unit are as following:

PCCNAL-SPS: set of parameters used and can be applied over the entire sequence

PCCNAL-PPS: set of parameters used and can be applied over the entire frame/picture

PCCNAL-GOF: set of parameters used and can be applied over the entire GOF

PCCNAL-OCC: set of occupancy map information

PCCNAL-GEO: set of geometry data information

PCCNAL-ATT: set of texture data information

PCCNAL-FRM: information on single frame

PCCNAL-GEOFRM: geometry information on single frame

PCCNAL-ATTFRM: texture information on single frame

PCCNAL-OCCFRM: occupancy map information on single frame

The above information could also be defined for sub-frames, e.g. slices, group of coding tree units (CTUs) or macroblocks, tiles, or groups of slices or tiles. The above information can also be specified for a group of frames that do not necessarily need to be equal to the number of frames in a GOF. Such group of frames may be smaller or even larger than a GOF. In the case that this is smaller, it is expected that all frames inside this group would be a subset of a GOF. If larger, it is expected that the number would include several complete GOFs, which might not necessarily be of equal length.

In some embodiments, the PCCNAL units may be signaled sequentially without any marker.

In some embodiments, PCCNAL units can have a PCCNAL header, which may include a start code and/or contain PCCNAL trailing bits. The PCCNAL header is located at the beginning of a PCCNAL unit and the PCCNAL trailing bits are located the end of a PCCNAL unit. By having a PCCNAL header and/or a PCCNAL trailing bits, the decoder can jump to the point where the proper data is located without decoding from the beginning of the data to the end of the data.

For example, in the PCCNAL header a start code can be included, which can help in detecting a PCCNAL unit. A start code is a unique sequence of bits that should not be used for representing any other data within such a unit. When such start code is detected, it may be known that the following bits would correspond to particular information relating to such a unit, including its identification information as well as any related payload that would correspond to such an identifier. For example, an identifier equal to 000000, assuming 6 bits for the identifier, can indicate the PCCNAL is GoF Header Unit, while an identifier equal to 000001 can indicate that the payload includes Geometry Data information. Other identifiers could correspond to occupancy information, attributes, and so on and such could be defined by the application or user (e.g. engineer config-

uring the encoder/decoder). It should be pointed out that although start codes are present at the beginning of a particular unit, it might be possible to also define a “start code” that follows a fixed number of bits or syntax elements, which may be referred to herein as a “preamble” sequence. For example, the preamble sequence may include the unit identifier as well as the PCCNAL POC (picture order count) parameter. If the parameters in the preamble sequence use variable arithmetic encoding, they may be encoded in right to left order in the bit stream (e.g. the least significant bit of the encoded parameter is written first in the stream and the most significant one is written last). This is not necessary, but could still be used for fixed length parameters.

In some embodiments, a PCCNAL header can contain the size of its own PCCNAL size instead of PCCNAL header bits.

In some embodiments, a PCCNAL header can have both PCCNAL size and PCCNAL header bits.

In some embodiments, a PCCNAL can have trailing bits to indicate the end of the PCCNAL unit.

In some embodiments, a PCCNAL can have its corresponding GoF index.

In some embodiments, a PCCNAL can have its corresponding POC index

In some embodiments, a PCCNAL can have its corresponding a type identifier.

In some embodiments, with the PCCNAL header, PCCNAL units in a bit stream can be located without any fixed order. For example, in some embodiments PCCNAL units can be placed in any order within a bit stream, within the limitations of the PCCNAL picture order count (POC). Reordering could still be performed during decoding or reconstruction using the value of PCCNAL POC. PCCNAL POC could be a periodic number however, and such reordering may account for such a characteristic. In some embodiments, PCCNAL units can be grouped by their GoF index. In some embodiments, PCCNAL units can be grouped by their POC as depicted in FIG. 7B. In some embodiments, PCCNAL units can be grouped by their types as depicted in FIG. 7C.

In some embodiments, PCCNAL units can be signaled in different bit streams. Even when signaled separately PCCNAL units can be reconstructed properly based on PCCNAL header information such as GoF index and/or POC.

For example, when an encoded PCC bit stream is received at the decoder, the decoder may start parsing PCCNAL unit headers. Using information in the headers, a decoder can jump through the bit stream to collect synced occupancy-geometry-texture data. If a header has the size of the PCCNAL unit, it may jump to the end by the size. If a header only contains a start code, it may read through the bit stream until it encounters a new header or a trailing bits sequence. The decoder can also analyze the PCCNAL POC information for each PCCNAL, determine which units contain the same information and then group and reorder them. Such process can permit the compressed point cloud data to be properly decoded and then reconstructed, e.g. by determining which frame in the geometry and attribute video signals correspond to the same set of points of the point cloud and could be used for its reconstruction.

In an encoded PCC bit stream, there can be several parameters per slice, per frame/picture, per GOP, or per sequence of point cloud data, which signal information that permits proper decoding and rendering of the point cloud data. The parameters can be present in a bit stream more than once and at different locations. For example, a parameter can

be signaled at both the sequence level and at the slice level. In this case, the parameter at the lower level can overwrite (e.g. override) the one at the higher level within the level's scope. In another embodiment, the parameter at the lower level can provide additional information that can clarify (e.g. supplement) the characteristics of the parameter at the higher level. Set of these parameters can be included in a PCCNAL unit. Some example of such parameters include the following:

PCC frame width, PCC frame height: the “nominal” width and the height of the frame that the PCC data is mapped. The size can be the same as the size of the output of the video codec. The size can be different from the size of the output of the video codec. In this case the outputs can be resized by a method indicated in the parameter sets or predefined by the user/codec.

Resizing type: type of resizing method from decoded video size to PCC frame size

Group of Frames size: the number of frames in one group of frames can be signaled.

Chroma format: Chroma format of the geometry data video and texture data video can be signaled. If necessary, Chroma format of occupancy map can be signaled as well. The format can be signaled once for both video layers or can be signaled separately for each video layer. Such information could also be inferred from the video bit stream and does not necessarily need to be present in the PCCNAL unit again.

Input, output bit depth: This syntax defines the bit depth of input PCC data and output PCC data are signaled.

Internal bit depth: This syntax element defines the bit depth of the data for internal computation in the PCC. During the internal computation, the input data is adjusted to be within the range of internal bit depth. Such information could also be inferred from the video bit stream and does not necessarily need to be present in the PCCNAL unit again.

Type of the video codec: This syntax element defines the video codec, e.g. AVC, HEVC, AV1 or some other codec, as well as the corresponding profile and level information, that is used for encoding the Geometry and Attribute projected video data. Such a syntax element can be signaled once for both the Geometry and Attribute video signals, or can be signaled independently for each video signal. Such information could also be omitted and be inferred by the characteristics of and information within the video stream.

Layers for each stream

Presence of layers: a flag that indicates that there are more than 1 layers for the geometry data/attribute data/occupancy map in the bit stream

Number of layers: in the case that the layers are more than 1, the number of layers is also signaled. This syntax element defines the number of layers that the Geometry and Attributes data video have. Each layer contains information about the points mapped to a same pixel in a patch but each one corresponds to a different depths.

Minimum number of layers: This is an optional syntax element that defines the minimum number of layers present in the bit streams

Each video layer can use a different type of a video codec. The type of the video codec used for a particular layers can be signaled.

Occupancy map

Presence of an occupancy map: a flag that indicates the presence of occupancy map in the bit stream

65

Coding type of the occupancy map: in case that occupancy map is present, the type of the coding method used for the occupancy map is also signaled. For example, the occupancy map can be coded with a video codec or another method defined in the specification.

Example Methods of Compressing and Decompressing Point Clouds

FIG. 8A illustrates a process for compressing attribute and spatial information of a point cloud, according to some embodiments.

At **802**, a point cloud is received by an encoder. The point cloud may be captured, for example by one or more sensors, or may be generated, for example in software.

At **804**, compressed point cloud information is determined, using any of the techniques described herein or using one more combinations of the techniques described herein.

At **806**, a compressed point cloud is encoded using the compressed point cloud information determined at **804**. The point cloud may be compressed using any of the techniques described herein.

FIG. 8B illustrates a process for decompressing attribute and spatial information of a point cloud, according to some embodiments.

At **803** an encoded point cloud is received. The point cloud may have been encoded using any of the encoding techniques described herein, such as patch images packed into an image frame that is then encoded by a video encoder. In some embodiments, the encoded point cloud may comprise point cloud projections, such as projections onto a cube, cylinder, sphere, etc. that are then encoded via a video encoder.

At **805**, spatial and attribute information for the encoded point cloud is determined. For example, a video decoder may be used to decode video encoded packed images or projects. Spatial information may then be determined based on the packed images or projections and combined to determine spatial information for points of the point cloud. For example, depth information for points of a patch may be matched with X and Y information for the points of the patch to determine spatial information for the points of the patch in 3D space. In a similar manner other attributes, included in patch images such as color attributes, texture attributes, etc. may be matched with corresponding points to determine attribute values for the points. Also, in the case of multiple projections, the same point may be identified in more than one of the projections to determine spatial information for the point in 3D space.

At **807**, a decompressed point cloud may be provided to a recipient device or module.

FIG. 8C illustrates patch images being generated and packed into an image frame to compress attribute and spatial information of a point cloud, according to some embodiments.

At **810**, patches are determined for portions of point cloud. For example patches may be determined as described above. At **825** patch information for the patches may be generated and at **826**, may be encoded to be sent to a decoder. In some embodiments, encoded patch information may be separately encoded from one or more image frames comprising packed patch images.

At **811**, a first patch (or next patch is selected). At **812** a color (e.g. attribute) patch image is generated for the points of the point cloud included in the patch. At **814**, one or more additional attribute images, such as a texture attribute image, are generated for the points of the point cloud included in the patch.

66

At **813**, spatial information images are generated for the points of the point cloud included in the patch. In some embodiments, to generate the spatial information images, the points of the point cloud are projected, at **815**, onto a patch plane perpendicular to a normal vector normal to a surface of the point cloud at the patch location. At **817** a first spatial image is generated for the patch based on the points being projected on the patch plane at **815**. In addition, depth information for the points of the patch relative to the patch plane is determined at **816**, and at **818** a depth patch image is generated based on the depth information determined at **816**.

At **819**, it is determined whether there are additional patches for which patch images are to be generated. If so, the process reverts to **811** for the next patch. If not, at **820** the patch images for the patches are packed into one or more image frames. In some embodiments, patch images for respective patches may be packed before patch images are determined for other patches. At **821**, an occupancy map is generated based on where the patch images were placed when being packed into the one or more image frames. At **824**, the occupancy map is encoded. As discussed above, in some embodiments, the occupancy map may be encoded using an arithmetic encoder, entropy encoder etc. Also, in some embodiments, the occupancy map may be encoded using a video encoder, wherein the occupancy map is organized as an additional image frame that corresponds with a patch image frame and that represents portions of the patch image frame that are occupied with patch images (e.g. occupied pixels) and portions of the patch image frame that are padded (e.g. non-occupied pixels). Video encoding of an occupancy map is discussed in more detail below in FIGS. 12A-12C.

At **822**, spaces in the one or more image frames that are not occupied by patch images are padded. In some embodiments, an occupancy map for a patch image frame may be generated before or after the patch image frame is padded at **822**.

At **823**, the one or more image frames are video encoded, such as in accordance with a high efficiency video coding (HEVC) standard. In some embodiments, in which an occupancy map is represented by an occupancy map video image frame, the occupancy map video image frame may be video encoded at **823**.

FIG. 9 illustrates patch images being generated and packed into an image frame to compress attribute and spatial information of a moving or changing point cloud, according to some embodiments.

At **930**, point cloud information for a previously encoded point cloud is received wherein the point cloud information represents a subsequent version of the previously encoded point cloud. For example, the subsequent version may be a representation of the point cloud at a subsequent moment in time, wherein the point cloud is moving or changing as time progresses.

At **931**, it is determined if any new patches need to be determined for the point cloud. For example, an object not currently in the previously encoded point cloud may have been added to the point cloud. For example, the point cloud may be a point cloud of a road and a ball may have entered into the road. If there is a need to add a new patch, the occupancy map is updated at **933** to include the new patch and encoded at **934**. Also, at **932** patch images are generated for the new patch in similar manner as described in **812-814**. The generated patch images are included in packing at **943**.

At **935**, a first or next patch of the patches generated for the reference (previous) point cloud is selected. At **936**, the

67

points of the patch are re-sampled as described herein. At 937 motion vectors for the points included in the selected patch between the reference point cloud and the current point cloud are determined. At 940 the motion vectors are used to generate a relative motion patch image. For example, in some embodiments, generating a relative motion patch image may comprise, encoding, at 941, vector motion in different directions using different image characteristics, as described herein. At 938 an updated color patch image is generated. In some embodiments, the updated color patch image may encode residual values indicating differences in colors of the points of the point cloud included in the patch between the reference point cloud and the current point cloud. In a similar manner, at 939, other attribute update patch images may be generated.

At 942, it is determined whether there are additional patches to be evaluated. If so, the process reverts to 935 for the next patch. If not, at 943 the patch images for the patches are packed into one or more image frames. In some embodiments, patch images for respective patches may be packed before patch images are determined for other patches.

At 944, spaces in the one or more image frames that are not occupied by patch images are padded.

At 945, the one or more image frames are video encoded, such as in accordance with a high efficiency video coding (HEVC) standard.

FIG. 10 illustrates a decoder receiving image frames comprising patch images, patch information, and an occupancy map, and generating a decompressed representation of a point cloud, according to some embodiments.

At 1050, an occupancy map is received by a decoder, at 1051 patch information is received by the decoder. In some embodiments the occupancy map and the patch information may be encoded and the decoder may decode the occupancy map and the patch information (not shown). At 1052, the decoder receives one or more encoded video image frames. At 1052 the decoder identifies patch images in the one or more encoded video image frames and at 1054 the decoder decodes the encoded video image frames. In some embodiments, the decoder may utilize the occupancy map and the patch information to identify active and non-active portions of the one or more encoded video images and may adjust one or more decoded parameters used to decode the encoded video images based on whether portions, e.g. blocks, sub-blocks, pixels, etc. comprise active or non-active information.

At 1055, the decoder determines spatial information and/or attribute information for the points of the respective patches and at 1056 generates a decompressed representation of the point cloud encoded in the one or more encoded video images.

In some embodiments, active and non-active portions of an image frame may be indicated by a "mask." For example, a mask may indicate a portion of an image that is a padding portion or may indicate non-active points of a point cloud, such as points that are hidden from view in one or more viewing angles.

In some embodiments, a "mask" may be encoded along with patch images or projections. In some embodiments, a "mask" may show "active/available" points and "non-active/non-available" points in space. In some embodiments, a mask may be independent from a texture and a depth patch image. In some embodiments, a mask may be combined with other information, such as a texture or depth patch image. For example, by indicating that certain values in a signal range correspond to active points, e.g. values above 16 and below 235 in an 8 bit image, and that other values

68

correspond to non-active points, e.g. values below 16 or values above 235 in an 8 bit image. In some embodiments, additional considerations may be taken to avoid or reduce contamination between active and non-active regions. For example, it may be necessary to make use of lossless or visually lossless coding at the boundaries between active and non-active regions

In some embodiments, a mask may be utilized in a variety of ways for improving coding efficiency. For example, a mask may be used with projection methods such as cylindrical, spherical or multiple projection as wells as decomposition into patches. In addition, a mask may be used with a cubic projection method.

FIG. 11A illustrates an encoder, adjusting encoding based on one or more masks for a point cloud, according to some embodiments.

At 1162, an encoder receives a point cloud. At 1164, the encoder generate multiple projections or patch images as described herein, for the received point cloud. At 1166, the encoder determines or more masks. The masks may be hidden points, padded portions of an image frame, points not viewable from a particular view-point, etc. At 1168, the encoder adjusts one or more encoding parameters based on the masks. For example the encoder may adjust a budget allocated to masked portions. Additional adjustments that an encoder may perform are described. At 1168, the encoder encodes a compressed point cloud, for example via one or more video encoded image frames.

FIG. 11B illustrates a decoder, adjusting decoding based on one or more masks for a point cloud, according to some embodiments.

At 1170, a decoder receives an encoded point cloud. At 1172, the decoder determines one or more masks for portions of the encoded point cloud. For example, the encoder may determine portions of image frames representing the compressed point cloud correspond to padding. Or, for a particular view of the point cloud being rendered by the decoder, the decoder may determine that one or more points of the compressed point cloud are not viewable from the particular point of view. In some embodiments, mask information may indicate which points are hidden when the point cloud is viewed from particular points of view. At 1174, the decoder adjusts one or more decoding parameters based on the masks. Adjustments that may be made by a decoder based on active/non-active regions or points (e.g. masks) are described in more detail below. At 1176 the decoder decodes the compressed point cloud

In some embodiments, a mask may be used when performing motion estimation and mode decision. Commonly distortion is computed for an entire block. However, some blocks may have blocks that contain a combination of texture data as well as empty/nonvisible areas. For these areas only the textured data are of interest and any distortion in the non-visible areas may be ignored. Therefore, since commonly when performing such processes as motion estimation and mode decision, a distortion computation, such as Sum of Absolute Differences (SAD) or Sum of Square Errors (SSE), is performed, a mask may be used to alter the computation to exclude distortion for the non-visible areas. For example, for the SAD case, distortion may be computed by computing the sum of absolute differences of only samples in a block that correspond to a visible area in a current image. All other samples may be ignored during the computation. In some embodiments, distortion may be normalized at the pixel level thus avoiding having to consider blocks with different number of pixels.

In some embodiments, instead of only considering non-visible samples, samples that are adjacent to non-visible samples, or samples identified to correspond to different projections (but are placed when encoding within the same coding block) may be assigned different weights. For example samples in particular blocks could be considered more important for subjective quality, and a lower distortion tolerance may be assigned. In such case, the weighting for those samples may be increased, thus biasing decisions where the distortion for those samples is lower. Knowledge also that different samples in the same block of a particular size $M \times N$ during motion estimation or mode decision correspond to different objects, may also help with the determination of the block partitioning mode, e.g. the encoder could make an early decision (based potentially on a preliminary search) on whether different partitioning could/should be used.

In some embodiments, masks may be used for rate control and rate allocation. For example, it may be desirable that blocks that correspond to areas that contain both visible and non-visible samples be encoded at a different, and some times higher, quality than blocks that contain only visible samples. This is done so as to avoid leakage between visible and not visible samples and ensure the best quality at the point-clouds "boundaries". Different quality may also be assigned based on depth information, which may also be available on the encoder. Flatter areas may tolerate much more distortion than areas with considerable variance in depth. Control of quality may be performed by adjusting quantization parameters/factors, but also by adjusting other parameters such as the lagrangian multiplier during mode decision, using different quantization matrices if available, enabling and/or adjusting quantization thresholding and the size and/or shapes of zonal quantization.

Quantization may also be adjusted according to the projection method used. If, for example an equirectangular projection method was used to project the object onto a sphere and then onto a 2D plane, it might be desirable to increase quantization on the top and bottom boundaries, and slowly decrease it when moving towards the center/equator. This may help compensate for some of the differences in resolution allocation when using a particular projection method. Different adjustments may also be made to the different color components, again based on similar assumptions, and in consideration again of the mask information.

Quantization may also be performed while considering whether a sample is a visible or a non-visible sample. For example, if a strategy involves the use of dynamic programming/trellis quantization methods for determining the value of a quantized coefficient. In such embodiments, an impact in distortion of a quantized coefficient, as well as its impact on bitrate at multiple reconstruction points may commonly be computed. This may be done for all coefficients while considering their bitrate interactions. Finally a decision may be made for all coefficients jointly by selecting the quantized values that would together result in the best rate distortion performance. In some embodiments, the visible and non-visible areas may be considered when computing such metrics.

Similar to the motion estimation and mode decision processes, sample adaptive offset (SAO) techniques also commonly compute the resulting distortion for each possible mode or SAO value that may be used. Again, the decision may exclude non-visible samples, or prioritize, with different weights samples that are close to non-visible samples or samples that correspond to areas with considerably varying depth.

In some embodiments, masks may be used in any other coding process that may involve a distortion computation.

In some embodiments, masks may be used in preprocessing/prefiltering. For example, samples may be prefiltered based on their proximity to non-visible samples so as to reduce the possibility of artifacts and/or remove noise that may make encoding more difficult. Any form of prefiltering, including spatio-temporal filters, may be used.

In some embodiments, prefiltering may be applied to both texture as well as depth information. Decisions in quantization parameters could also be made at the picture level (temporally) given the amount of visible/non-visible samples and depth variance on different pictures. Such decisions could be quite useful, for example, in a multi-pass coding system where analyze the entire sequence is first analyzed to determine the complexity and relationship of each frame with other frames. The coding parameters may then be decided that will be used for that frame in relationship to all other frames and given an expected quality or bitrate target. Similar decisions may also be made, not only for quantization parameters, but also for the picture coding types (i.e. I, P, or B), structures (e.g. hierarchical or not coding of N frames that follows a particular coding order of frames), references to use, weighting parameters, etc. Encoding and Decoding (Normative Concepts)

Since a mask is likely to be available losslessly or visually losslessly at the decoder, as well as the depth information, this information may also be used at the decoder (and of course at the encoder) to further improve quality.

For example, deblocking and sample adaptive offset (SAO), as well as adaptive loop filtering (ALF) and deringing (in codecs that support such mechanisms), with exclusion of non-visible samples, samples that correspond to different projections, or samples with very different depth characteristics may use masking information. Instead, it may be desirable to only consider for such filtering methods samples that correspond to the same projection and are not so far from each other (depth wise). This may reduce blockiness and/or other artifacts that these methods try to mitigate. Other future types of in-loop post filtering may also be performed in a similar manner.

As another example, out of loop post filtering with visible/non-visible/different area segmentation may utilize masking information.

Implicit adjustment of QP quality parameters based on a certain percentage of visible/non-visible samples within a block may be performed. This may reduce signaling of coding parameters if such switching occurs frequently in a bit stream.

Adjustment of the transform type based on the percentage of visible/non-visible samples may be performed, including the consideration of shape adaptive discrete cosine transforms (DCT transforms).

Adjustment of overlapped block motion compensation techniques may utilize masking information, if existing in a codec, to mask away non-visible samples. A similar consideration may be performed for block motion compensation and/or intra prediction (including an intra block copy method). Samples that are considered visible may be considered when constructing a prediction signal, including also when interpolating to perform subpixel motion compensation or when performing bi-prediction. Masks from the current picture may be considered, but if desired, both the masks from the current picture as well as the masks corresponding to the reference pictures could be considered. Such considerations may be made adaptive at the encoder,

through some form of signaling, i.e. at the sequence, picture, tile, slice, or even CTU/block level

In some embodiments, clipping of the final value based on the mask or depth information may be performed.

In some embodiments, other prediction methods that may exist inside a codec (e.g. in AV1 or the Versatile Video Coding (VVC) standard currently being developed by the JVET team in MPEG) may be similarly adjusted or constrained based on the existence (and amount) of visible and non-visible points, and points corresponding to different projections.

In some embodiments, different control/adjustments may be applied to different color components as well as to the depth information.

Methods of Occupancy Map Compression Arithmetic Compression

FIG. 12A illustrates more detail regarding compression of an occupancy map, according to some embodiments. The steps shown in FIG. 12A may be performed as part of steps 821 or 933 as described above. In some embodiments, any of the occupancy map compression techniques described herein may be performed at 821 or 933.

At 1280 a list of candidate patches is determined for each block or modified block of an occupancy map.

At 1281, the lists of candidate patches for each block are ordered in a reverse order as an order in which the patches were packed into the image frame. For example, the patches may be packed into an image, with larger patches packed before smaller patches. In contrast, the candidate list for each block of an occupancy map may include smaller patches before larger patches. At 1282, an arithmetic encoder may be used to encode the patch candidate list for each block. In some embodiments, an entropy encoder may be used. Also, in some embodiments, empty blocks may be assigned a special value, such as zero, whereas patch candidates may be assigned numbers corresponding to a patch number, such as 1, 2, 3, etc.

At 1283, for each block sub-blocks are determined according to a determined precision value. The determined precision value may be encoded with the occupancy map such that a decoder may determine the determined precision value used at the encoder.

At 1284, for each block, a binary value (e.g. 0 or 1) is determined for each sub-block of the block. Full sub-blocks are assigned a different binary value than non-full sub-blocks. If all sub-blocks of a block are full, the block may be assigned a binary "full" value.

At 1285, for each non-full sub-block, a traversal order is determined. For example, any of the example traversal orders shown in FIG. 12B (or other traversal orders) may be determined. A run-length encoding strategy as described above in regard to occupancy map compression may be used to encode the binary values for the sub-blocks using the determined traversal order.

FIG. 12B illustrates example blocks and traversal patterns for compressing an occupancy map, according to some embodiments. The traversal patterns may be used as described above in regard to occupancy map compression and in FIG. 12A. For example, FIG. 12B illustrates block 1286 of an occupancy map that includes multiple sub-blocks 1287.

Video-Based Compression

FIG. 12C illustrates more detail regarding compression of an occupancy map, according to some embodiments. The steps shown in FIG. 12C may be performed as part of steps 821 or 933 as described above. In some embodiments, any

of the occupancy map compression techniques described herein may be performed at 821 or 933.

At 1290, for each patch image frame, such as image frames comprising depth information patches or attribute value patches, a corresponding occupancy map image frame is generated. The occupancy map image frame may have a same frame size as the patch image frame and when placed over the patch image frame the pixels of the occupancy map image frame may match up with corresponding pixels in the patch image frame. Because of this, a binary value may be assigned to each of the pixels of the occupancy map image frames to indicate whether the same corresponding pixel in the patch image frame is an occupied pixel or a padded pixel. In some embodiments, an occupancy map image frame may be smaller than its corresponding patch image frame. In such embodiments, different values may be assigned to the occupancy map image frame pixel to indicate whether its corresponding pixels in the patch image frame are occupied or padded. For example, a single occupancy map image frame pixel may correspond to multiple patch image frame pixels, wherein a first value indicates a first combination of occupied and unoccupied pixels in the patch image frame and another value indicates a different combination. As an example, say a single occupancy map pixel corresponds to two patch image map pixels. In such an embodiment, a first value in the occupancy map for the occupancy map pixel may indicate that both the patch image pixels are unoccupied, a second value may indicate that both are occupied, a third value may indicate that an upper one of the two pixels is occupied and the other is un-occupied and a fourth value may indicate that a lower one of the two pixels is occupied and the other is un-occupied. For example, each block of the occupancy map may be a multiple larger than a block of the patch image frame (e.g. the occupancy map may have a lower resolution than the patch image frame). Different color values (e.g. the 1st, 2nd, 3rd, 4th value, etc.) may represent sub-divisions of the occupancy map that correspond with multiple pixels of the patch image frame.

At 1292 a list of candidate patches is determined for each block of the patch image frames.

At 1294, the lists of candidate patches for each block are ordered in a reverse order as an order in which the patches were packed into the image frame. For example, the patches may be packed into an image, with larger patches packed before smaller patches. In contrast, the candidate list for each block of the patch image frame may include smaller patches before larger patches.

At 1296, the occupancy map image frames are video encoded.

At 1298, an arithmetic encoder may be used to encode auxiliary information, such as the patch candidate list for each block. In some embodiments, an entropy encoder may be used.

Example Applications Using Point Cloud Encoders and Decoder

FIG. 13 illustrates compressed point clouds being used in a 3-D telepresence application, according to some embodiments.

In some embodiments, a sensor, such as sensor 102, an encoder, such as encoder 104 or any of the other encoders described herein, and a decoder, such as decoder 116 or any of the decoders described herein, may be used to communicate point clouds in a 3-D telepresence application. For example, a sensor, such as sensor 102, at 1302 may capture a 3D image and at 1304, the sensor or a processor associated with the sensor may perform a 3D reconstruction based on sensed data to generate a point cloud.

At **1306**, an encoder such as encoder **104** may compress the point cloud and at **1308** the encoder or a post processor may packetize and transmit the compressed point cloud, via a network **1310**. At **1312**, the packets may be received at a destination location that includes a decoder, such as decoder **116**. The decoder may decompress the point cloud at **1314** and the decompressed point cloud may be rendered at **1316**. In some embodiments a 3-D telepresence application may transmit point cloud data in real time such that a display at **1316** represents images being observed at **1302**. For example, a camera in a canyon may allow a remote user to experience walking through a virtual canyon at **1316**.

FIG. **14** illustrates compressed point clouds being used in a virtual reality (VR) or augmented reality (AR) application, according to some embodiments.

In some embodiments, point clouds may be generated in software (for example as opposed to being captured by a sensor). For example, at **1402** virtual reality or augmented reality content is produced. The virtual reality or augmented reality content may include point cloud data and non-point cloud data. For example, a non-point cloud character may traverse a landscape represented by point clouds, as one example. At **1404**, the point cloud data may be compressed and at **1406** the compressed point cloud data and non-point cloud data may be packetized and transmitted via a network **1408**. For example, the virtual reality or augmented reality content produced at **1402** may be produced at a remote server and communicated to a VR or AR content consumer via network **1408**. At **1410**, the packets may be received and synchronized at the VR or AR consumer's device. A decoder operating at the VR or AR consumer's device may decompress the compressed point cloud at **1412** and the point cloud and non-point cloud data may be rendered in real time, for example in a head mounted display of the VR or AR consumer's device. In some embodiments, point cloud data may be generated, compressed, decompressed, and rendered responsive to the VR or AR consumer manipulating the head mounted display to look in different directions.

In some embodiments, point cloud compression as described herein may be used in various other applications, such as geographic information systems, sports replay broadcasting, museum displays, autonomous navigation, etc.

Example Computer System

FIG. **15** illustrates an example computer system **1500** that may implement an encoder or decoder or any other ones of the components described herein, (e.g., any of the components described above with reference to FIGS. **1-14**), in accordance with some embodiments. The computer system **1500** may be configured to execute any or all of the embodiments described above. In different embodiments, computer system **1500** may be any of various types of devices, including, but not limited to, a personal computer system, desktop computer, laptop, notebook, tablet, slate, pad, or netbook computer, mainframe computer system, handheld computer, workstation, network computer, a camera, a set top box, a mobile device, a consumer device, video game console, handheld video game device, application server, storage device, a television, a video recording device, a peripheral device such as a switch, modem, router, or in general any type of computing or electronic device.

Various embodiments of a point cloud encoder or decoder, as described herein may be executed in one or more computer systems **1500**, which may interact with various other devices. Note that any component, action, or functionality described above with respect to FIGS. **1-14** may be implemented on one or more computers configured as computer

system **1500** of FIG. **15**, according to various embodiments. In the illustrated embodiment, computer system **1500** includes one or more processors **1510** coupled to a system memory **1520** via an input/output (I/O) interface **1530**. Computer system **1500** further includes a network interface **1540** coupled to I/O interface **1530**, and one or more input/output devices **1550**, such as cursor control device **1560**, keyboard **1570**, and display(s) **1580**. In some cases, it is contemplated that embodiments may be implemented using a single instance of computer system **1500**, while in other embodiments multiple such systems, or multiple nodes making up computer system **1500**, may be configured to host different portions or instances of embodiments. For example, in one embodiment some elements may be implemented via one or more nodes of computer system **1500** that are distinct from those nodes implementing other elements.

In various embodiments, computer system **1500** may be a uniprocessor system including one processor **1510**, or a multiprocessor system including several processors **1510** (e.g., two, four, eight, or another suitable number). Processors **1510** may be any suitable processor capable of executing instructions. For example, in various embodiments processors **1510** may be general-purpose or embedded processors implementing any of a variety of instruction set architectures (ISAs), such as the x86, PowerPC, SPARC, or MIPS ISAs, or any other suitable ISA. In multiprocessor systems, each of processors **1510** may commonly, but not necessarily, implement the same ISA.

System memory **1520** may be configured to store point cloud compression or point cloud decompression program instructions **1522** and/or sensor data accessible by processor **1510**. In various embodiments, system memory **1520** may be implemented using any suitable memory technology, such as static random access memory (SRAM), synchronous dynamic RAM (SDRAM), nonvolatile/Flash-type memory, or any other type of memory. In the illustrated embodiment, program instructions **1522** may be configured to implement an image sensor control application incorporating any of the functionality described above. In some embodiments, program instructions and/or data may be received, sent or stored upon different types of computer-accessible media or on similar media separate from system memory **1520** or computer system **1500**. While computer system **1500** is described as implementing the functionality of functional blocks of previous Figures, any of the functionality described herein may be implemented via such a computer system.

In one embodiment, I/O interface **1530** may be configured to coordinate I/O traffic between processor **1510**, system memory **1520**, and any peripheral devices in the device, including network interface **1540** or other peripheral interfaces, such as input/output devices **1550**. In some embodiments, I/O interface **1530** may perform any necessary protocol, timing or other data transformations to convert data signals from one component (e.g., system memory **1520**) into a format suitable for use by another component (e.g., processor **1510**). In some embodiments, I/O interface **1530** may include support for devices attached through various types of peripheral buses, such as a variant of the Peripheral Component Interconnect (PCI) bus standard or the Universal Serial Bus (USB) standard, for example. In some embodiments, the function of I/O interface **1530** may be split into two or more separate components, such as a north bridge and a south bridge, for example. Also, in some embodiments some or all of the functionality of I/O interface **1530**, such as an interface to system memory **1520**, may be incorporated directly into processor **1510**.

75

Network interface **1540** may be configured to allow data to be exchanged between computer system **1500** and other devices attached to a network **1585** (e.g., carrier or agent devices) or between nodes of computer system **1500**. Network **1585** may in various embodiments include one or more networks including but not limited to Local Area Networks (LANs) (e.g., an Ethernet or corporate network), Wide Area Networks (WANs) (e.g., the Internet), wireless data networks, some other electronic data network, or some combination thereof. In various embodiments, network interface **1540** may support communication via wired or wireless general data networks, such as any suitable type of Ethernet network, for example; via telecommunications/telephony networks such as analog voice networks or digital fiber communications networks; via storage area networks such as Fibre Channel SANs, or via any other suitable type of network and/or protocol.

Input/output devices **1550** may, in some embodiments, include one or more display terminals, keyboards, keypads, touchpads, scanning devices, voice or optical recognition devices, or any other devices suitable for entering or accessing data by one or more computer systems **1500**. Multiple input/output devices **1550** may be present in computer system **1500** or may be distributed on various nodes of computer system **1500**. In some embodiments, similar input/output devices may be separate from computer system **1500** and may interact with one or more nodes of computer system **1500** through a wired or wireless connection, such as over network interface **1540**.

As shown in FIG. **15**, memory **1520** may include program instructions **1522**, which may be processor-executable to implement any element or action described above. In one embodiment, the program instructions may implement the methods described above. In other embodiments, different elements and data may be included. Note that data may include any data or information described above.

Those skilled in the art will appreciate that computer system **1500** is merely illustrative and is not intended to limit the scope of embodiments. In particular, the computer system and devices may include any combination of hardware or software that can perform the indicated functions, including computers, network devices, Internet appliances, PDAs, wireless phones, pagers, etc. Computer system **1500** may also be connected to other devices that are not illustrated, or instead may operate as a stand-alone system. In addition, the functionality provided by the illustrated components may in some embodiments be combined in fewer components or distributed in additional components. Similarly, in some embodiments, the functionality of some of the illustrated components may not be provided and/or other additional functionality may be available.

Those skilled in the art will also appreciate that, while various items are illustrated as being stored in memory or on storage while being used, these items or portions of them may be transferred between memory and other storage devices for purposes of memory management and data integrity. Alternatively, in other embodiments some or all of the software components may execute in memory on another device and communicate with the illustrated computer system via inter-computer communication. Some or all of the system components or data structures may also be stored (e.g., as instructions or structured data) on a computer-accessible medium or a portable article to be read by an appropriate drive, various examples of which are described above. In some embodiments, instructions stored on a computer-accessible medium separate from computer system **1500** may be transmitted to computer system **1500** via

76

transmission media or signals such as electrical, electromagnetic, or digital signals, conveyed via a communication medium such as a network and/or a wireless link. Various embodiments may further include receiving, sending or storing instructions and/or data implemented in accordance with the foregoing description upon a computer-accessible medium. Generally speaking, a computer-accessible medium may include a non-transitory, computer-readable storage medium or memory medium such as magnetic or optical media, e.g., disk or DVD/CD-ROM, volatile or non-volatile media such as RAM (e.g. SDRAM, DDR, RDRAM, SRAM, etc.), ROM, etc. In some embodiments, a computer-accessible medium may include transmission media or signals such as electrical, electromagnetic, or digital signals, conveyed via a communication medium such as network and/or a wireless link.

The methods described herein may be implemented in software, hardware, or a combination thereof, in different embodiments. In addition, the order of the blocks of the methods may be changed, and various elements may be added, reordered, combined, omitted, modified, etc. Various modifications and changes may be made as would be obvious to a person skilled in the art having the benefit of this disclosure. The various embodiments described herein are meant to be illustrative and not limiting. Many variations, modifications, additions, and improvements are possible. Accordingly, plural instances may be provided for components described herein as a single instance. Boundaries between various components, operations and data stores are somewhat arbitrary, and particular operations are illustrated in the context of specific illustrative configurations. Other allocations of functionality are envisioned and may fall within the scope of claims that follow. Finally, structures and functionality presented as discrete components in the example configurations may be implemented as a combined structure or component. These and other variations, modifications, additions, and improvements may fall within the scope of embodiments as defined in the claims that follow.

What is claimed is:

1. A non-transitory, computer-readable, storage medium storing program instructions, that when executed using one or more processors, cause the one or more processors to:
 - receive three-dimensional (3D) content to be compressed;
 - generate, for the three-dimensional content:
 - attribute image frames comprising attribute patches for the 3D content packed into respective two-dimensional (2D) frames;
 - geometry image frames comprising geometry patches for the 3D content packed into respective 2D frames;
 - and
 - encode the generated attribute image frames and the generated geometry image frames,
- wherein:
 - at least some coding parameters used to encode the attribute image frames are different than at least some coding parameter used to encode the geometry image frames, wherein the differing coding parameters comprise one or more of:
 - differing bit-depths used to encode the attribute image frames and the geometry image frames; or
 - differing sizing parameters used to encode the attribute image frames and the geometry image frames; and
 - an encoded bitstream, resulting from the encoding, comprises one or more network abstraction layer (NAL) units that indicate a first one of the differing

77

coding parameters to be used to decode the attribute image frames and a second one of the differing coding parameters to be used to decode the geometry image frames.

2. The non-transitory, computer-readable, storage medium of claim 1, wherein the one or more NAL units indicate:

a first frame height or frame width for a portion of a given one of the 2D frames that corresponds to one or more of the attribute image frames; and

a second frame height or frame width for a portion of a given one of the 2D frames that corresponds to one or more of the geometry image frames.

3. The non-transitory, computer-readable, storage medium of claim 1, wherein the one or more NAL units indicate:

a resizing parameter that indicates a re-sizing between a video frame size and an image frame size of one or more of the attribute image frames or one or more of the geometry image frames.

4. The non-transitory, computer-readable, storage medium of claim 1, wherein the one or more NAL units indicate:

a color format to be used for decoding a respective set of image frames corresponding to one or more of the attribute image frames or one or more of the geometry image frames.

5. The non-transitory, computer-readable, storage medium of claim 1, wherein the one or more NAL units indicate:

a first video bit-depth of geometry values encoded for patch images of one or more of the geometry image frames;

a first corresponding reconstructed bit-depth for the geometry values to be used to reconstruct the 3D content;

a second video bit-depth of attribute values encoded for patch images of one or more of the attribute image frames; and

a second corresponding reconstructed bit-depth for the attribute values to be used to reconstruct the 3D content.

6. The non-transitory, computer-readable, storage medium of claim 1, wherein the one or more NAL units indicate:

an internal bit-depth used to determine reconstructed attribute values or reconstructed geometry values based on attribute values or geometry values included in patch images of one or more of the attribute image frames or one or more of the geometry image frames.

7. The non-transitory, computer-readable, storage medium of claim 1, wherein the one or more NAL units indicate:

sequence parameter set (SPS) information for a sequence of the video encoded attribute and/or geometry image frames; or

frame parameter set (FPS) information for individual ones of the video encoded attribute or geometry image frames.

8. The non-transitory, computer-readable, storage medium of claim 7, wherein:

the encoded bitstream comprises one or more NAL units indicating sequence parameter set (SPS) information and one or more NAL units indicating frame parameter set (FPS) information; and

the FPS information indicates one or more parameters, to be applied for a given attribute or geometry image

78

frame, that differ from one or more parameters specified in the sequence parameter set (SPS) information.

9. The non-transitory, computer-readable, storage medium of claim 7, wherein the encoded bitstream comprises:

a first NAL unit indicating a sequence parameter set to be applied to a first set of image frames;

a set of the geometry image frames, included in the first set, corresponding to the first NAL unit indicating the sequence parameter set;

a set of the attribute image frames, included in the first set, corresponding to the first NAL unit indicating the sequence parameter set; and

a set of occupancy image frames, included in the first set, corresponding to the first NAL unit indicating the sequence parameter set.

10. The non-transitory, computer-readable, storage medium of claim 9, wherein, the video image frames of the respective sets of the first set (geometry, attribute, and occupancy) are signaled together.

11. The non-transitory, computer-readable, storage medium of claim 9, wherein, the video image frames of the respective sets of the first set (geometry, attribute, and occupancy) are interleaved.

12. A non-transitory, computer-readable, storage medium storing program instructions, that when executed using one or more processors, cause the one or more processors to: receive a bit stream comprising:

attribute image frames comprising attribute patches for three-dimensional (3D) content packed into respective two-dimensional (2D) frames;

geometry image frames comprising geometry patches for the 3D content packed into respective 2D frames; and

one or more network abstraction layer (NAL) units that indicate differing coding parameters to be used to decode the attribute image frames and the geometry image frames, wherein the differing coding parameters comprise a first set of coding parameter to be used to decode the attribute image frames and a second set of coding parameters to be used to decode the geometry image frames, wherein the differing coding parameters comprise one or more of:

differing bit-depths used to encode the attribute image frames and the geometry image frames; or differing sizing parameters used to encode the attribute image frames and the geometry image frames;

decode the attribute image frames using the first set of coding parameters;

decode the geometry image frames using the second set of coding parameters, wherein the first set and the second set of coding parameters are determined based on the one or more NAL units, and wherein at least one coding parameter differs between the first set and the second set of coding parameters; and

reconstruct three-dimensional content using attribute information and geometry information from the decoded attribute image frame and the decoded geometry image frames.

13. The non-transitory, computer-readable, storage medium of claim 12, wherein the one or more NAL units indicate:

a frame height or frame width for a portion of a given one of the 2D frames that corresponds to one or more of the attribute image frames or one or more of the geometry image frames.

79

14. The non-transitory, computer-readable, storage medium of claim 12, wherein the one or more NAL units indicate:

a resizing parameter that indicates a re-sizing between a video frame size and an image frame size of one or more of the attribute image frames or one or more of the geometry image frames.

15. The non-transitory, computer-readable, storage medium of claim 12, wherein the one or more NAL units indicate:

a video bit-depth of geometry values or attribute values encoded for patch images of one or more of the attribute image frames or one or more of the geometry image frames; and

a corresponding reconstructed bit-depth for the geometry values or attribute values to be used to reconstruct the 3D content.

16. The non-transitory, computer-readable, storage medium of claim 12, wherein the received bitstream comprises:

a first NAL unit indicating a sequence parameter set to be applied to a first set of image frames;

a set of the geometry image frames, included in the first set, corresponding to the first NAL unit indicating the sequence parameter set;

a set of the attribute image frames, included in the first set, corresponding to the first NAL unit indicating the sequence parameter set; and

a set of occupancy image frames, included in the first set, corresponding to the first NAL unit indicating the sequence parameter set.

17. The non-transitory, computer-readable, storage medium of claim 16, wherein, the video image frames of the respective sets of the first set (geometry, attribute, and occupancy) are interleaved.

18. A device, comprising:

a memory storing program instructions; and

one or more processors, wherein the program instructions, when executed using the one or more processors, cause the one or more processors to:

receive a bit stream comprising:

attribute image frames comprising attribute patches for three-dimensional (3D) content packed into respective two-dimensional (2D) frames;

geometry image frames comprising geometry patches for the 3D content packed into respective 2D frames; and

one or more network abstraction layer (NAL) units that indicate differing coding parameters to be used to decode the attribute image frames and the geometry image frames, wherein the differing coding parameters comprise a first set of coding parameter to be used to decode the attribute image frames and a

80

second set of coding parameters to be used to decode the geometry image frames, wherein the differing coding parameters comprise one or more of:

differing bit-depths used to encode the attribute image frames and the geometry image frames; or
differing sizing parameters used to encode the attribute image frames and the geometry image frames;

decode the attribute image frames using the first set of coding parameters;

decode the geometry image frames using the second set of coding parameters, wherein the first set and the second set of coding parameters are determined based on the one or more NAL units, and wherein at least one coding parameter differs between the first set and the second set of coding parameters; and

reconstruct three-dimensional content using attribute information and geometry information from the decoded attribute image frame and the decoded geometry image frames.

19. The device of claim 18, wherein the one or more NAL units indicate one or more of:

a frame height or frame width for a portion of a given one of the 2D frames that corresponds to one or more of the attribute image frames or one or more of the geometry image frames;

a resizing parameter that indicates a re-sizing between a video frame size and an image frame size of one or more of the attribute image frames or one or more of the geometry image frames;

a color format to be used for decoding a respective set of image frames corresponding to one or more of the attribute image frames or one or more of the geometry image frames;

a video bit-depth of geometry values or attribute values encoded for patch images of one or more of the attribute image frames or one or more of the geometry image frames and a corresponding reconstructed bit-depth for the geometry values or attribute values to be used to reconstruct the 3D content; or

an internal bit-depth used to determine reconstructed attribute values or reconstructed geometry values based on attribute values or geometry values included in patch images of one or more of the attribute image frames or one or more of the geometry image frames.

20. The device of claim 18, further comprising:

a display,

wherein the program instructions, when executed using the one or more processors, further cause the one or more processors to:

render the reconstructed 3D content for viewing on the display.

* * * * *